

PROJET DE FIN D'ÉTUDES

École Nationale Supérieure Polytechnique de Yaoundé
Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique
Année académique 2025–2026

Dossier de remise

Prise en main du projet et méthode de conception avec Claude

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Contenu du dossier

Ce dossier réunit tout ce qu'il faut pour **recupérer**, **lancer** et **tester** le projet, puis pour **comprendre** comment il a été conçu avec Claude et **tester la méthode** sur un exemple.

Chaque document a sa propre page de garde. La colonne « Page » indique la page de ce fichier où commence le document.

N°	Document	Rôle	Page
1	README : récupérer, lancer et tester le système	Partie 1 : vérifier que le projet fonctionne	3
2	Guide 1 : les outils Claude	Partie 2 : comprendre les outils	33
3	Guide 2 : la méthode de conception	Partie 2 : comprendre la méthode	54
4	Guide 3 : conception du MVP (test en direct)	Partie 3 : tester la méthode sur un exemple	74
5	Fiche de démonstration	Le déroulé de la séance	92

Par où commencer

1. **Lisez la fiche de démonstration** : elle décrit la séance en trois parties.
2. **Partie 1** : suivez le README pour récupérer le projet depuis GitHub, le lancer et le tester.
3. **Partie 2** : lisez les guides 1 et 2, qui expliquent les outils et la méthode.
4. **Partie 3** : le guide 3 contient le document de conception à confier à Claude Code.

À part

Le **guide 4** (autres fonctions de Claude : fichiers Excel, Word, PowerPoint, Skills, connecteurs, PDF à partir de Markdown) est un document séparé : 06_Guide4_Autres_fonctions_Claude.pdf.

Les sources

Chaque document existe en trois formats : Markdown (docs/guides/, la source), LaTeX (docs/tex/, pour ajouter des images ou changer la mise en page) et PDF (docs/pdf/). docs/build/build_all.sh refait les .tex et les PDF.

PROJET DE FIN D'ÉTUDES

École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique

Année académique 2025–2026

Guide de prise en main

Récupérer, lancer et tester le système de scoring de crédit

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Table des matières

CREDIX : système intelligent de scoring du risque de crédit	1
Comment lire ce guide	1
1. Présentation du projet	1
2. Architecture : comment le projet est organisé	6
3. Ce qu'il faut installer avant de commencer	7
4. Récupérer le projet depuis GitHub	9
5. Lancer le backend	10
6. Lancer le frontend	17
7. Tester l'application de bout en bout	19
8. Arrêter, relancer, tout effacer	22
9. Modifier le projet et le remettre à jour avec Git	23
10. Dépannage	25
11. Annexes	27

CREDIX : système intelligent de scoring du risque de crédit

Guide complet : comprendre, récupérer, lancer, tester et modifier le projet.

Projet de fin d'études (ENSPY, Université de Yaoundé I). Dépôt GitHub : <https://github.com/HendrixPenka2/credix>

Comment lire ce guide

- Si vous voulez seulement lancer le projet, allez directement aux sections 3 à 7, dans l'ordre. Chaque étape suppose que la précédente a réussi.
- Les blocs gris sont des **commandes à copier-coller** dans un terminal, ou des **résultats** que vous devez voir à l'écran. Le texte « Résultat attendu » précède toujours ce que vous devez obtenir.
- Pour chaque commande, le guide dit **ce qu'elle fait**, **ce qu'il faut voir**, et **quoi faire si ce n'est pas le cas**.
- Ce guide est écrit et testé pour **Linux**. Windows (avec WSL 2 et Docker Desktop) et macOS peuvent fonctionner, mais n'ont pas été testés.
- Un **terminal** est la fenêtre où l'on tape des commandes (sur Ubuntu : touches Ctrl + Alt + T). Les commandes de base sont rappelées dans les annexes.

1. Présentation du projet

1.1 Qu'est-ce que CREDIX ?

Quand une banque ou une microfinance accorde un crédit, elle doit estimer le risque que le client ne rembourse pas : c'est le **risque de défaut**. **CREDIX** est une application web qui aide l'agent de crédit à prendre cette décision. Elle :

- calcule la **probabilité de défaut** du client avec un modèle d'apprentissage automatique, puis la convertit en un **score de 300 à 850** ;
- **explique** chaque score avec des phrases simples (quels facteurs ont pesé, dans quel sens) ;
- mesure la **quantité d'information disponible** sur le client (l'indice de couverture ρ_C) ;
- repère les **profils atypiques**, c'est-à-dire ceux qui ne ressemblent pas aux clients sur lesquels le modèle a appris ;
- décide seule des dossiers clairs, et **envoie à un superviseur humain** les dossiers douteux ;

- garde une **trace complète** de chaque action (journal d'audit) et surveille la **dérive** du modèle dans le temps.

1.2 Dans quel contexte a-t-il été réalisé ?

- CREDIX est le projet du **mémoire de fin d'études** de SINGHE PENKA Hendrix Donavan, pour le Diplôme d'Ingénieur de Conception en Génie Informatique (École Nationale Supérieure Polytechnique de Yaoundé, Université de Yaoundé I), année académique 2025-2026. Le mémoire s'intitule *Conception et développement d'un système intelligent de scoring du risque de défaut pour l'aide à la décision de crédit*. Il a été supervisé par le Pr Bernabé Batchakui (ENSPY) et Mme Abjean (IT Nearshore).
- Le travail a été conduit au sein d'une société de services numériques qui conçoit des solutions pour des institutions financières, **sans accès aux données d'un établissement réel**. Le système a donc été construit et évalué sur un corpus public, **Home Credit Default Risk** (compétition Kaggle) : 307 511 demandes de crédit, dont 24 825 défauts (8,07 %).
- Le cadre réglementaire (Comité de Bâle, règles sur les décisions automatisées) impose qu'une décision de crédit soit **justifiable, traçable et reprenable par un humain**. Tout le système est conçu pour cela.

1.3 Ce que le mémoire a établi (résultats principaux)

Constat	Valeur
Capacité du modèle à séparer bons et mauvais payeurs (AUC, jeu de test de 46 128 demandes)	0,751
Probabilité moyenne annoncée avant calibration, après calibration, et taux de défaut réellement observé	43,5 % puis 9,3 %, pour 10,1 % observé
Dossiers accordés automatiquement, et taux de défaut sur cette part	67,3 % des dossiers, avec 5,1 % de défaut (la moitié de celui du portefeuille)

1.4 Les trois acteurs de l'application

Trois profils se connectent, chacun avec ses propres écrans. Ce ne sont pas des droits différents sur les mêmes pages : ce sont trois espaces de travail distincts.

Acteur	Son rôle	Ce qu'il fait dans l'application
Agent (agent de crédit en agence)	Instruit les demandes	Recherche ou crée un client, lance un scoring , fait des simulations « et si... », consulte l'historique d'un client, télécharge le rapport PDF d'une décision
Superviseur (superviseur risque)	Tranche les cas douteux, surveille le portefeuille	Traite la file des dossiers en revue manuelle (accorde ou refuse avec une justification obligatoire), suit la répartition des scores, la santé du modèle (dérive) et ses versions

Acteur	Son rôle	Ce qu'il fait dans l'application
Administrateur	Gère la plateforme	Crée les comptes (agents, superviseurs), règle les seuils de décision, charge et promeut les versions du modèle, consulte le journal d'audit

Page principale de l'espace Agent

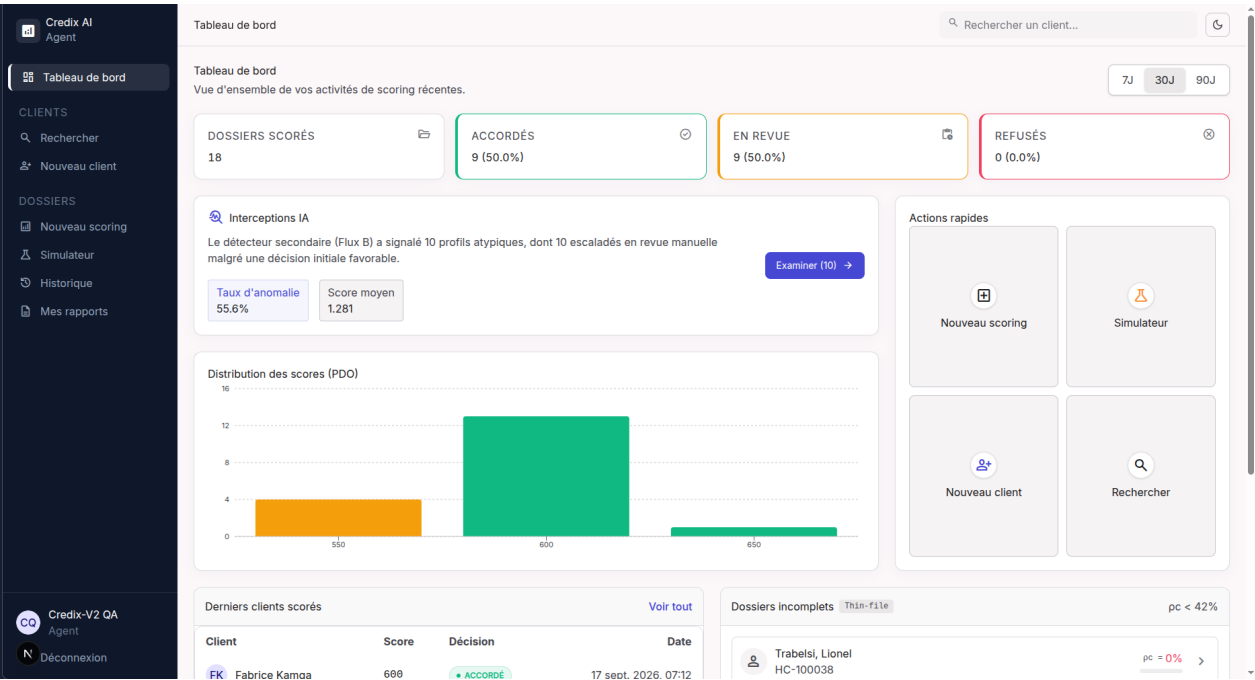


FIGURE 1 – Espace Agent : tableau de bord.

Page principale de l'espace Superviseur

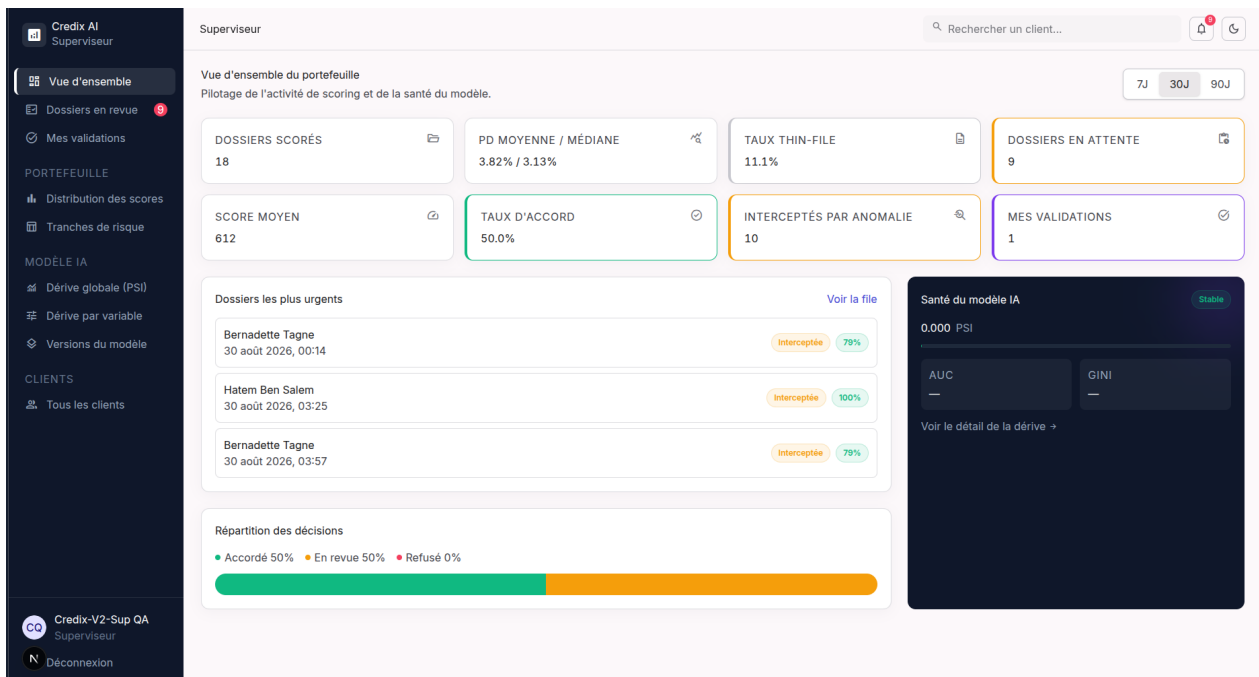


FIGURE 2 – Espace Superviseur : vue d'ensemble.

Page principale de l'espace Administrateur

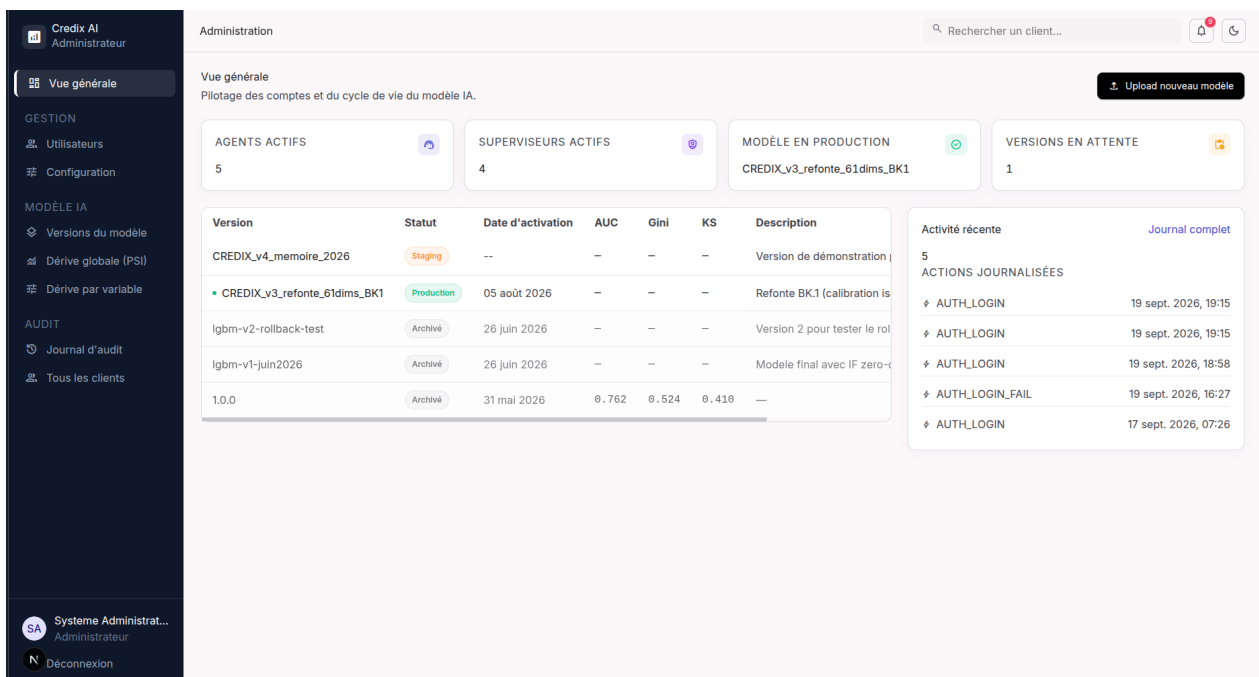


FIGURE 3 – Espace Administrateur : vue générale.

La liste détaillée de tous les écrans se trouve dans `credix-v2/FONCTIONNALITES.md`.

1.5 Comment le système décide (les règles en clair)

Le système travaille en **deux flux** qui se contrôlent l'un l'autre :

- **Flux A (le score).** Un modèle (LightGBM) estime la probabilité de défaut (PD). Elle est corrigée par calibration, puis convertie en score : $\text{Score} = 515,06 - 28,85 \times \ln(\text{PD} / (1 - \text{PD}))$.
- **Flux B (le contrôle).** Un second modèle (un autoencodeur) mesure si le profil du client est **atypique**. Il ne juge pas le risque : il repère les dossiers sur lesquels le système ne devrait pas se prononcer seul.

Règle	Seuil	Conséquence
PD inférieure à 10 % (score au moins égal à 578,5)	Décision initiale ACCORDÉ	Accord automatique
PD supérieure à 30 % (score inférieur à 539,5)	Décision initiale REFUSÉ	Refus automatique
PD entre les deux	Décision REVUE MANUELLE	Un superviseur tranche
Profil atypique sur un dossier accordé	Seuil : percentile 95 (réglable par l'administrateur)	Le dossier passe en revue manuelle . Un dossier refusé, lui, reste refusé.
Couverture ρ_c inférieure à 0,42	Information limitée	Bandeau « confiance limitée » et recommandation de documents à demander au client
Couverture ρ_c inférieure à 0,25	Information très insuffisante	Revue manuelle forcée , et le détecteur d'anomalies est désactivé

Le ρ_c n'est calculé qu'au moment du **scoring** : avant, un client affiche une couverture de 0, c'est normal.

1.6 Les outils utilisés

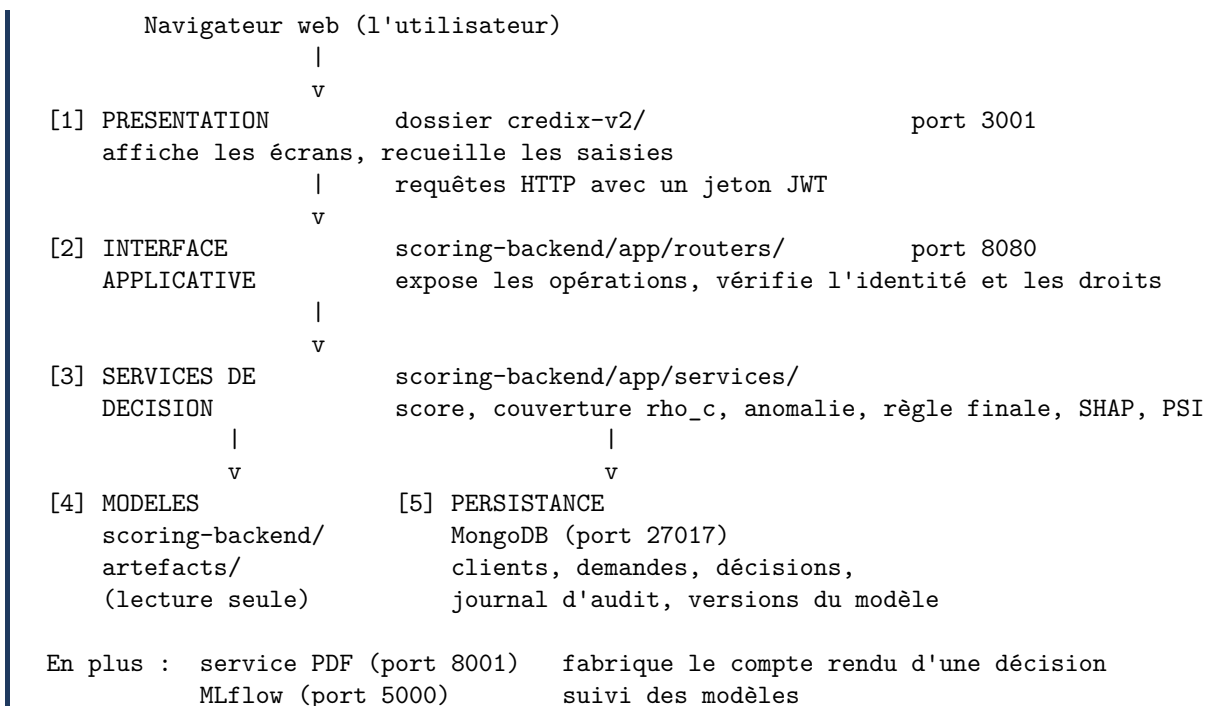
Domaine	Outils
Backend (le « cerveau »)	Python 3.12, FastAPI 0.111, Uvicorn, Pydantic 2
Modèle et calculs	LightGBM 4.3, scikit-learn 1.6, optbinning 0.21 (transformation WoE), SHAP 0.45 (explications), TensorFlow CPU (autoencodeur), pandas, NumPy
Base de données	MongoDB 7 (avec GridFS pour les fichiers du modèle)
Sécurité	Jetons JWT, mots de passe hachés avec bcrypt
Rapports PDF	WeasyPrint, dans un petit service séparé
Suivi des modèles	MLflow 2.12 (démarré avec le reste ; l'interface n'en dépend pas)
Frontend (l'interface web)	Next.js 15, React 19, TypeScript, Tailwind CSS 4, Radix UI, Recharts, react-hook-form et zod, axios
Lancement	Docker et Docker Compose
Conception	Maquettes Google Stitch ; schémas draw.io et PlantUML (dossier docs/diagrammes/)

Domaine	Outils
Assistance à la conception et au développement	Claude (claude.ai en conversation, puis Claude Code dans VS Code). Le détail de la méthode fait l'objet de guides séparés, dans docs/guides/.

2. Architecture : comment le projet est organisé

2.1 Les cinq couches

Le système est organisé en **cinq couches**, chacune avec une seule responsabilité. Une couche ne dépend jamais d'une couche au-dessus d'elle.



2.2 Les services qui tournent, et leurs ports

Service	Rôle	Adresse sur votre ordinateur
Frontend (Next.js)	L'interface web	http://localhost:3001
API (FastAPI), service api	Le backend	http://localhost:8080 (documentation interactive : http://localhost:8080/docs)
MongoDB, service mongodb	La base de données	localhost:27017
Service PDF, service pdf-worker	Fabrique les rapports PDF	http://localhost:8001
MLflow, service mlflow	Suivi des modèles	http://localhost:5000

Le backend et ses trois services annexes tournent dans des **conteneurs Docker**. Le frontend tourne directement sur votre machine, avec Node.js.

2.3 L'arborescence des dossiers

```

credix/
|-- README.md          dossier créé par « git clone »
                        ce document
|-- scoring-backend/   LE BACKEND
|   |-- app/           le code de l'API
|   |   |-- main.py    démarrage, chargement du modèle, route /health
|   |   |-- routers/   les routes : auth, scoring, clients, decisions,
|   |   |               dashboard, monitoring, admin, upload_model
|   |   |-- services/  la logique de décision : pipeline (WoE et LightGBM),
|   |   |               calibration, score PDO, couverture rho_c, SHAP, PSI,
|   |   |               percentiles, client du service PDF
|   |   |-- schemas/   la forme des données échangées
|   |   |-- core/      configuration (config.py) et sécurité (JWT, droits)
|   |   |-- db/        accès à MongoDB et à GridFS
|   |-- artefacts/     les 16 fichiers du modèle entraîné (3,5 Mo),
|   |                 dont 14 chargés par l'API au démarrage
|   |-- demo_data/     phrases d'explication et clients de démonstration
|   |-- scripts/       scripts de premier lancement (voir section 5)
|   |-- adapters/, configs/ lecture des données Home Credit (rechargement
↪   complet)
|   |-- pdf-worker/    le petit service qui fabrique les PDF
|   |-- mock/, tests/  fichiers factices et tests
|   |-- contexte claud code/ notes de travail avec Claude Code
|   |-- docker-compose.yml la description des 4 conteneurs
|   |-- Dockerfile.api  la recette de fabrication de l'image de l'API
|   |-- requirements.txt les bibliothèques Python
|   |-- .env.example    le modèle du fichier de configuration
|   |-- README.md       documentation technique détaillée du backend
|-- credix-v2/         LE FRONTEND
|   |-- app/           les pages (un dossier par écran)
|   |-- components/    les briques visuelles réutilisables
|   |-- contexts/, hooks/, lib/ connexion, appels à l'API, types
|   |-- package.json    les bibliothèques JavaScript et les commandes npm
|   |-- .env.example    le modèle du fichier de configuration
|   |-- FONCTIONNALITES.md la liste détaillée des écrans
|-- docs/              schémas (diagrammes/), captures (captures_memoire/),
                        images (images/), guides (guides/), PDF (pdf/),
                        fichiers LaTeX (tex/), outils de fabrication (build/)
|-- stitch_credix_design_system/ cahier des charges du frontend et système de design

```

3. Ce qu'il faut installer avant de commencer

3.1 Ordinateur nécessaire

Ressource	Minimum conseillé
Système	Linux 64 bits (testé sur Ubuntu)
Espace disque libre	10 Go (les images Docker pèsent environ 6 Go)
Mémoire vive (RAM)	8 Go recommandés
Ports libres	3001, 8080, 8001, 5000, 27017
Connexion Internet	Oui, pour le premier lancement (téléchargement des images et des bibliothèques)

3.2 Les outils

Outil	À quoi il sert	Version utilisée pour les tests
Git	Récupérer et versionner le projet	2.43
Docker et Docker Compose	Lancer le backend, la base de données et les services annexes sans les installer un par un	Docker 29.5, Compose 5.1
Node.js et npm	Lancer et compiler le frontend	Node 22.22, npm 11.14 (Node 18.18 minimum)
curl	Vérifier depuis le terminal que les services répondent	8.16
OpenSSL	Fabriquer une clé secrète aléatoire	3.5

3.3 Vérifier ce qui est déjà installé

Ouvrez un terminal et tapez ces commandes, une par une :

```
git --version
docker --version
docker compose version
node --version
npm --version
curl --version
openssl version
```

Ce que fait chaque commande : elle affiche le numéro de version de l'outil.

Résultat attendu (les numéros peuvent différer légèrement) :

```
git version 2.43.0
Docker version 29.5.3, build d1c06ef
Docker Compose version v5.1.4
v22.22.0
11.14.1
curl 8.16.0 (Linux) libcurl/8.16.0 ...
OpenSSL 3.5.5 27 Jan 2026 ...
```

Si une commande répond « command not found » : l'outil n'est pas installé, passez à la section 3.4 pour cet outil.

3.4 Installer ce qui manque (Ubuntu ou Debian)

Les commandes de cette section viennent de la documentation officielle de chaque outil. Elles n'ont **pas été rejouées** pour ce guide, car la machine de test avait déjà tout installé. En cas de doute, suivez la documentation officielle indiquée.

Git, curl et OpenSSL :

```
sudo apt-get update
sudo apt-get install -y git curl openssl
```

Docker et Docker Compose (documentation officielle : <https://docs.docker.com/engine/install/ubuntu/>) :

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
↳ /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
↳ https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
↳ "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | sudo tee
↳ /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
↳ docker-compose-plugin
```

Ensuite, **autorisez votre utilisateur à utiliser Docker sans** `sudo` (sinon vous aurez l'erreur « permission denied ») :

```
sudo usermod -aG docker $USER
```

Fermez votre session Linux puis rouvrez-la (ou redémarrez), puis testez :

```
docker run --rm hello-world
```

Résultat attendu : un message qui contient Hello from Docker!.

Node.js et npm (avec l'outil nvm, documentation : <https://github.com/nvm-sh/nvm>) :

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

Fermez puis rouvrez le terminal, et tapez :

```
nvm install 22
node --version
```

Résultat attendu : v22. suivi de deux numéros.

4. Récupérer le projet depuis GitHub

4.1 Le lien du dépôt

- Page du projet : <https://github.com/HendrixPenka2/credix>
- Adresse à utiliser pour copier le projet (HTTPS) : <https://github.com/HendrixPenka2/credix.git>

Le dépôt est **privé** : seules les personnes invitées peuvent le voir et le copier. Avant de faire `git clone`, il faut donc :

1. **avoir un compte GitHub** (gratuit) ;
2. **être invité par l'auteur** comme collaborateur (l'auteur ouvre le dépôt sur GitHub, puis *Settings, Collaborators, Add people*) ; vous recevez un e-mail d'invitation, à **accepter** ;
3. **créer un jeton d'accès personnel**, car Git n'accepte plus le mot de passe du compte : sur GitHub, menu de votre photo de profil, *Settings, Developer settings, Personal access tokens, Tokens (classic), Generate new token*, puis *Generate new token (classic)* ; donnez-lui un nom, cochez la permission `repo`, cliquez sur *Generate token*, puis **copiez le jeton affiché** (il n'est montré qu'une seule fois). Documentation : <https://docs.github.com/fr/authentication>.

Ces trois étapes exigent un compte GitHub et n'ont pas pu être rejouées pour ce guide.

4.2 Copier le projet sur votre ordinateur

```
cd ~
git clone https://github.com/HendrixPenka2/credix.git
```

```
cd credix
ls
```

Ce que fait chaque commande :

Commande	Effet
cd ~	Va dans votre dossier personnel
git clone <adresse>	Copie tout le projet (le code et son historique) dans un nouveau dossier credix
cd credix	Entre dans ce dossier
ls	Liste son contenu

Pendant le git clone, Git vous demande de vous identifier (le dépôt est privé) :

```
Username for 'https://github.com':
Password for 'https://votre_nom@github.com':
```

Tapez votre **nom d'utilisateur GitHub**, puis, à la ligne « Password », **collez le jeton** créé en 4.1 (rien ne s'affiche pendant la saisie, c'est normal), puis appuyez sur **Entrée**. Pour ne pas le retaper à chaque fois, vous pouvez taper une fois `git config --global credential.helper store` avant le clone (attention : le jeton est alors enregistré en clair dans votre dossier personnel).

Si vous obtenez Repository not found ou Authentication failed : l'invitation n'a pas été acceptée, ou vous avez saisi le mot de passe du compte au lieu du jeton.

Résultat attendu de ls :

```
credix-v2 docs README.md scoring-backend stitch_credix_design_system
```

4.3 Vérifier la copie

```
git status
git log --oneline -3
ls scoring-backend/artefacts | wc -l
```

Ce que fait chaque commande : `git status` indique la branche et si des fichiers ont changé ; `git log --oneline -3` affiche les 3 dernières modifications enregistrées ; la dernière commande **compte les fichiers du modèle**.

Résultat attendu : `git status` indique « Sur la branche main » et « la copie de travail est propre » ; la dernière commande affiche 16.

Si vous obtenez un autre nombre que 16 : la copie est incomplète. Supprimez le dossier credix (`cd ~ && rm -rf credix`) et recommencez le git clone.

5. Lancer le backend

Toutes les commandes de cette section se tapent **dans le dossier scoring-backend**. Placez-vous-y :

```
cd ~/credix/scoring-backend
pwd
```

Résultat attendu de pwd (affiche le dossier courant) : `/home/votre_nom/credix/scoring-backend`.

5.1 Configurer : créer le fichier .env

Le fichier .env contient les réglages du backend, dont un **secret** qui ne doit jamais être publié. Le dépôt ne contient qu'un modèle, .env.example.

```
cp .env.example .env
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$(openssl rand -hex 32)|" .env
grep -E "^(USE MOCK ARTEFACTS|MONGO_URI|ALLOWED_ORIGINS|JWT_SECRET)=" .env
```

Ce que fait chaque commande :

Commande	Effet
cp .env.example .env	Copie le modèle vers le vrai fichier de configuration
sed -i "s ^JWT_SECRET=.* JWT_SECRET=\$(openssl rand -hex 32) " .env	Remplace la ligne JWT_SECRET= par une clé secrète aléatoire de 64 caractères , fabriquée par openssl rand -hex 32. Cette clé sert à signer les jetons de connexion.
grep -E "^(USE MOCK ARTEFACTS MONGO_URI ALLOWED_ORIGINS JWT_SECRET)=" .env	Affiche les 4 réglages les plus importants pour que vous les contrôliez

Résultat attendu (la valeur de JWT_SECRET sera différente chez vous, mais elle doit faire 64 caractères) :

```
USE MOCK ARTEFACTS=false
MONGO_URI=mongodb://mongodb:27017
JWT_SECRET=3f9c1e...(64 caractères au total)
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3001,http://localhost:5173
```

Ce que signifient ces réglages :

- USE MOCK ARTEFACTS=false : on utilise le **vrai modèle** (le dossier artefacts/) et non des fichiers factices.
- MONGO_URI=mongodb://mongodb:27017 : l'adresse de la base de données **à l'intérieur de Docker** ; ne la changez pas.
- ALLOWED_ORIGINS : les adresses autorisées à appeler l'API. **http://localhost:3001 doit y figurer**, sinon le navigateur bloquera le frontend.
- Les autres lignes du fichier (GOOGLE_API_KEY, HOME_CREDIT_DATA_DIR...) sont **facultatives** et ne servent pas pour tester ; le fichier .env.example les explique une par une.

Si JWT_SECRET contient encore CHANGE_MOI : la deuxième commande a échoué ; vérifiez qu'OpenSSL est installé (openssl version) et relancez-la.

5.2 Démarrer les conteneurs

```
docker compose up -d --build
```

Ce que fait cette commande :

- docker compose lit le fichier docker-compose.yml, qui décrit **4 conteneurs** : l'API, MongoDB, MLflow et le service PDF ;
- up les **démarré** ;
- -d les lance **en arrière-plan** (vous récupérez la main dans le terminal) ;
- --build **fabrique d'abord les images** qui en ont besoin (l'API et le service PDF).

Ce qu'il faut savoir : au **premier lancement**, Docker télécharge des images (MongoDB, MLflow) et

fabrique celles de l'API et du service PDF : il installe des paquets système puis des bibliothèques Python, dont TensorFlow (250 Mo à lui seul). Il y a **environ 700 Mo à télécharger**, donc la durée dépend surtout de votre connexion. Lors de notre test, avec une connexion lente (de 75 à 300 Ko/s), cela a duré **environ 50 minutes** (dont 45 pour l'installation des bibliothèques Python de l'API) ; avec une bonne connexion, comptez nettement moins. Prévoyez aussi **de la place sur le disque** : la construction a besoin d'environ 4 Go de marge temporaire à la toute fin (voir la section 3.1). Les lancements suivants sont rapides : environ **une minute** (les images sont déjà fabriquées).

Comment savoir que c'est terminé ?

- La commande affiche beaucoup de lignes (#15 [api ...] Downloading ...) et **ne vous rend pas la main** tant qu'elle travaille : l'invite du terminal (le \$) ne réapparaît pas.
- Pendant le téléchargement d'un gros fichier comme TensorFlow, **rien ne s'affiche pendant plusieurs minutes** : ce n'est pas un blocage. Pour le vérifier, ouvrez un **autre terminal** et tapez `docker ps` : tant que vos conteneurs `scoring-...` n'apparaissent pas dans la liste, la construction est en cours.
- C'est **terminé** quand l'invite du terminal **réapparaît**, après les lignes `Container ... Started` ci-dessous. Passez alors à la section 5.3.

Résultat attendu à la fin : des lignes de ce type (le mot « error » ne doit pas apparaître) :

```
Container scoring-mongodb Started
Container scoring-mlflow Started
Container scoring-pdf Started
Container scoring-api Started
```

5.3 Vérifier que le backend tourne

Le démarrage de l'API prend **une à deux minutes** (elle charge le modèle en mémoire). Faites les vérifications dans cet ordre.

Vérification 1 : l'état des conteneurs

```
docker compose ps
```

Ce que fait la commande : elle liste les conteneurs du projet et leur état.

Résultat attendu :

NAME	IMAGE	COMMAND	SERVICE
↪ CREATED	STATUS	PORTS	
scoring-api	scoring-backend-api	"uvicorn app.main:ap..."	api
↪ 2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:8080->8000/tcp,	
↪ [::]:8080->8000/tcp			
scoring-mlflow	ghcr.io/mlflow/mlflow:v2.12.1	"mlflow server --hos..."	mlflow
↪ 2 minutes ago	Up 2 minutes	0.0.0.0:5000->5000/tcp,	
↪ [::]:5000->5000/tcp			
scoring-mongodb	mongo:7.0	"docker-entrypoint.s..."	mongodb
↪ 2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:27017->27017/tcp,	
↪ [::]:27017->27017/tcp			
scoring-pdf	scoring-backend-pdf-worker	"uvicorn main:app --..."	
↪ pdf-worker	2 minutes ago	Up 2 minutes	0.0.0.0:8001->8001/tcp,
↪ [::]:8001->8001/tcp			

Comment lire ce tableau : les **4 conteneurs** doivent être Up. `scoring-api` et `scoring-mongodb` doivent afficher (healthy). Juste après le démarrage, `scoring-api` peut afficher (health: starting) : **attendez**, puis retapez la commande. Pour la suivre en direct, tapez `watch -n 5 docker compose ps` et quittez avec Ctrl + C.

Si un conteneur est Exited ou unhealthy : voyez la vérification 2 pour lire l'erreur, puis la section 10.

Vérification 2 : les messages (logs) de l'API

```
| docker compose logs -f --tail 60 api
```

Ce que fait la commande : logs affiche les messages écrits par le service api ; --tail 60 limite aux 60 dernières lignes ; -f **suit les messages en direct**. Pour arrêter de suivre, tapez Ctrl + C : cela **n'arrête pas** l'application.

Résultat attendu au tout premier lancement (l'ordre de quelques lignes peut varier légèrement) :

```
INFO:      Started server process [1]
INFO:      Waiting for application startup.
2026-09-19 17:32:02.014168: I tensorflow/core/platform/cpu_feature_guard.cc:210] This
→ TensorFlow binary is optimized to use available CPU instructions in
→ performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
→ TensorFlow with the appropriate compiler flags.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
=====
      Scoring Backend - Démarrage
=====
[DB] Connecté à MongoDB : scoring_db
[Lifespan] Aucun modèle GridFS en PRODUCTION - fallback seed
[Artefacts] Mode seed - ./artefacts
[Artefacts] ✓ woe_transformers.pkl
[Artefacts] ✓ nap_features.pkl
[Artefacts] ✓ lgbm_final.pkl
[Artefacts] ✓ iv_scores_final.csv
[Artefacts] ✓ feature_stats.json
[Artefacts] ✓ isotonic_calibrator.pkl
[Artefacts] ✓ decision_config.json
[Artefacts] ✓ autoencoder.keras
[Artefacts] ✓ ae_metadata.json
[Artefacts] ✓ isolation_forest.pkl
[Artefacts] ✓ if_metadata.json
[Artefacts] ✓ scaler_if.pkl
[Artefacts] ✓ encoder_hybrid.pkl
[Artefacts] ✓ colonnes_ordonnees_61.json
[Artefacts] Seed chargé - 27 features NAP
[Metadata] 0 documents feature_metadata chargés
[Seuils] Premier démarrage - seuils PDO initialisés depuis decision_config.json :
→ accorde=578.5, refuse=539.5
[OK] Backend prêt
```

Comment lire ces messages :

- Aucun modèle GridFS en PRODUCTION - fallback seed : normal au premier lancement. Comme aucune version du modèle n'a été chargée depuis l'interface, l'API prend **le modèle fourni dans le dossier artefacts/**.
- Les lignes ✓ confirment que **chaque fichier du modèle** est chargé.
- [Metadata] 0 documents ... chargés : normal pour l'instant, les phrases d'explication seront chargées à l'étape 5.4.
- [Seuils] Premier démarrage ... : les **seuils de décision** sont initialisés (accordé à partir de 578,5, refusé en dessous de 539,5).
- Le message I tensorflow/... This TensorFlow binary is optimized ... est une

simple information de TensorFlow : ce n'est pas une erreur.

- [OK] Backend prêt : **c'est la ligne à attendre**. Elle signifie que le backend est opérationnel. (Les lignes INFO : s'affichent parfois **avant** les autres : c'est normal.)
- Une ligne contenant [WARNING] signale un fichier facultatif du modèle absent ; une ligne Traceback OU FileNotFoundError signale une erreur (voir section 10).

Après quelques minutes, vous verrez aussi, toutes les 30 secondes, des lignes GET /health HTTP/1.1" 200 OK : ce sont les **contrôles de santé** automatiques de Docker. Elles sont bon signe.

Vérification 3 : l'état de santé de l'API

```
| curl -s http://localhost:8080/health
```

Ce que fait la commande : curl interroge l'adresse /health de l'API, que l'API utilise pour dire dans quel état elle est ; -s supprime la barre de progression.

Résultat attendu (une seule ligne, avant l'étape 5.4) :

```
{ "statut": "ok", "model_run_id": "seed-initial", "features_chargees": 27, "metadata_chargee"
↪ s": 0, "calibration_isotonique_active": true, "decision_config_charge": true, "seuils_p"
↪ do_default": { "accorde": 578.5, "refuse": 539.5 }, "isolation_forest_charge": true, "autoe"
↪ ncoder_charge": true, "encodage_hybride_61dims_charge": true, "flux_b_actif": true, "fl"
↪ ux_b_percentile": 95, "mock_mode": false }
```

Comment le lire :

Champ	Valeur attendue	Ce que cela veut dire
statut	"ok"	L'API répond
model_run_id	"seed-initial"	Le modèle vient du dossier artefacts/
features_chargees	27	Les 27 variables du modèle sont chargées
metadata_chargees	0 (puis 27 après l'étape 5.4)	Nombre de phrases d'explication chargées
calibration_isotonique_active	true	Les probabilités sont calibrées
decision_config_charge	true	Les seuils de décision sont chargés
isolation_forest_charge, autoencoder_charge, encodage_hybride_61dims_charge	true	Les composants du détecteur d'anomalies sont chargés
flux_b_actif	true	Le détecteur d'anomalies (Flux B) est actif
mock_mode	false	C'est le vrai modèle , pas des fichiers factices

Si curl répond « Connection refused » : l'API n'a pas fini de démarrer. Attendez 30 secondes et recommencez. Si cela persiste : docker compose logs api (voir section 10).

Vérification 4 : les autres services

```
| curl -s http://localhost:8001/health
| curl -sI http://localhost:5000 | head -1
| curl -s -o /dev/null -w "documentation de l'API : HTTP %{http_code}\n"
↪ http://localhost:8080/docs
```

Résultat attendu :

```

{"status":"ok"}
HTTP/1.1 200 OK
documentation de l'API : HTTP 200

```

Ces trois lignes confirment que le **service PDF**, **MLflow** et la **documentation interactive de l'API** répondent. Vous pouvez aussi ouvrir <http://localhost:8080/docs> dans un navigateur : c'est la liste de toutes les routes de l'API, que l'on peut essayer directement.

5.4 Premier lancement : créer l'administrateur et charger les données

À ce stade, la base de données est **vide** : il n'y a ni compte, ni client. Un script prépare tout en une commande. Il fait quatre choses :

1. **crée le compte administrateur** (le tout premier compte, sans lequel on ne peut pas se connecter) ;
2. **charge les 27 phrases d'explication** des variables du modèle (elles ont été écrites une fois avec l'IA Gemini, puis enregistrées dans `demo_data/feature_metadata.json` : **vous n'avez besoin d'aucune clé API**) ;
3. **charge 150 clients de démonstration** (`demo_data/clients_demo.json`, extraits du jeu Home Credit, noms fictifs : **vous n'avez besoin d'aucun fichier Kaggle**) ;
4. **enregistre le modèle fourni** comme version en production.

Pour ce test, utilisez ce mot de passe simple, le même pour les trois comptes (administrateur, agent, superviseur) : `Demo12345`. C'est un mot de passe d'exemple, sans risque sur votre propre ordinateur ; ne l'utilisez jamais sur un vrai serveur. (Vous pouvez en choisir un autre : 8 caractères minimum, de préférence des lettres et des chiffres, sans guillemets ni espaces.)

```
export ADMIN_PASSWORD='Demo12345'
```

Ce que fait cette commande : `export` mémorise le mot de passe **uniquement pour la fenêtre de terminal courante**. Il n'est écrit dans aucun fichier. Ce sera le mot de passe du compte admin.

Lancez ensuite le script :

```
docker compose exec -T -e ADMIN_PASSWORD="$ADMIN_PASSWORD" api python
↪ scripts/init_demo.py
```

Ce que fait cette commande : `exec` exécute une commande **dans le conteneur api qui tourne déjà** ; `-T` indique qu'il n'y a pas de terminal interactif ; `-e ADMIN_PASSWORD=...` transmet le mot de passe au script ; `python scripts/init_demo.py` lance le script de premier lancement.

Résultat attendu :

```

=== 1/4 Compte administrateur ===
[OK] Admin cree: username='admin' / role=ADMIN
      user_id: 21d0207e-d341-4ff7-a86a-125499fde6be

=== 2/4 Phrases d'explication ===
[OK] Phrases d'explication : 27 ajoutes, 0 remplacees, 0 deja presentes (base
↪ 'scoring_db' : 27 au total).
      Redemarre l'API pour qu'elle les recharge : docker compose restart api

=== 3/4 Clients de demonstration ===
[OK] Clients de demonstration : 150 ajoutes, 0 deja presents (base 'scoring_db' : 150
↪ clients au total).

=== 4/4 Enregistrement du modele ===

```

```
[OK] Modele enregistre: run_id=lgbm-run-v1 version=1.0.0 statut=PRODUCTION

[OK] Initialisation terminee.
    Derniere etape : docker compose restart api    (pour que l'API recharge les
    ↪ phrases d'explication)
```

(Le user_id sera différent chez vous.)

Bon à savoir : ce script peut être **relancé sans danger** : il n'ajoute rien en double et ne modifie jamais un compte, une phrase ou un client déjà présent.

Si vous voyez [ERREUR] Mot de passe ADMIN manquant : la variable ADMIN_PASSWORD n'est pas définie dans ce terminal (refaites la commande export). **Si vous voyez trop court :** choisissez un mot de passe d'au moins 8 caractères.

Maintenant, **redémarrez l'API** pour qu'elle recharge les phrases d'explication :

```
docker compose restart api
docker compose up -d --wait --wait-timeout 240 api
```

Ce que font ces commandes : la première redémarre le conteneur de l'API ; la seconde attend (au plus 240 secondes) que l'API soit de nouveau healthy avant de vous rendre la main.

Pourquoi ce redémarrage est nécessaire : l'API charge les phrases d'explication **une seule fois, au démarrage**. Sans redémarrage, elle continuerait à en voir 0.

Vérifiez :

```
curl -s http://localhost:8080/health
```

Résultat attendu : la même ligne qu'en 5.3, mais avec "metadata_chargees":27.

5.5 Vérifier que l'administrateur peut se connecter

```
curl -s -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"username\":\"admin\", \"password\":\"$ADMIN_PASSWORD\"}"
```

Ce que fait la commande : elle envoie à l'API un identifiant et un mot de passe, exactement comme le fera la page de connexion.

Résultat attendu (une seule ligne ; le token est une longue suite de caractères, ici raccourcie) :

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ", "role": "ADMIN", "user_id": "3628800d-5f36-4d9b-9
↪ 4bd-b8012a95660f", "nom": "Administrateur", "prenom": "Systeme", "expires_at": "2026-09
↪ -20T01:34:23.430433+00:00" }
```

Le jeton est valable **8 heures** (expires_at).

Avec un mauvais mot de passe, la réponse est (code 401) :

```
{ "detail": "Identifiants incorrects" }
```

5.6 Récapitulatif : le backend est prêt si...

Contrôle	Commande	Résultat
Conteneurs	docker compose ps	4 conteneurs Up, dont scoring-api et scoring-mongodb (healthy)
Messages de l'API	docker compose logs -tail 60 api	La ligne [OK] Backend prêt

Contrôle	Commande	Résultat
Santé de l'API	<code>curl -s http://localhost:8080/health</code>	<code>"statut":"ok", "features_chargees":27, "metadata_chargees":27, "mock_mode":false</code>
Service PDF	<code>curl -s http://localhost:8001/health</code>	<code>{"status":"ok"}</code>
Connexion	commande de la section 5.5	Un token et <code>"role":"ADMIN"</code>

6. Lancer le frontend

Le frontend est l'interface web. Il tourne avec Node.js et parle au backend, qui doit donc **déjà tourner** (section 5 terminée).

Ouvrez un **nouveau terminal** (ou gardez celui-ci : le frontend occupera la fenêtre tant qu'il tourne) et placez-vous dans le dossier du frontend :

```
cd ~/credix/credix-v2
pwd
```

Résultat attendu de `pwd` : `/home/votre_nom/credix/credix-v2`.

6.1 Configurer

```
cp .env.example .env.local
cat .env.local
```

Ce que fait chaque commande : la première crée le fichier de configuration du frontend ; la seconde l'affiche.

Résultat attendu : une ligne `NEXT_PUBLIC_API_URL=http://localhost:8080`. C'est l'adresse du backend que le navigateur va appeler. **Si votre backend est ailleurs, changez cette adresse maintenant :** elle est **intégrée au moment de la compilation** (section 6.3).

6.2 Installer les bibliothèques

```
npm ci
```

Ce que fait la commande : elle télécharge et installe, dans un dossier `node_modules/`, **exactement les bibliothèques listées dans `package-lock.json`** (les mêmes versions que celles utilisées par l'auteur). Comptez de 30 secondes à quelques minutes selon la connexion.

Résultat attendu (le nombre de paquets et la durée peuvent varier un peu) :

```
added 477 packages in 28s

154 packages are looking for funding
  run `npm fund` for details
```

Sur la machine de test, l'installation a pris **28 secondes** et le dossier `node_modules/` occupe environ **520 Mo**. Des lignes `npm warn` éventuelles sont **sans gravité**.

Si vous obtenez `EBADENGINE` ou une erreur de version : votre Node.js est trop ancien. Vérifiez `node --version` (il faut 18.18 au minimum, 22 conseillé) et reprenez la section 3.4.

6.3 Compiler et démarrer (mode « production »)

```
npm run build
npm run start
```

Ce que fait chaque commande :

Commande	Effet
npm run build	Compile l'application : elle vérifie le code TypeScript, transforme chaque page en fichiers optimisés et les range dans le dossier <code>.next/</code> . Comptez de 30 secondes à deux minutes.
npm run start	Démarre le serveur web sur le port 3001 avec la version compilée

Résultat attendu de `npm run build` (une quarantaine de secondes sur la machine de test) :

```

▲ Next.js 15.5.23
- Environments: .env.local

Creating an optimized production build ...
✓ Compiled successfully in 9.0s
  Skipping linting
  Checking validity of types ...
  Collecting page data ...
✓ Generating static pages (29/29)
  Finalizing page optimization ...
  Collecting build traces ...

Route (app)                                Size  First Load JS
+ ○ /                                         128 B           102 kB
+ ○ /admin                                   2.35 kB         161 kB
+ ○ /admin/audit                             4.08 kB         168 kB
...(une ligne par page, 28 au total)...
+ ○ /superviseur/tranches                    3.31 kB         136 kB
+ First Load JS shared by all               102 kB

```

La commande **se termine sans message d'erreur** et crée un dossier `.next/`. Si elle affiche `Failed to compile. suivi de Type error`, le code contient une erreur de type : voir la section 10.2.

Résultat attendu de `npm run start` :

```

> credix-v2@0.1.0 start
> next start -p 3001

▲ Next.js 15.5.23
- Local:      http://localhost:3001
- Network:    http://192.168.1.20:3001

✓ Starting...
✓ Ready in 857ms

```

(l'adresse « Network » sera celle de votre machine). **Laissez ce terminal ouvert** : si vous le fermez, le frontend s'arrête. Si vous voyez `EADDRINUSE: address already in use :::3001`, le port 3001 est déjà pris (voir la section 10.2).

Variante pour développer (mode « développement ») : `npm run dev` démarre le frontend sur le même port, **sans compilation préalable**, et recharge la page automatiquement à chaque modification du code. C'est le mode à utiliser si vous modifiez le frontend (section 9).

6.4 Vérifier que le frontend tourne

Dans un **autre terminal** :

```
curl -s -o /dev/null -w "page de connexion : HTTP %{http_code}\n"
↪ http://localhost:3001/login
```

Résultat attendu : page de connexion : HTTP 200.

Puis ouvrez <http://localhost:3001> dans un navigateur. **Résultat attendu** : la page de connexion « **Credix AI** », avec le titre « Bienvenue », les champs « Identifiant ou Email » et « Mot de passe », et un bouton « **Se connecter** ».

Si la page ne s'affiche pas : vérifiez que le terminal de `npm run start` est toujours ouvert et sans erreur, puis voyez la section 10.

7. Tester l'application de bout en bout

Ce parcours vérifie que **les trois espaces fonctionnent et communiquent**. Il dure une quinzaine de minutes. Utilisez de préférence des noms d'exemple.

7.1 Se connecter en administrateur

1. Ouvrez <http://localhost:3001>.
2. **Identifiant** : `admin`. **Mot de passe** : `Demo12345` (celui de l'étape 5.4).
3. Cliquez sur « **Se connecter** ».

Résultat attendu : vous arrivez sur la **Vue générale** de l'espace Administrateur, avec 4 indicateurs et le tableau des versions du modèle.

Si vous voyez « Identifiants incorrects » (message rouge sous le mot de passe) : vérifiez le mot de passe. **Si le navigateur affiche une erreur réseau ou de type CORS** : l'adresse <http://localhost:3001> n'est pas dans `ALLOWED_ORIGINS` (section 10).

7.2 Créer un compte Agent et un compte Superviseur

1. Dans le menu de gauche, cliquez sur « **Utilisateurs** », puis sur le bouton « **Nouveau compte** ».
2. Section « **Identité & agence** » : renseignez **Prénom**, **Nom**, **Email**, **Agence** (par exemple Awa, Test, `awa.test@example.com`, Agence Centrale).
3. Section « **Compte & accès** » : choisissez le rôle **Agent**, puis un **Identifiant** (par exemple `agent.test`), le **Mot de passe** `Demo12345` et sa **confirmation** (la même valeur).
4. Cliquez sur « **Créer le compte** ».

Résultat attendu : un écran de confirmation avec l'identifiant créé.

Recommencez avec le rôle **Superviseur** (identifiant `superviseur.test` par exemple, même mot de passe `Demo12345`).

Puis ouvrez de nouveau « **Utilisateurs** ». **Résultat attendu** : la page indique « **3 comptes enregistrés** » et liste les trois comptes par leur **nom et leur e-mail** (« Systeme Administrateur », « Awa Agent », « Samir Superviseur »), avec leur rôle (Admin, Agent, Superviseur) et le statut « Actif ». La liste n'affiche pas les identifiants de connexion.

Un compte n'est **jamais supprimé** dans l'application : il peut seulement être désactivé puis réactivé.

7.3 Se déconnecter, puis se connecter en agent

1. En bas du menu de gauche, cliquez sur « **Déconnexion** ».
2. Connectez-vous avec `agent.test` et le mot de passe `Demo12345`.

Résultat attendu : vous arrivez sur le **Tableau de bord** de l'espace Agent (les statistiques sont à zéro : personne n'a encore été scoré).

7.4 Scorer un client (espace Agent)

1. Menu « **Nouveau scoring** ».
2. **Étape 1, choisir le client** : tapez `HC-100001` dans la recherche et sélectionnez-le.
3. **Étape 2, saisir la demande** (le formulaire est fourni par le backend) :

Champ	Valeur d'exemple
Type de contrat	Cash loans
Annuité mensuelle (FCFA)	36000
Montant du crédit demandé (FCFA)	1000000
Valeur du bien financé (FCFA), facultatif	800000

4. Cliquez sur « **Continuer** » : le calcul démarre (quelques secondes).

Résultat attendu (étape 3), pour HC-100001 avec ces valeurs : un écran de résultat avec la jauge « **Score PDO global** » (**603**, sur une échelle de 300 à 850), la décision **ACCORDÉ**, une **couverture de données de 100 %**, les **5 facteurs qui ont le plus pesé** (« Facteurs d'influence ») expliqués en phrases, un bloc « **Analyse de profil (Flux B) — Aucune anomalie** » quand le profil est normal, un bloc « **Positionnement** » (le percentile du client), et trois boutons : « **Télécharger le rapport PDF** », « Lancer une simulation » et « Recommencer ». Le rapport PDF est fabriqué par le service PDF de la section 5 ; sur la machine de test, le fichier pèse environ 21 Ko.

Pour voir tous les cas de décision, scorez les clients ci-dessous **avec les mêmes valeurs d'exemple** (résultats vérifiés sur une installation neuve). Avec d'autres montants, le score et la décision changent.

Ce que l'on veut voir	Client	Score	Décision	Couverture ρ_C
Un accord net	HC-101368	660	ACCORDÉ	100 %
Un accord (parcours ci-dessus)	HC-100001	603	ACCORDÉ	100 %
Une revue à cause du score (zone entre 539,5 et 578,5)	HC-100241	571	EN REVUE	77 %
Un refus (probabilité de défaut de 32 %)	HC-101327	536	REFUSÉ	89 %
Une revue à cause d'une anomalie (Flux B), alors que le score seul accorderait	HC-100169	589	EN REVUE	91 %

Pour HC-100169, l'écran contient un bloc « **Détection d'anomalie (Flux B) — Revue forcée** », qui explique que le profil a été jugé statistiquement atypique par le détecteur secondaire (l'autoencodeur) et que la décision a été forcée en revue manuelle, quel que soit le score initial.

Voir une couverture faible. Aucun des 150 clients de démonstration n'a une couverture inférieure à 42 %. Pour voir ce cas, **créez un nouveau client**, dont on ne connaît que les informations déclarées :

1. Menu « **Nouveau client** », puis remplissez le formulaire, par exemple : Prénom Awa, Nom Ndiaye, Date de naissance 1996-02-18, Genre Femme, Situation familiale Célibataire, Enfants à charge 0, Téléphone +237 677 12 34 56, Agence Agence Centrale, Type de poste Sales staff, Type de revenu Salarié, Niveau d'éducation Secondaire, Ancienneté emploi (mois) 18, Ancienneté domicile (mois) 24.
2. Cliquez sur « **Créer le dossier** » : la fiche du client s'ouvre, avec les onglets « Vue d'ensemble », « Nouveau scoring », « Simulation », « Historique », « Explicabilité » et « Progression ».
3. Ouvrez l'onglet « **Nouveau scoring** ». Comme ce client est nouveau, le formulaire compte **11 champs** (et non 4). Saisissez : Annuité mensuelle 35000, Montant du crédit demandé 650000, Valeur du bien financé 700000, Type de contrat Cash loans, Profession du demandeur Sales staff, Ancienneté dans l'emploi actuel (mois) 18, Niveau d'éducation Secondary / secondary special, Genre F, Date de naissance 1996-02-18, Type de revenu Working, Ancienneté à l'adresse actuelle (mois) 24, puis cliquez sur « **Continuer** ».

Résultat attendu : une couverture de **35 %**, un bandeau « **Confiance limitée — ce score repose principalement sur les données déclaratives** », une liste de **documents recommandés** à demander au client, un score de **565** et la décision EN REVUE.

Explorez ensuite : « **Simulateur** » (même parcours, mais le résultat n'est **pas enregistré**), « **Historique** », « **Mes rapports** », et la **fiche du client** (menu « **Rechercher** »).

7.5 Traiter un dossier en revue (espace Superviseur)

1. Déconnectez-vous, puis connectez-vous avec `superviseur.test`.
2. Sur la **Vue d'ensemble**, la cloche en haut indique le **nombre de dossiers en attente**.
3. Menu « **Dossiers en revue** » : sélectionnez un dossier dans la liste.
4. Lisez l'analyse (score, couverture, anomalie, profil, facteurs), choisissez **Accordé** ou **Refusé**, écrivez un **commentaire de justification d'au moins 20 caractères** (champ « Expliquez votre décision (minimum 20 caractères)... »), puis cliquez sur « **Valider la décision** ».

Résultat attendu : le dossier quitte la file ; il apparaît dans « **Mes validations** ».

S'il n'y a aucun dossier en revue : revenez en agent (section 7.3) et scorez HC-100241 (score 571, en revue), HC-100169 (revue pour anomalie) ou le nouveau client créé en 7.4. Chaque dossier EN REVUE attend alors dans la file du superviseur.

7.6 Contrôler le journal d'audit et la configuration (espace Administrateur)

1. Reconnectez-vous en admin.
2. Menu « **Journal d'audit** ».

Résultat attendu : une liste datée des actions faites pendant ce test : pour chaque ligne, l'utilisateur, l'**action** (AUTH_LOGIN pour une connexion, USER_CREATE pour une création de compte, SCORING_REQUEST pour un scoring, DECISION_OVERRIDE pour la décision du superviseur), la ressource concernée et le statut « Succès ».

3. Menu « **Versions du modèle** ».

Résultat attendu : une carte « **Production** » indiquant la version **1.0.0** avec **AUC 0.751**, **Gini 0.502** et **KS 0.373**, et, dessous, un tableau des versions avec cette même ligne.

4. Menu « **Configuration** ». **Résultat attendu** : les **seuils de décision** « Seuil Refusé / Revue » à **539,5** et « Seuil Revue / Accordé » à **578,5**, une carte « **Règle active** » (refus sous 540, revue de 540 à 579, accord au-dessus), les **seuils de couverture** ρc (critique : moins de 25 % ; partielle : de 25 à 42 % ; suffisante : à partir de 42 %) et la **sensibilité du détecteur d'anomalies** (percentile de référence P95). Ne modifiez rien pour ce test.

7.7 Liste de contrôle finale

Contrôle	Réussi si...
Connexion des 3 rôles	Chaque rôle arrive sur son espace
Création de comptes	Les deux comptes apparaissent dans « Utilisateurs »
Scoring	Un score, une décision, des facteurs expliqués et un anneau ρc s'affichent
Rapport PDF	Le téléchargement d'un rapport fonctionne
Revue	Un dossier est tranché par le superviseur avec un commentaire
Audit	Les actions ci-dessus figurent dans le journal

8. Arrêter, relancer, tout effacer

Je veux...	Commande (à taper dans scoring-backend/, sauf mention contraire)	Effet sur les données
Arrêter le backend	<code>docker compose stop</code>	Conservées
Relancer le backend arrêté	<code>docker compose start</code>	Conservées
Supprimer les conteneurs (arrêt complet)	<code>docker compose down</code>	Conservées (la base est dans un « volume »)
Tout effacer et repartir de zéro	<code>docker compose down -v</code>	Supprimées : la base est vidée, il faut refaire l'étape 5.4
Arrêter le frontend	<code>Ctrl + C</code> dans son terminal	Sans effet sur les données
Relancer le frontend	<code>npm run start</code> dans credix-v2/ (ou <code>npm run dev</code>)	Sans effet

Pour **voir combien de place Docker occupe** : `docker system df`. Pour **recupérer de la place** (supprime les images et caches inutilisés) : `docker system prune`.

9. Modifier le projet et le remettre à jour avec Git

Git enregistre l'historique du projet. **GitHub** en garde une copie en ligne. Ce chapitre explique comment proposer une modification, sans risquer de casser ce qui fonctionne.

9.1 Une seule fois : se présenter à Git et à GitHub

```
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@example.com"
```

Ces deux lignes indiquent qui signe vos modifications (utilisez l'adresse email de votre compte GitHub).

Pour **envoyer** des modifications vers GitHub, il faut ensuite prouver votre identité. Deux méthodes :

- **Par HTTPS avec un jeton (le plus simple).** C'est le même jeton que celui de la section 4.1 (autorisation repo). Au premier `git push`, Git demande un identifiant (votre nom d'utilisateur GitHub) et un mot de passe : **collez le jeton à la place du mot de passe**.
- **Par SSH.** Générez une clé avec `ssh-keygen -t ed25519 -C "votre.email@example.com"`, copiez le contenu de `~/.ssh/id_ed25519.pub` dans GitHub (*Settings, SSH and GPG keys*), puis testez avec `ssh -T git@github.com`. Utilisez alors l'adresse `git@github.com:HendrixPenka2/credix.git` (`git remote set-url origin git@github.com:HendrixPenka2/credix.git`).

Ces deux méthodes exigent un compte GitHub et n'ont pas pu être jouées pour ce guide. Les commandes Git des sections suivantes, elles, ont été vérifiées.

9.2 Le cycle normal d'une modification

1. Se mettre à jour avant de commencer (récupérer le travail des autres) :

```
cd ~/credix
git checkout main
git pull
```

2. Créer une « branche », c'est-à-dire une copie de travail séparée, pour ne pas toucher à la version stable :

```
git checkout -b ma-modification
```

3. Modifier les fichiers avec votre éditeur (par exemple VS Code : `code .`), puis **tester** (section 9.3).

4. Voir ce qui a changé :

```
git status
git diff
```

`git status` liste les fichiers modifiés ; `git diff` montre les lignes ajoutées et retirées.

5. Enregistrer la modification (en deux temps : choisir les fichiers, puis valider avec un message qui explique **pourquoi**) :

```
git add chemin/vers/le/fichier
git commit -m "Explique en une phrase ce que la modification apporte"
```

6. Envoyer sur GitHub :

```
git push -u origin ma-modification
```

7. Intégrer la modification à la version principale. Le plus sûr : sur GitHub, ouvrez une « **Pull request** » depuis votre branche vers `main`, relisez-la, puis fusionnez-la. Sans passer par GitHub :

```
git checkout main
git merge ma-modification
git push origin main
```

8. Récupérer la version à jour sur un autre ordinateur :

```
git checkout main
git pull
```

9.3 Appliquer une modification et la tester

Ce que vous avez modifié	Comment l'appliquer
Un fichier Python du backend (dossier <code>scoring-backend/app/</code>)	<code>docker compose restart api</code> (le dossier <code>app/</code> est partagé avec le conteneur, pas besoin de reconstruire)
<code>requirements.txt</code> OU <code>Dockerfile.api</code>	<code>docker compose up -d --build</code>
Un fichier du dossier <code>artefacts/</code>	<code>docker compose restart api</code>
Un fichier du dossier <code>demo_data/</code>	Relancer le script de l'étape 5.4 (il n'ajoute que ce qui manque), puis <code>docker compose restart api</code>
Le fichier <code>.env</code> du backend	<code>docker compose up -d --force-recreate api</code> (un simple <code>restart</code> ne relit pas <code>.env</code>)
Le frontend	Avec <code>npm run dev</code> , la page se met à jour toute seule. Pour la version compilée : <code>npm run build</code> , puis relancer <code>npm run start</code> .

Après chaque modification, **refaites les vérifications** de la section 5.6 (backend) ou 6.4 (frontend).

9.4 Revenir en arrière

Situation	Commande
Annuler les changements non enregistrés d'un fichier	<code>git restore chemin/vers/le/fichier</code>
Retirer un fichier de la liste <code>git add</code>	<code>git restore --staged chemin/vers/le/fichier</code>
Annuler un enregistrement déjà validé, en gardant l'historique	<code>git revert <numéro du commit></code> (les numéros s'affichent avec <code>git log --oneline</code>). Un éditeur de texte s'ouvre pour le message : enregistrez et fermez-le (dans <code>nano</code> : <code>Ctrl + O</code> , Entrée, puis <code>Ctrl + X</code>).

9.5 Ce qu'il ne faut jamais publier

Le fichier `.gitignore` de la racine bloque déjà ces éléments. **Vérifiez toujours avec `git status` avant un `git add`** qu'aucun d'eux n'apparaît :

- les fichiers `.env` et `.env.local` (ils contiennent des secrets) ;
- les dossiers `node_modules/` et `.next/` (volumineux et reconstructibles) ;
- tout **mot de passe** ou **clé d'API** écrit dans un fichier.

Si un secret a été publié par erreur, **changez-le immédiatement** (le retirer d'un fichier ne suffit pas : il reste dans l'historique de Git).

9.6 Cas d'erreur fréquents avec Git

Message ou situation	Cause et solution
Connection refused sur le port 22 avec SSH	Votre réseau bloque SSH. Faites passer SSH par le port 443 (voir l'encadré sous ce tableau) ou utilisez l'adresse HTTPS.
Permission denied (publickey)	Votre clé SSH n'est pas enregistrée sur GitHub (section 9.1).
Authentication failed avec HTTPS	Vous avez saisi le mot de passe du compte au lieu du jeton (section 9.1).
! [rejected] main -> main (fetch first) (ou non-fast-forward) au git push	Quelqu'un a envoyé des modifications entre-temps : faites <code>git pull</code> , résolvez les éventuels conflits, puis <code>git push</code> .
CONFLICT pendant un <code>git pull</code> ou <code>git merge</code>	Deux personnes ont modifié les mêmes lignes. Git marque les zones en conflit dans les fichiers (<<<<<<, =====, >>>>>>) : gardez la bonne version, retirez les marqueurs, puis <code>git add</code> et <code>git commit</code> .

Faire passer SSH par le port 443 (quand le port 22 est bloqué) : créez ou complétez le fichier `~/.ssh/config` avec ces quatre lignes, puis testez avec `ssh -T git@github.com`.

```
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
```

10. Dépannage

Quand quelque chose ne marche pas, **lisez d'abord le message d'erreur**, puis cherchez-le ici.

10.1 Docker et backend

Symptôme	Cause probable	Solution
permission denied while trying to connect to the Docker daemon socket	Votre utilisateur n'est pas autorisé à utiliser Docker	<code>sudo usermod -aG docker \$USER</code> , puis fermez et rouvrez votre session (section 3.4)
<code>docker compose : env fileenv not found</code>	Vous n'avez pas créé <code>.env</code> , ou vous n'êtes pas dans <code>scoring-backend/</code>	<code>cd ~/credix/scoring-backend</code> , puis <code>cp .env.example .env</code> (section 5.1)

Symptôme	Cause probable	Solution
port is already allocated OU address already in use (8080, 27017, 5000, 8001)	Un autre programme utilise déjà ce port	Trouvez le programme avec <code>ss -ltnp grep 8080</code> (remplacez 8080 par le port en cause), arrêtez-le, puis relancez <code>docker compose up -d</code>
L'API reste (health: starting) ou passe à unhealthy	Elle charge encore le modèle, ou une erreur bloque son démarrage	Attendez 2 minutes ; sinon <code>docker compose logs --tail 100 api</code> et cherchez Traceback OU Error
Une commande <code>curl</code> répond Connection refused juste après le démarrage	L'API n'a pas fini de charger le modèle	Attendez 30 secondes et recommencez
Dans les logs : FileNotFoundError pour un fichier du modèle	Le dossier artefacts/ est incomplet	<code>ls scoring-backend/artefacts wc -l</code> doit afficher 16. Sinon refaites le <code>git clone</code> (section 4.2)
metadata_chagees vaut 0 dans /health	L'étape 5.4 n'a pas été faite, ou l'API n'a pas été redémarrée après	Refaites la commande <code>init_demo.py</code> , puis <code>docker compose restart api</code>
[ERREUR] Mot de passe ADMIN manquant	ADMIN_PASSWORD n'est pas défini dans ce terminal	Refaites <code>export ADMIN_PASSWORD='...'</code> puis la commande
Le premier <code>docker compose up --build</code> est très long	Normal : TensorFlow est volumineux	Patientez ; les lancements suivants sont rapides
no space left on device	Le disque est plein	<code>docker system df</code> , puis <code>docker system prune</code>
Vous avez modifié .env mais rien ne change	<code>restart</code> ne relit pas .env	<code>docker compose up -d --force-recreate api</code>
Vous avez oublié le mot de passe administrateur		Créez un autre administrateur : <code>docker compose exec -T -e ADMIN_PASSWORD="nouveau_mot_de_passe" api python scripts/create_admin.py --username admin2</code> . Ou repartez de zéro avec <code>docker compose down -v</code> (efface toutes les données).
Permission denied en supprimant un dossier <code>__pycache__</code> dans <code>scoring-backend/app/</code>	Ces dossiers sont créés par Docker (donc appartiennent à root)	<code>sudo rm -rf scoring-backend/app/__pycache__</code>

10.2 Frontend

Symptôme	Cause probable	Solution
Sur la page de connexion : erreur réseau, ou message CORS dans la console du navigateur	L'API n'autorise pas l'adresse du frontend	Vérifiez que <code>ALLOWED_ORIGINS</code> (fichier <code>scoring-backend/.env</code>) contient <code>http://localhost:3001</code> , puis <code>docker compose up -d --force-recreate api</code>
La page se charge mais aucune donnée ne s'affiche	Le frontend n'atteint pas le backend	<code>curl -s http://localhost:8080/health</code> doit répondre ; vérifiez <code>NEXT_PUBLIC_API_URL</code> dans <code>credix-v2/.env.local</code> . Après l'avoir modifié, refaites <code>npm run build</code> (l'adresse est intégrée à la compilation).
<code>npm ci</code> échoue, <code>EBADENGINE</code> <code>Failed to start server</code> puis <code>EADDRINUSE: address already in use :::3001</code>	Node.js trop ancien Le port 3001 est déjà pris (souvent par un autre frontend qui tourne)	Installez Node 22 (section 3.4) Fermez le terminal de cet autre frontend, ou cherchez le programme avec <code>ss -ltnp grep 3001</code>
<code>Failed to compile</code> . suivi de <code>Type error: ...</code> pendant <code>npm run build</code>	Le code contient une erreur de type TypeScript	Lisez le fichier et la ligne indiqués. <code>npx tsc --noEmit</code> liste toutes les erreurs d'un coup (<code>npm run build</code> n'en affiche qu'une à la fois)
Le navigateur affiche la page de connexion en boucle	Le jeton de connexion a expiré (il dure 8 heures) ou est invalide	Reconnectez-vous
Un agent qui tape <code>/admin</code> dans l'adresse voit un écran d'administration vide	Limite connue du frontend : il ne bloque pas l'affichage de la coquille. Le backend, lui, refuse toutes les données d'un espace non autorisé	Comportement sans danger pour les données

11. Annexes

11.1 Glossaire

Terme	Explication simple
Terminal	La fenêtre où l'on tape des commandes
Backend	Le « cerveau » de l'application : il calcule, stocke et vérifie. On ne le voit pas.
Frontend	L'interface web que l'utilisateur voit et sur laquelle il clique

Terme	Explication simple
API	La façon dont le frontend « parle » au backend, par des adresses web (/api/scoring/predict...)
Docker, conteneur, image	Un conteneur est une petite boîte isolée qui fait tourner un logiciel avec tout ce dont il a besoin. Une image est le moule à partir duquel on fabrique un conteneur.
Docker Compose	L'outil qui démarre plusieurs conteneurs d'un coup, d'après le fichier <code>docker-compose.yml</code>
Volume	Le disque où un conteneur garde ses données, pour qu'elles survivent à un arrêt
MongoDB	La base de données de l'application
JWT (jeton)	Un « badge » temporaire délivré après connexion, envoyé à chaque requête pour prouver qui l'on est
Probabilité de défaut (PD)	La chance estimée qu'un client ne rembourse pas son crédit
Score PDO	La PD convertie en un score lisible de 300 à 850 : plus il est haut, plus le risque est faible
Calibration	La correction des probabilités pour qu'elles correspondent à la fréquence réelle des défauts
WoE	Transformation qui remplace chaque valeur d'une variable par son pouvoir à distinguer bons et mauvais payeurs
SHAP	Méthode qui indique combien chaque variable a poussé le score vers le haut ou vers le bas
ρc (couverture)	La part d'information utile dont le système dispose sur un client. Un dossier très incomplet a un ρc bas.
Flux B, atypicité	Le contrôle qui repère les profils différents de ceux sur lesquels le modèle a appris
PSI, dérive	Indicateur qui mesure si les clients d'aujourd'hui ressemblent encore à ceux de l'entraînement. En dessous de 0,10 : stable ; de 0,10 à 0,25 : attention ; à partir de 0,25 : dérive.
Artefacts	Les fichiers du modèle entraîné (le dossier <code>artefacts/</code>)
Revue manuelle	Un dossier que le système ne tranche pas seul et qu'un superviseur humain doit décider
Dépôt (repository)	Le projet et tout son historique, sur GitHub
Branche, commit, push, pull	Une branche est une copie de travail. Un commit est une modification enregistrée. <code>push</code> envoie vers GitHub, <code>pull</code> récupère depuis GitHub.

11.2 Récapitulatif de toutes les commandes

```
# --- Récupérer le projet (section 4) ---
cd ~ && git clone https://github.com/HendrixPenka2/credix.git && cd credix

# --- Backend (section 5) ---
cd ~/credix/scoring-backend
cp .env.example .env
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$(openssl rand -hex 32)|" .env
```

```

docker compose up -d --build
docker compose ps
docker compose logs --tail 60 api
curl -s http://localhost:8080/health
export ADMIN_PASSWORD='Demo12345'
docker compose exec -T -e ADMIN_PASSWORD="$ADMIN_PASSWORD" api python
↳ scripts/init_demo.py
docker compose restart api
docker compose up -d --wait --wait-timeout 240 api
curl -s http://localhost:8080/health

# --- Frontend (section 6) ---
cd ~/credix/credix-v2
cp .env.example .env.local
npm ci
npm run build
npm run start                # puis ouvrir http://localhost:3001

# --- Arrêter (section 8) ---
docker compose stop          # dans scoring-backend/
docker compose down -v       # tout effacer

```

11.3 Pour aller plus loin

Document	Contenu
scoring-backend/ README.md	Documentation technique détaillée du backend (collections MongoDB, pipeline, routes). Certains chiffres y datent de juin 2026 : en cas de différence, ce README fait foi.
credix-v2/ FONCTIONNALITES.md	Description de tous les écrans, page par page
scoring-backend/ demo_data/ LISEZMOI.md	D'où viennent les phrases d'explication et les clients de démonstration, et comment les régénérer
docs/diagrammes/ stitch_credix_ design_system/	Schémas de conception (draw.io et PlantUML) Cahier des charges du frontend et système de design
docs/guides/	Guides sur la méthode de conception avec Claude : fiche de démonstration (00_), outils Claude (01_), méthode de conception (02_), conception du MVP (03_), autres fonctions de Claude (04_)
docs/pdf/	Les mêmes documents en PDF, dont le dossier de remise complet (00_Dossier_de_remise.pdf)
docs/tex/	Le code LaTeX (.tex) de chaque document, à recompiler pour y insérer des images (voir docs/tex/LISEZMOI.md)
docs/build/	Outils qui fabriquent les .tex et les PDF à partir des fichiers Markdown (guide 4, section 8)

PROJET DE FIN D'ÉTUDES

École Nationale Supérieure Polytechnique de Yaoundé
Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique
Année académique 2025–2026

Guide 1 : les outils Claude

Abonnements, sessions, mode Projet et Claude Code

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Table des matières

Guide 1 : les outils Claude	1
1. À quoi sert ce guide	1
2. Ce qu'est Claude	1
3. Créer un compte et choisir un abonnement	2
4. Comprendre les limites et les sessions	4
5. Utiliser Claude dans le navigateur (claude.ai)	7
6. Le mode Projet, indispensable pour un travail long	9
7. Claude Code : installer et utiliser l'assistant dans VS Code	13
8. Les modes de Claude Code, dont le mode Plan	17
9. Bonnes pratiques et précautions	19
10. Glossaire et sources officielles	20

Guide 1 : les outils Claude

Abonnements, sessions, mode Projet et Claude Code dans VS Code : tout ce qu'il faut savoir pour commencer, même sans aucune expérience.

À propos des captures d'écran. Les captures viennent de l'interface **en français** de claude.ai et de VS Code (l'extension Claude Code, elle, s'affiche en anglais). Elles servent de repères : chaque étape est décrite dans le texte, bouton par bouton, et se suit **sans** l'image. Quand un libellé diffère légèrement chez vous, cherchez l'option de même sens.

À propos des prix et des menus. Les prix et les noms de menus de Claude changent avec le temps. Ceux de ce guide ont été **vérifiés sur les pages officielles le 19 septembre 2026** (sources en section 10). En cas de doute, la page officielle des tarifs fait foi.

1. À quoi sert ce guide

Ce guide est le premier d'une série. Il répond à une question simple : **comment prendre en main Claude pour mener un vrai projet, comme la conception d'une application ?**

À la fin de ce guide, vous saurez :

- choisir un **abonnement** adapté et savoir **ce qu'il coûte** ;
- comprendre le mécanisme de **session** et **surveiller votre consommation** ;
- travailler dans le **navigateur** (claude.ai), et surtout en **mode Projet**, qui est bien plus adapté à un travail long qu'un simple « nouveau chat » ;
- **installer Claude Code dans VS Code**, l'assistant qui lit et modifie les fichiers de votre projet ;
- utiliser le **mode Plan**, qui fait d'abord réfléchir Claude avant de toucher au moindre fichier.

Les guides suivants montrent comment ces outils ont servi à concevoir CREDIX (guide 2, la méthode) et comment rejouer la démarche sur un exemple réduit (guide 3, le MVP).

2. Ce qu'est Claude

Claude est un assistant d'intelligence artificielle développé par l'entreprise **Anthropic**. On le fait travailler en lui écrivant en langage courant, comme à un collègue. Il sait lire des documents, rédiger, analyser, programmer.

On l'utilise de deux façons principales, complémentaires :

Outil	Ce que c'est	On s'en sert pour
Claude sur le web (claude.ai)	Une page web de discussion, avec des conversations, des projets et l'envoi de fichiers	Réfléchir, cadrer le besoin, chercher dans la littérature, rédiger les documents de conception
Claude Code	Un assistant de programmation qui travaille directement dans vos dossiers : il lit vos fichiers, les modifie, lance des commandes	Écrire, tester et corriger le code d'une application

Claude Code existe dans le terminal, dans **VS Code** (l'éditeur de code utilisé ici), dans les environnements JetBrains, en application de bureau et sur le web.

Règle d'or du travail avec Claude : *Claude propose, vous décidez*. Il peut se tromper : chaque décision importante doit être relue et validée par une personne.

3. Créer un compte et choisir un abonnement

3.1 Créer un compte

1. Ouvrez un navigateur (Chrome, Firefox...), tapez **claude.ai** dans la barre d'adresse, puis appuyez sur Entrée.
2. La page d'accueil propose trois façons de se connecter. Choisissez-en une :
 - cliquez sur le bouton « **Continuer avec Google** » ;
 - ou sur le bouton « **Continuer avec Apple** » ;
 - ou tapez votre adresse e-mail dans le champ « **Saisissez votre e-mail** », puis cliquez sur « **Continuer par l'e-mail** ».
3. Avec l'e-mail, Claude vous envoie un message pour confirmer votre adresse : ouvrez votre boîte e-mail et suivez les indications du message.
4. Vous arrivez sur la page de discussion de Claude.

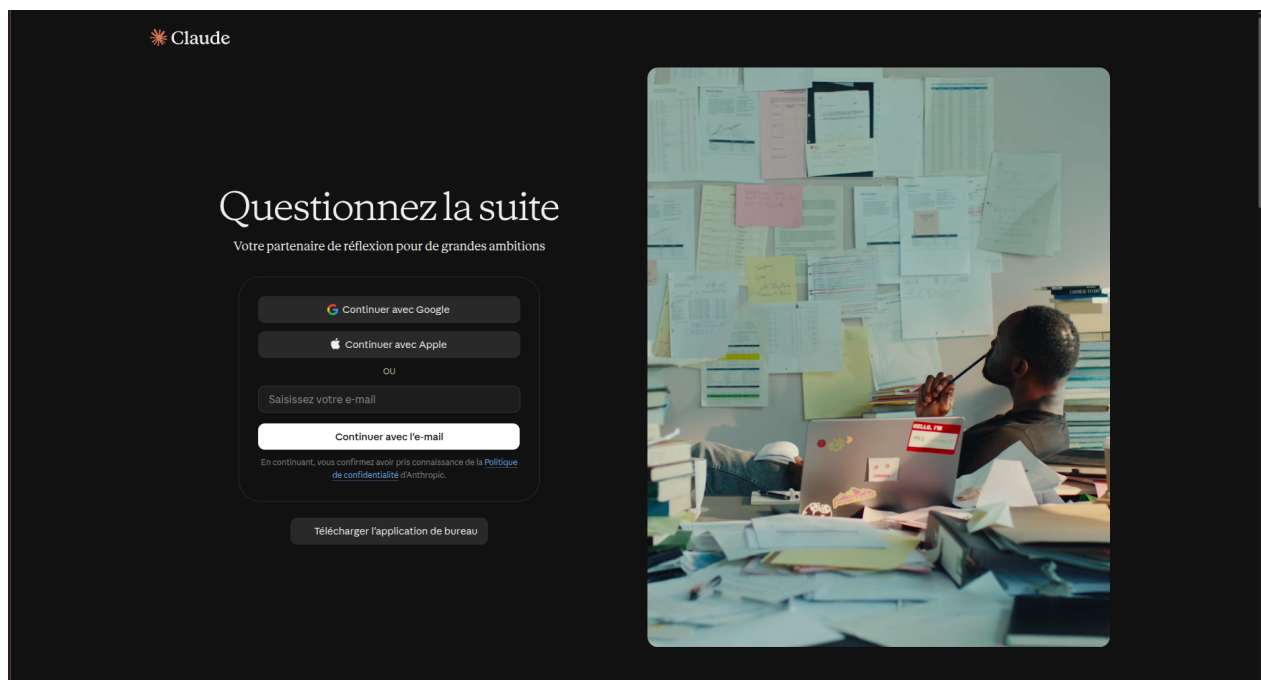


FIGURE 1 – La page d'accueil de claude.ai et ses trois façons de se connecter.

3.2 Les abonnements et leurs prix

Un compte gratuit existe, mais un **abonnement payant** est nécessaire pour un vrai projet : il donne beaucoup plus d'usage, et il est **indispensable pour utiliser Claude Code** (avec, à défaut, un compte de développeur « Console » facturé à l'usage).

Prix relevés sur les pages officielles le 19 septembre 2026 (en dollars américains, hors taxes) :

Abonnement	Pour qui	Prix
Gratuit	Découvrir	0 \$, usage limité
Pro	Usage personnel (étudiant, indépendant) : le plus courant	20 \$ par mois (un tarif annuel, plus avantageux, existe)
Max	Usage personnel intensif, notamment beaucoup de Claude Code	100 \$ par mois (« 5x » : cinq fois l'usage de Pro) ou 200 \$ par mois (« 20x » : vingt fois)
Team	Une équipe ou une petite entreprise (au moins 2 membres)	Siège standard : 25 \$ par membre et par mois (20 \$ si facturé à l'année) ; siège premium : 125 \$ (100 \$ à l'année)
Enterprise	Une grande organisation : sécurité et administration renforcées (connexion unique, journaux d'audit, etc.)	Non publics : le siège donne accès à la plateforme, et l'usage est facturé à part, aux tarifs de l'API. Achat en libre-service à partir de 20 sièges, avec un commercial à partir de 50.

Lequel choisir ?

— **Pour ce projet, un compte Pro suffit** pour commencer. Si vous utilisez beaucoup Claude

Code, vous atteindrez plus vite les limites et Max peut devenir utile.

- **Team** et **Enterprise** sont pensés pour le travail en entreprise : partage de projets entre collègues, administration centralisée.

Où voir cette page depuis votre compte ? Cliquez sur votre profil, en bas à gauche, puis sur « **Mettre le forfait à niveau** ». La page propose deux onglets : « **Particuliers** » (Free, Pro et Max) et « **Team et Enterprise** ».

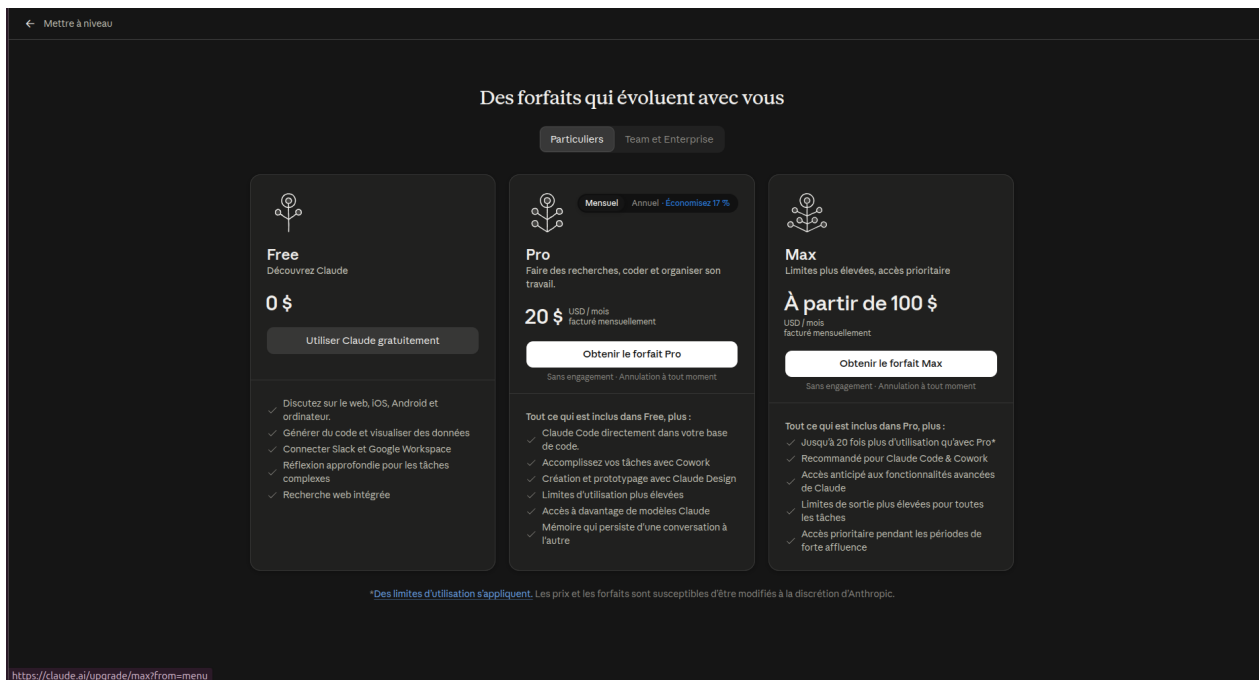


FIGURE 2 – La page des forfaits, onglet « Particuliers » (prix à vérifier au moment de l'achat).

4. Comprendre les limites et les sessions

C'est le point que les débutants comprennent le moins, et celui qui « bloque » le plus souvent. Il faut distinguer **deux limites** et **deux sens du mot « session »**.

4.1 Les deux limites

Limite	Ce qu'elle mesure	Ce qui se passe quand on l'atteint
Limite d'usage	Le « budget de conversation » sur une période : combien vous avez interagi avec Claude (sur claude.ai , dans Claude Code, dans l'application de bureau)	Il faut attendre la remise à zéro
Limite de longueur	La taille de la « mémoire de travail » de Claude dans une même conversation (sa fenêtre de contexte)	Il faut ouvrir une nouvelle conversation, ou laisser Claude résumer les échanges anciens

4.2 La « session d'usage » de 5 heures

Sur les abonnements Pro et Max, l'usage se mesure par **sessions** :

- **toutes les cinq heures**, votre limite de session est **remise à zéro** ;
- en plus, il existe une **limite hebdomadaire**, qui s'applique à l'ensemble des modèles ;
- ce qui compte n'est pas le nombre de messages mais leur « poids » : la longueur du message, les fichiers joints, la complexité de la réponse.

Ce qui consomme le plus : les conversations très longues, les nombreuses pièces jointes, la réflexion étendue, l'effort élevé, les connecteurs et la recherche sur le web. Claude Code, qui lit beaucoup de fichiers, consomme aussi vite.

4.3 Où surveiller sa consommation

1. Cliquez sur **votre profil, en bas à gauche** de claude.ai : c'est le bouton qui affiche votre prénom et votre forfait (par exemple « Hendrix · Pro »). Un menu s'ouvre.
2. Dans ce menu, cliquez sur « **Paramètres** » (raccourci : Ctrl + Maj + ,).
3. Dans la colonne de gauche de la fenêtre qui s'ouvre, cliquez sur « **Utilisation** ».
4. Lisez les trois blocs de la page :
 - « **Session actuelle** » : une barre indique la part déjà utilisée (par exemple « 63 % utilisés ») et le temps avant la remise à zéro (par exemple « Réinitialisation dans 2 h 16 min ») ;
 - « **Limites hebdomadaires** », ligne « **Tous les modèles** » : la même chose pour la semaine, avec le jour et l'heure de la remise à zéro ;
 - « **Crédits d'utilisation** » : un interrupteur pour continuer à utiliser Claude quand la limite du forfait est atteinte. Des frais peuvent s'appliquer : laissez-le désactivé tant que vous n'avez pas vérifié.
5. La ligne « **Dernière mise à jour** » indique quand ces chiffres ont été rafraîchis ; le petit bouton rond à côté les met à jour.

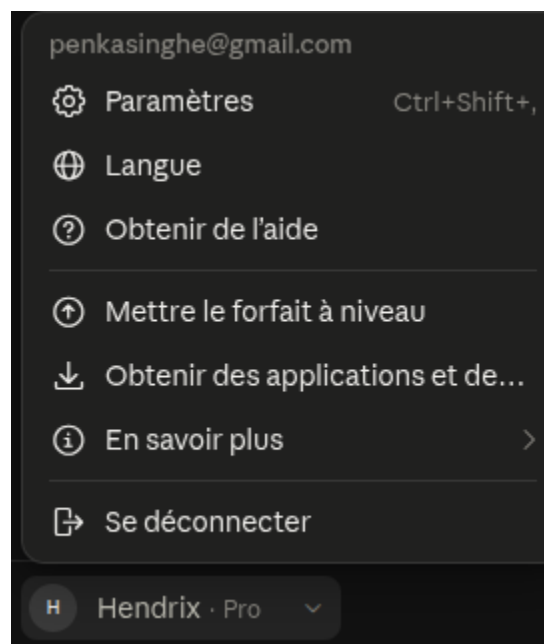


FIGURE 3 – Le menu du profil : l'entrée « Paramètres » ouvre la fenêtre des réglages.

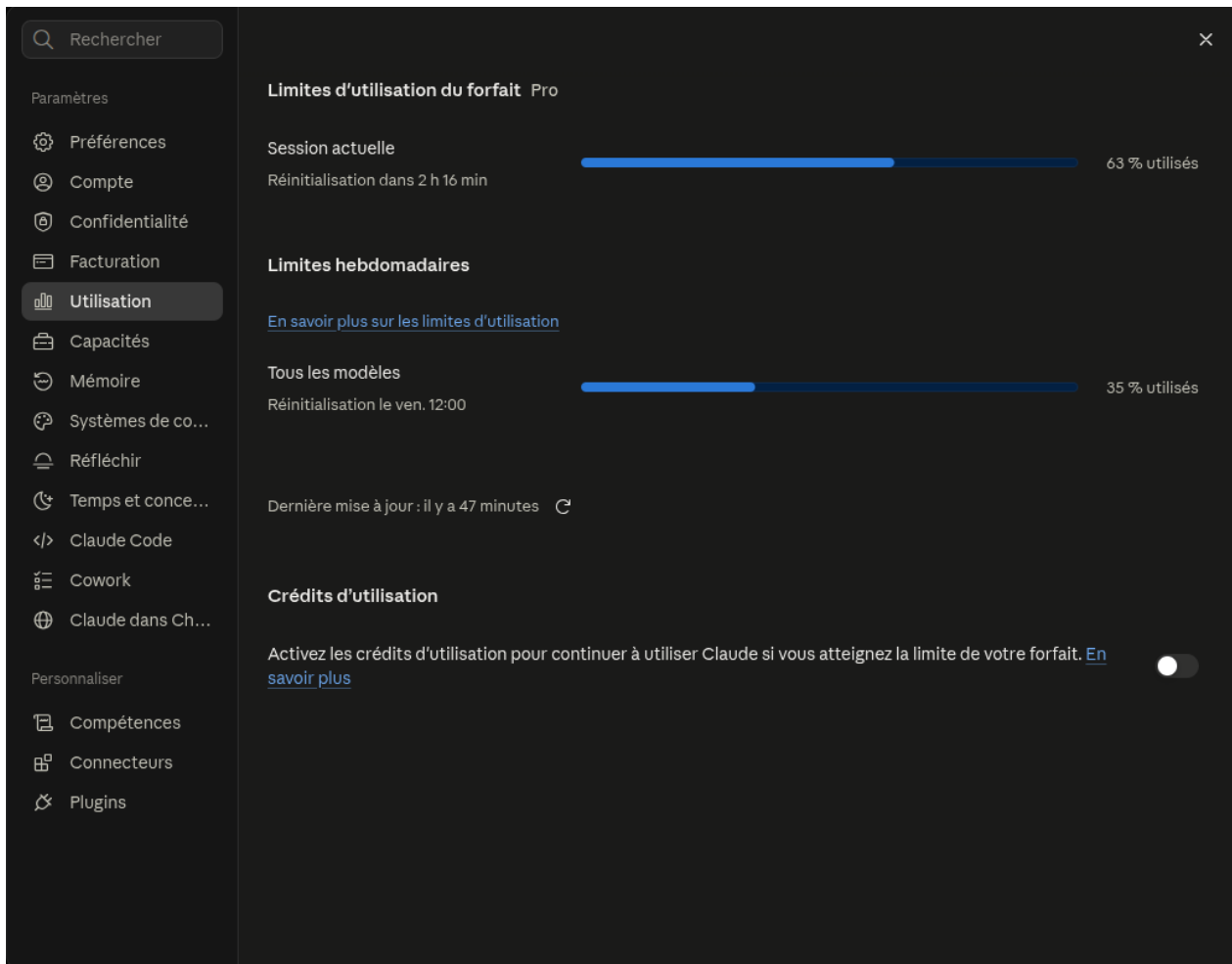


FIGURE 4 – L'onglet « Utilisation » : session actuelle et limites hebdomadaires.

4.4 Comment consommer moins

- **Une conversation par sujet.** Une conversation qui s'allonge coûte de plus en plus cher : ouvrez-en une nouvelle quand vous changez de sujet.
- **Utilisez le mode Projet** (section 6) : les documents y sont chargés une fois, sans les rejoinde à chaque message.
- **Retirez les fichiers inutiles** d'un projet.
- **Coupez les options non nécessaires** : réflexion étendue, recherche web, connecteurs.
- **Demandez des choses précises** plutôt que de faire refaire plusieurs fois le même travail.

4.5 Le deuxième sens de « session » : la session de travail

Au-delà de la limite technique, on parle de **session de travail** : un moment de travail suivi (une demi-journée, par exemple) sur un objectif. C'est une notion de **méthode**, expliquée dans le guide 2 :

Les produits d'une session sont les entrées de la session suivante. À la fin d'une session, on demande à Claude de produire un document de reprise (ce qui a été fait, décidé, ce qui reste). Ce document est déposé dans le projet et sert de point de départ à la session suivante.

5. Utiliser Claude dans le navigateur (claude.ai)

5.1 Une première conversation

1. Sur claude.ai, cliquez sur le bouton « **+ Nouveau** », en haut de la colonne de gauche.
2. Cliquez dans la zone de saisie (« Comment puis-je vous aider aujourd'hui ? » sur l'écran d'accueil, « Écrivez un message. . . » dans une conversation) et tapez votre demande.
3. Appuyez sur **Entrée** pour l'envoyer. **Maj + Entrée** passe à la ligne sans envoyer.
4. Pour **joindre un fichier** : cliquez sur le bouton « **+** » à gauche de la zone de saisie, puis sur « **Ajouter des fichiers ou des photos** » (raccourci **Ctrl + U**), et choisissez le fichier (PDF, texte, image, tableau) dans la fenêtre qui s'ouvre.

Sur l'écran d'accueil, deux repères utiles : le sélecteur « **Chat / Cowork** » sous la zone de saisie (restez sur « **Chat** ») et, plus bas, la liste de vos conversations passées : « **Épinglés** » puis « **Récents** ». À droite de la zone de saisie, le nom du modèle est suivi d'un niveau (par exemple « Sonnet 5 Élevé »).

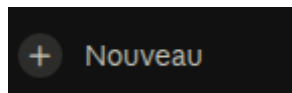


FIGURE 5 – Le bouton « + Nouveau » ouvre une nouvelle conversation.

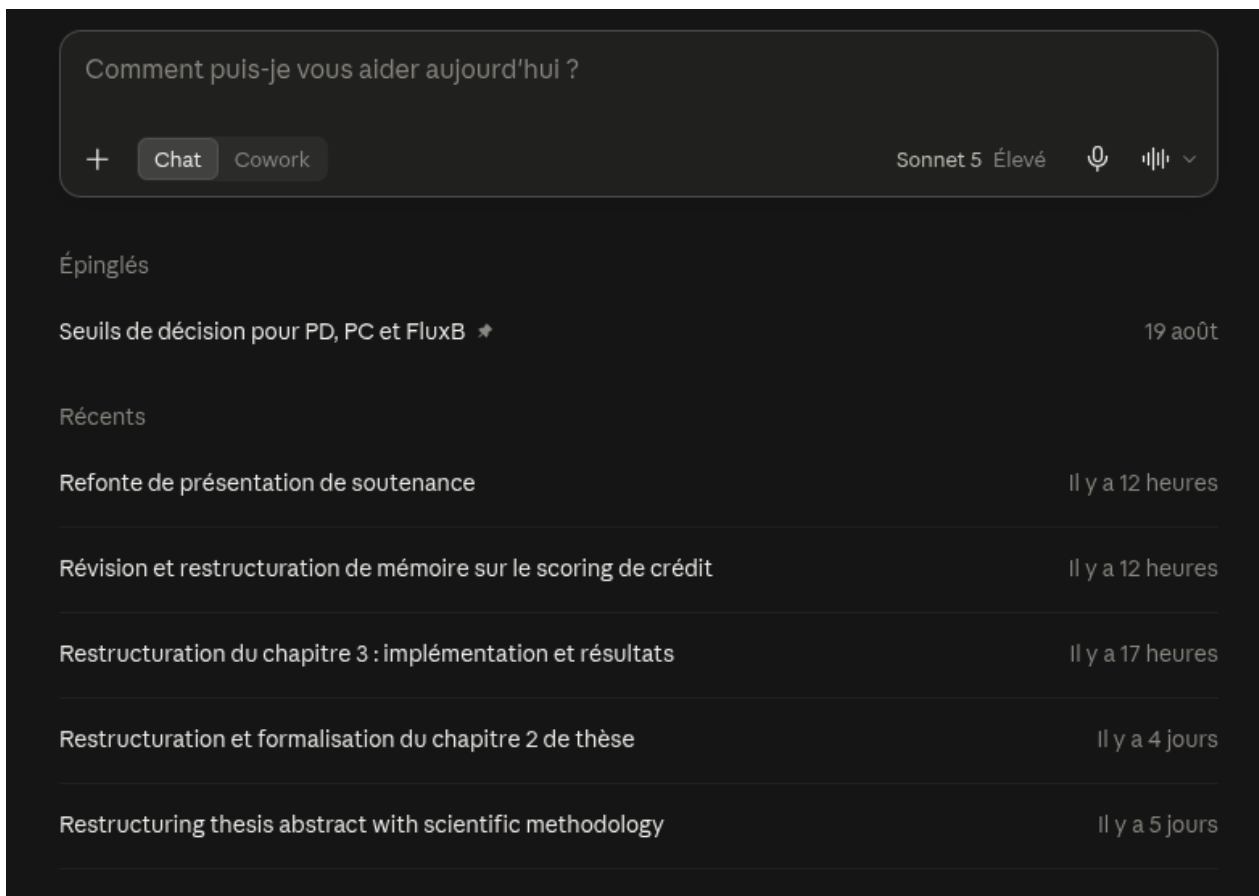


FIGURE 6 – L'écran d'accueil : zone de saisie, sélecteur « Chat / Cowork », modèle utilisé et conversations récentes.

Exemples de demandes utiles :

Explique-moi en termes simples la différence entre un score de crédit et une probabilité de défaut.

Voici un article scientifique en PDF. Résume-le en dix lignes et dis-moi en quoi il concerne le choix d'une méthode de sélection de variables.

5.2 Les outils de recherche et de réflexion

1. Cliquez sur le bouton « + » à gauche de la zone de saisie. Un menu s'ouvre, avec notamment : « Ajouter des fichiers ou des photos », « Prendre une capture d'écran », « Ajouter au projet », « Compétences », « Connecteurs », « Système de design », « Ajouter des plugins », « **Recherche** » et « **Recherche web** ».
2. Cliquez sur « **Recherche web** » pour l'activer ou la désactiver : une **coche bleue** apparaît en face quand elle est active.
3. Cliquez sur « **Recherche** » pour demander une recherche approfondie (selon votre abonnement et votre pays, cette option peut varier : vérifiez dans votre menu).
4. Pour la réflexion, regardez à droite de la zone de saisie : le nom du modèle est suivi d'un niveau (par exemple « Sonnet 5 Élevé »). Ce niveau règle l'effort de réflexion : plus il est haut, plus Claude réfléchit longtemps, et plus la réponse consomme.

Outil	Quand l'utiliser	En bref
Recherche web	Une question factuelle, une information récente	Claude fait une ou deux recherches et répond
Réflexion étendue (niveau de réflexion du modèle)	Un raisonnement difficile, sans besoin d'Internet : mathématiques, déboguer du code, comparer des choix	Claude réfléchit plus longtemps avant de répondre
Recherche approfondie (<i>Research</i> , entrée « Recherche » du menu)	Un vrai travail de recherche : état de l'art, comparaison de méthodes	Claude enchaîne au moins cinq recherches pendant une à trois minutes et rédige un rapport avec des citations vers ses sources

La recherche approfondie est **très utile pour la partie « littérature »** d'un projet (guide 2).

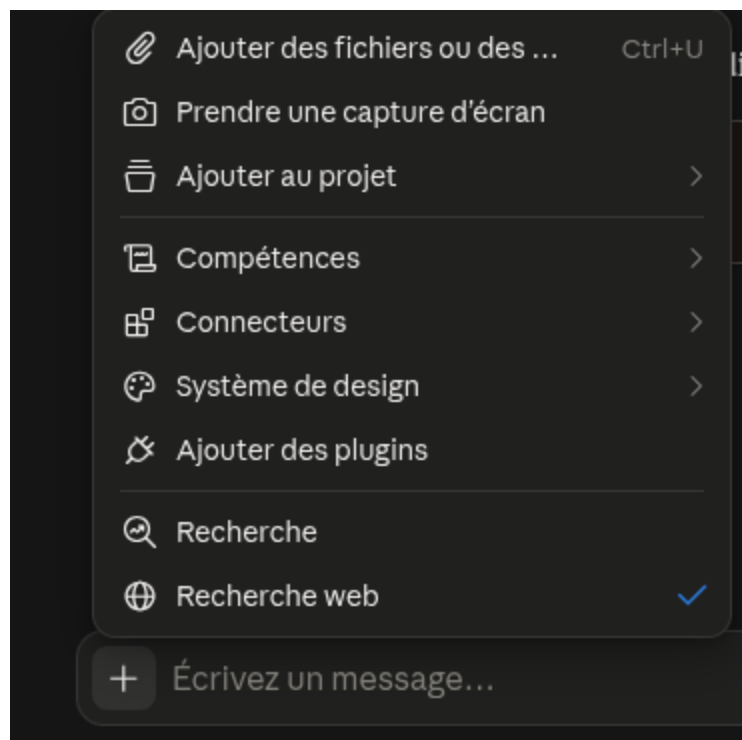


FIGURE 7 – Le menu « + » : joindre des fichiers, ajouter au projet, « Recherche » et « Recherche web » (coche bleue quand elle est active).

Toujours vérifier les sources. Même avec des citations, ouvrez les références importantes et vérifiez qu'elles disent bien ce que Claude rapporte. C'est une règle de rigueur pour tout travail académique.

6. Le mode Projet, indispensable pour un travail long

6.1 Chat ou Projet ?

Un **nouveau chat** est une conversation isolée : Claude ne connaît que ce qui s'y dit. Un **Projet** est un **espace de travail** qui regroupe plusieurs conversations et des documents partagés.

	Nouveau chat	Projet
Documents	À rejoindre à chaque conversation	Chargés une fois dans la « base de connaissances » du projet, disponibles dans toutes ses conversations
Consignes	À répéter	Instructions du projet , appliquées automatiquement à chaque conversation
Mémoire entre conversations	Aucune	Les conversations partagent le même contexte documentaire
Volume de documents	Limité à la conversation	Plus grand : sur les abonnements payants, quand le contenu s'approche de la limite, Claude passe en mode de recherche (RAG) et peut multiplier par 10 la capacité
Travail d'équipe	Non	Partage possible (Team et Enterprise) : « Peut voir » ou « Peut modifier »

	Nouveau chat	Projet
Adapté à	Une question rapide	Un projet qui dure plusieurs jours ou semaines

Les projets sont disponibles pour tous les comptes, y compris gratuits (dans la limite de cinq projets).

6.2 Créer un projet, pas à pas

1. Dans la colonne de gauche de claude.ai, cliquez sur « **Projets** ». La page « **Projets** » s'ouvre : chaque projet y est une carte avec son nom et la date de sa dernière activité.
2. Cliquez sur le bouton « **Nouveau projet** », en haut à droite de la page.
3. La fenêtre « **Créer un projet** » s'ouvre :
 - dans le champ « **Sur quoi travaillez-vous ?** » (« Nommez votre projet »), tapez le nom, par exemple CREDIX ;
 - dans le champ « **Qu'essayez-vous de faire ?** », décrivez le projet, ses objectifs, son sujet.
4. Cliquez sur « **Créer un projet** » (ou sur « Annuler » pour abandonner).
5. La page du projet s'ouvre avec trois zones : « **Instructions** », « **Contexte** » et « **Programmé** ». Les deux premières servent aux sections 6.3 et 6.4 ; « Programmé » (tâches récurrentes) ne sert pas ici.
6. Démarrez ensuite une conversation **à l'intérieur** du projet, avec la zone de saisie du projet.

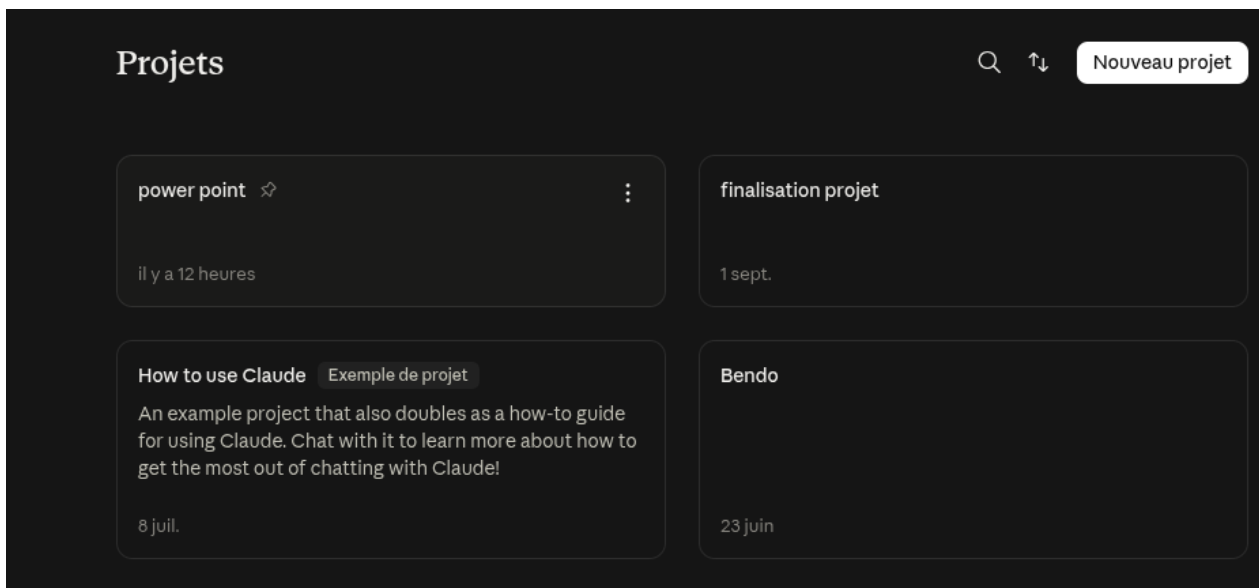


FIGURE 8 – La page « Projets » et son bouton « Nouveau projet ».

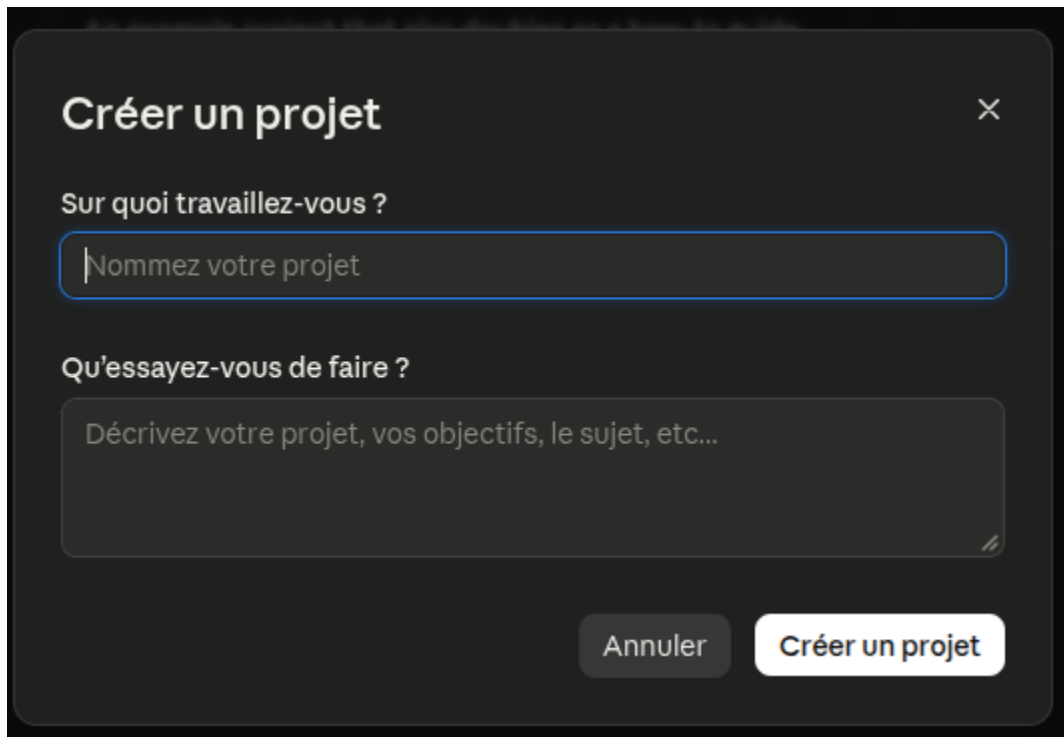


FIGURE 9 – La fenêtre « Créer un projet » : le nom, puis la description.

6.3 Bien écrire les instructions du projet

Les instructions décrivent **qui est Claude dans ce projet et comment il doit travailler**. Exemple utilisé pour CREDIX, à adapter :

Tu es mon assistant de conception pour un mémoire d'ingénieur sur un système de scoring de crédit.

Règles de travail :

1. Tu proposes, je décide. Avant toute décision importante, tu présentes les options, leurs avantages et leurs limites, puis tu attends mon accord.
2. Aucun chiffre inventé : tout chiffre vient d'un document du projet ou d'un calcul que je t'ai transmis. Sinon, tu dis que tu ne sais pas.
3. Tu cites tes sources (auteur, année) et tu signales quand une référence reste à vérifier.
4. Tu réponds en français, en termes simples, en structurant avec des titres et des listes.
5. À la fin de chaque session, tu produis un document de reprise en Markdown : ce qui a été fait, ce qui a été décidé, ce qui reste à faire.

Où saisir ces instructions ? Dans la page du projet, cliquez sur le bouton « + » à droite de « **Instructions** » (le texte grisé indique « Ajoutez des instructions pour personnaliser les réponses de Claude »). Collez le texte, puis validez.

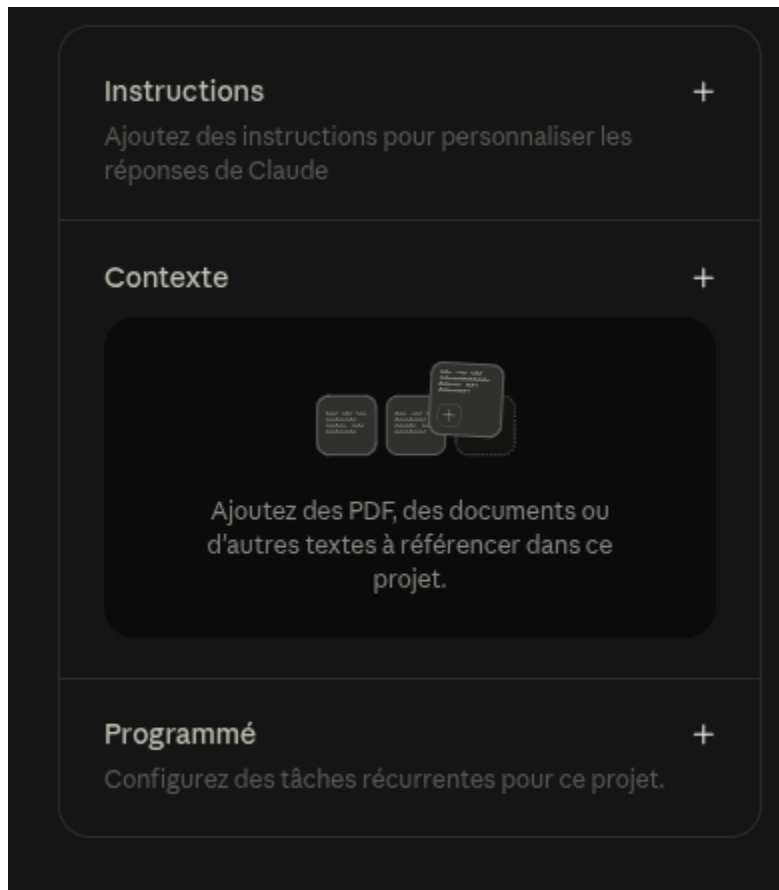


FIGURE 10 – Le panneau d'un projet neuf : « Instructions », « Contexte » et « Programmé ».

6.4 Que mettre dans la base de connaissances (le « Contexte ») ?

- le **cahier des charges** et les documents de conception déjà produits ;
- les **articles scientifiques** (PDF) sur lesquels s'appuient les choix ;
- les **résultats** d'expériences (exports, rapports) ;
- les **documents de reprise** de chaque session précédente ;
- le **journal de bord** et le **registre des décisions** (voir guide 2).

Pour CREDIX, la première séance de travail a justement consisté à **faire l'inventaire des ressources du projet** et à rédiger les instructions du projet, avant de poser la moindre question de fond.

Comment ajouter des fichiers ? Dans la page du projet, cliquez sur le bouton « + » à droite de « **Contexte** » (le texte indique « Ajoutez des PDF, des documents ou d'autres textes à référencer dans ce projet »), puis choisissez vos fichiers sur l'ordinateur. Chaque fichier apparaît sous la forme d'une carte avec son nom, sa taille et son type (TXT, MD, PDF...). Une **barre de capacité** indique la part déjà utilisée (par exemple « 58 % de la capacité du projet utilisée ») ; quand le contenu devient volumineux, le « **Mode de recherche** » s'active : Claude cherche alors les passages utiles au lieu de tout relire.

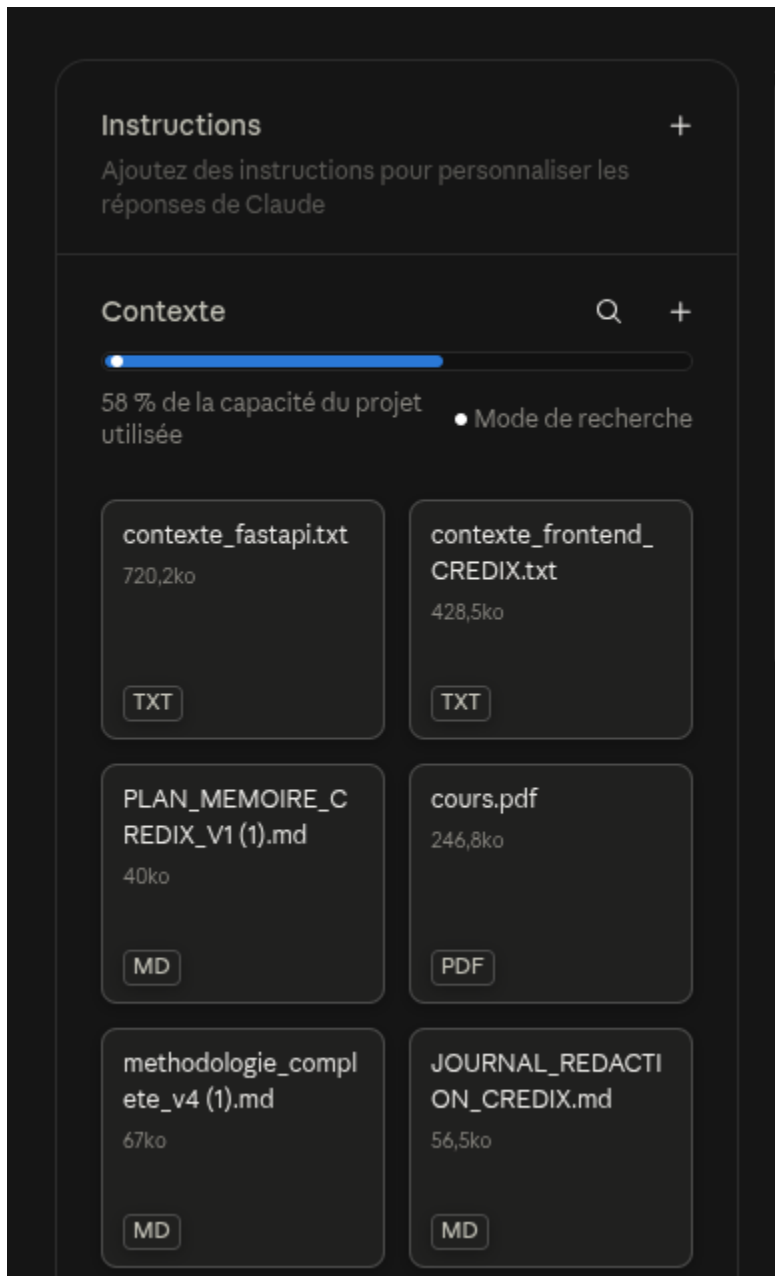


FIGURE 11 – Le « Contexte » d'un projet : les fichiers ajoutés, la barre de capacité et le mode de recherche.

7. Claude Code : installer et utiliser l'assistant dans VS Code

7.1 Ce que fait Claude Code

Claude Code est l'assistant qui **travaille dans votre projet** : il lit les fichiers, comprend l'organisation du code, propose et effectue des modifications, lance des commandes (tests, installation) et sait utiliser Git. Vous lui parlez en français, comme dans `claude.ai`, mais **il agit sur vos fichiers**.

7.2 Ce qu'il faut avant d'installer

- **VS Code, version 1.94 ou plus** (menu *Aide*, puis *À propos*, pour voir votre version) ;
- un **abonnement Claude payant** (Pro, Max, Team ou Enterprise), ou un compte développeur « Console » : aucune clé d'API n'est nécessaire avec un abonnement.

7.3 Installer l'extension dans VS Code

1. Ouvrez **VS Code**.
2. Ouvrez la vue des **extensions** : cliquez sur l'icône des extensions dans la barre verticale de gauche (quatre carrés, dont un détaché), ou tapez **Ctrl + Maj + X** (sur Mac : **Cmd + Maj + X**). Le panneau « **EXTENSIONS: MARKETPLACE** » s'ouvre.
3. Cliquez dans la barre de recherche du panneau et tapez **claude code**.
4. Dans la liste des résultats, repérez l'extension « **Claude Code for VS Code** ». Son éditeur est **Anthropic**, avec un **badge bleu de vérification** à côté du nom. Attention : la liste contient aussi des extensions d'autres éditeurs, au nom presque identique (par exemple « Chat for Claude Code » ou « Claude Code Assistant for VSCode »). **Ne choisissez pas celles-là.**
5. Cliquez sur l'extension officielle pour ouvrir sa page, puis sur le bouton bleu « **Install** » (« Installer »).
6. Attendez la fin de l'installation. Si le message « **Restart Required** » (redémarrage requis) s'affiche à côté du nom de l'extension, redémarrez VS Code : fermez-le puis rouvrez-le.
7. Pour vérifier : l'extension figure dans la liste des extensions installées, et une **icône d'étincelle** apparaît dans VS Code (section 7.4).

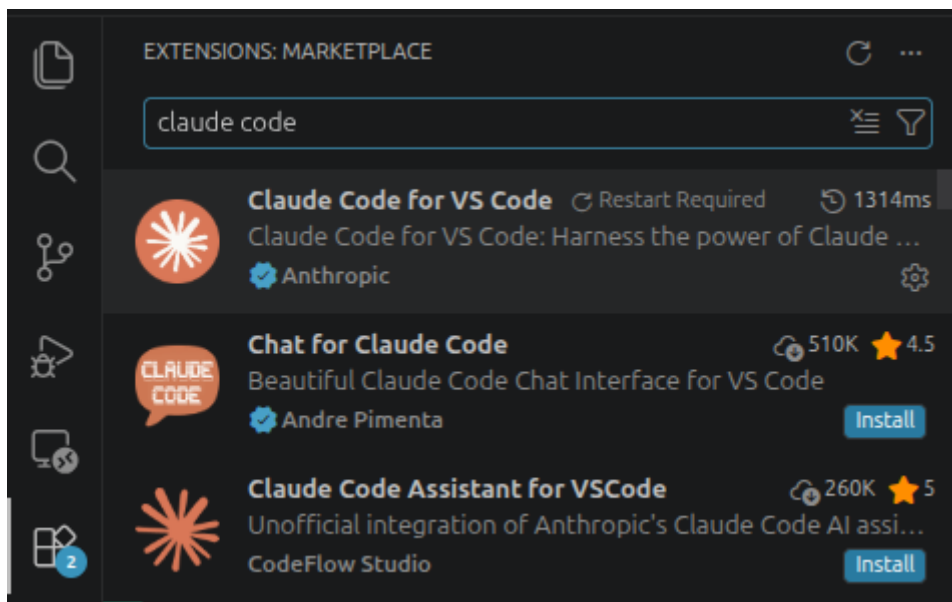


FIGURE 12 – Recherche « claude code » : l'extension officielle a pour éditeur Anthropic et un badge bleu ; les autres viennent d'autres éditeurs.

7.4 Ouvrir Claude Code et se connecter

Ouvrir le panneau. Trois façons, au choix :

1. **Depuis l'éditeur.** Ouvrez d'abord un fichier de votre projet (l'icône n'apparaît que lorsqu'un fichier est ouvert), puis cliquez sur l'**icône d'étincelle**, en haut à droite de l'éditeur.
2. **Depuis la barre d'activité** (la barre verticale de gauche) : cliquez sur l'icône d'étincelle, qui ouvre la liste de vos sessions.
3. **Depuis la palette de commandes** : tapez **Ctrl + Maj + P**, écrivez « Claude Code » et choisissez « **Open in New Tab** » (raccourci : **Ctrl + Maj + Échap**).

Le panneau « **Claude Code** » s'ouvre dans un onglet de l'éditeur.

Se connecter (au premier lancement seulement).

1. Le panneau vous demande de vous connecter : cliquez sur le bouton de connexion proposé.
2. Votre **navigateur** s'ouvre sur une page de claude.ai. Vérifiez que c'est bien le compte qui possède l'abonnement payant, puis autorisez l'accès avec le bouton de la page.
3. Revenez dans **VS Code** : le panneau affiche maintenant la zone de saisie. Vous êtes connecté.

Lire le panneau. L'extension s'affiche **en anglais**. De haut en bas : le titre « Claude Code » ; au milieu, parfois un petit guide « Learn Claude Code » (une liste d'étapes) et des messages d'information, par exemple « Auto mode is enabled », que l'on ferme avec la croix ; en bas, la **zone de saisie**. Sous elle se trouvent, de gauche à droite : le bouton « + » (joindre un élément), le bouton « / » (commandes), le **nom du modèle** avec son niveau d'effort (par exemple « Sonnet 5 Extra high »), puis, à droite, le **mode en cours** (par exemple « Auto ») et le **bouton d'envoi** (une flèche).



FIGURE 13 – L'icône d'étincelle, à l'extrémité de la barre d'outils de l'éditeur.

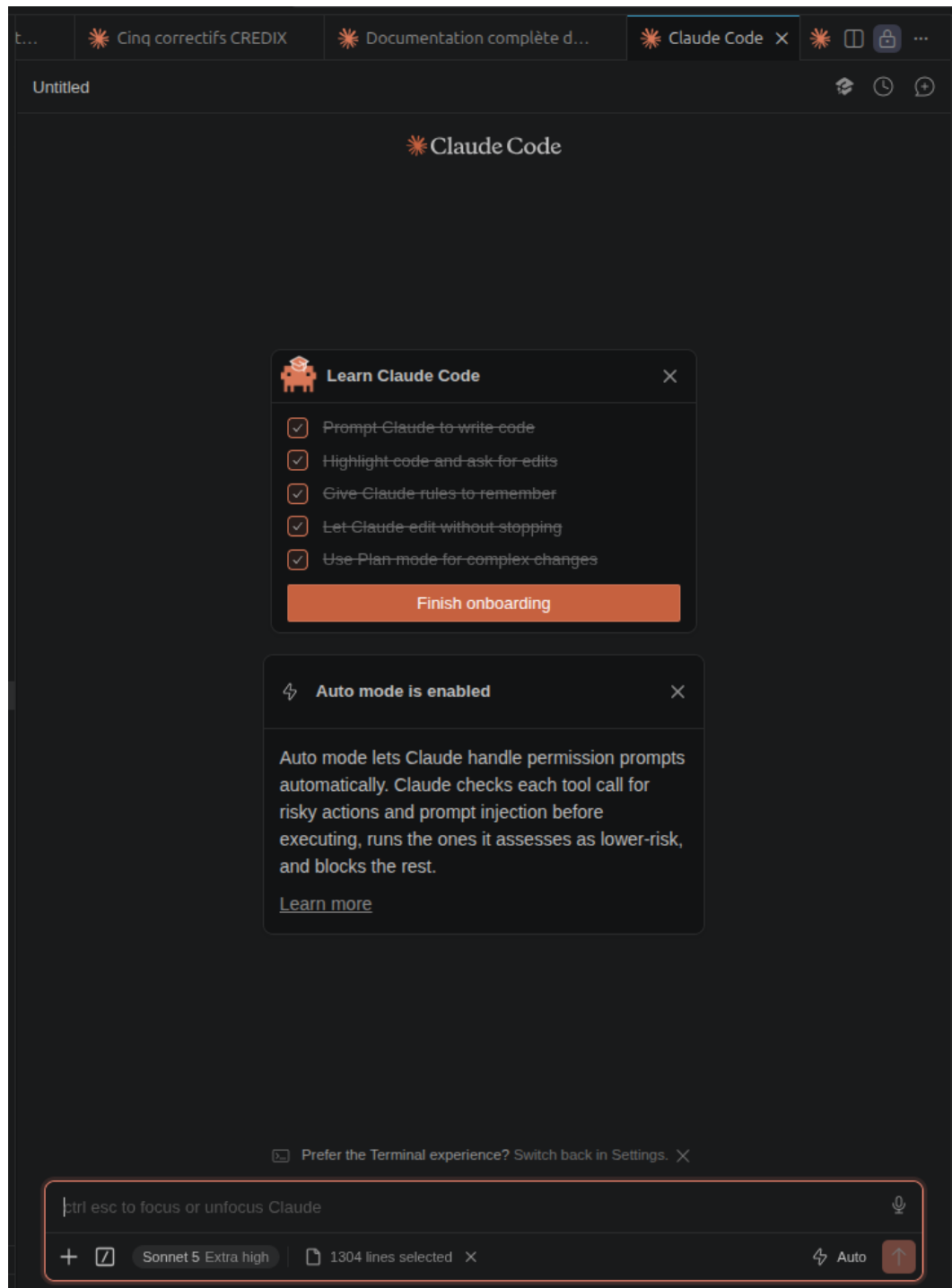


FIGURE 14 – Le panneau Claude Code : messages d'accueil, puis zone de saisie et réglages en bas.

Alternative en ligne de commande. Claude Code s'installe aussi dans un terminal. Sous Linux, macOS ou WSL : `curl -fsSL https://claude.ai/install.sh | bash`. Puis `claude --version` vérifie l'installation, et la commande `claude`, tapée dans le dossier d'un projet, démarre l'assistant (connexion au premier lancement).

7.5 Première utilisation

1. **Ouvrez le dossier de votre projet** : menu « **Fichier** » (*File*), puis « **Ouvrir le dossier...** » (*Open Folder...*) ; choisissez le dossier et validez. Son contenu apparaît dans la colonne de gauche de VS Code.

2. **Ouvrez le panneau Claude Code** en cliquant sur l'icône d'étincelle (section 7.4).
3. **Cliquez dans la zone de saisie**, en bas du panneau, et tapez une première question :

| Que fait ce projet ? Explique-moi son organisation en quelques lignes.

4. **Envoyez** avec la touche Entrée, ou avec le bouton en forme de flèche à droite de la zone de saisie.
5. **Suivez le travail de Claude** : le panneau affiche ce qu'il fait au fur et à mesure (par exemple la lecture de fichiers), puis sa réponse rédigée. Selon le mode choisi (section 8), il peut vous demander une autorisation avant une action : lisez la demande, puis acceptez ou refusez.
6. **Continuez la conversation** en écrivant dans la même zone de saisie.

Claude lit les fichiers dont il a besoin : vous n'avez pas à les lui fournir un par un. Pour lui désigner un fichier précis, tapez le symbole @ suivi du début du nom du fichier, puis choisissez-le dans la liste qui s'affiche.

Quelques commandes utiles, à taper dans la zone de saisie :

Commande	Effet
/help	Affiche l'aide et les commandes disponibles
/clear	Efface la conversation courante (repartir d'un contexte propre)
/resume	Reprend une conversation précédente
/plan devant une demande	Fait d'abord un plan, sans rien modifier (section 8)
/init	Crée automatiquement un fichier CLAUDE.md décrivant le projet (voir ci-dessous)

7.6 Le fichier CLAUDE.md : la mémoire du projet

Chaque nouvelle conversation de Claude Code démarre « à blanc ». Pour qu'il connaisse les règles de **votre** projet, on les écrit dans un fichier CLAUDE.md, placé à la racine du dossier : Claude le lit au début de chaque conversation.

On y met, par exemple : les commandes pour lancer et tester le projet, les conventions de code, les décisions d'architecture, les interdits (« ne jamais modifier tel dossier »). La commande /init en produit une première version automatiquement, que l'on complète ensuite.

8. Les modes de Claude Code, dont le mode Plan

8.1 Les modes de permission

Claude Code peut lire, modifier des fichiers et lancer des commandes. Le **mode de permission** règle ce qu'il peut faire **sans vous demander**. Pour le changer :

1. repérez le **nom du mode en cours**, en bas à droite de la zone de saisie (par exemple « Auto ») ;
2. cliquez dessus : la liste « **Modes** » s'ouvre, avec pour chaque mode une courte description ;
3. cliquez sur le mode voulu. Sous la liste, un curseur « **Effort** » règle l'effort de réflexion de Claude.

Raccourci : Maj + Tab passe au mode suivant (dans le terminal aussi). Dans l'extension, les modes s'affichent **en anglais** : *Manual*, *Edit automatically*, *Plan* et *Auto*.

Mode	Ce qui s'exécute sans demander	Quand l'utiliser
Manuel (<i>default</i>)	La lecture seulement : tout le reste demande votre accord	Travail sensible, on veut tout relire
Modifier automatiquement (<i>acceptEdits</i>)	Lecture, modification de fichiers et commandes simples (créer un dossier, copier...)	On relit le code au fur et à mesure
Plan (<i>plan</i>)	Lecture seulement : Claude explore et propose un plan, sans rien modifier	Avant de changer quoi que ce soit
Auto (<i>auto</i>)	Presque tout, avec des vérifications de sécurité en arrière-plan	Longues tâches, moins d'interruptions
Contournement (<i>bypassPermissions</i>)	Tout, sans aucune vérification	À éviter (réservé à des machines isolées)

Sur les abonnements Pro, Max et Team, le mode de départ est **Auto**. Les libellés peuvent légèrement varier selon la version de l'extension.

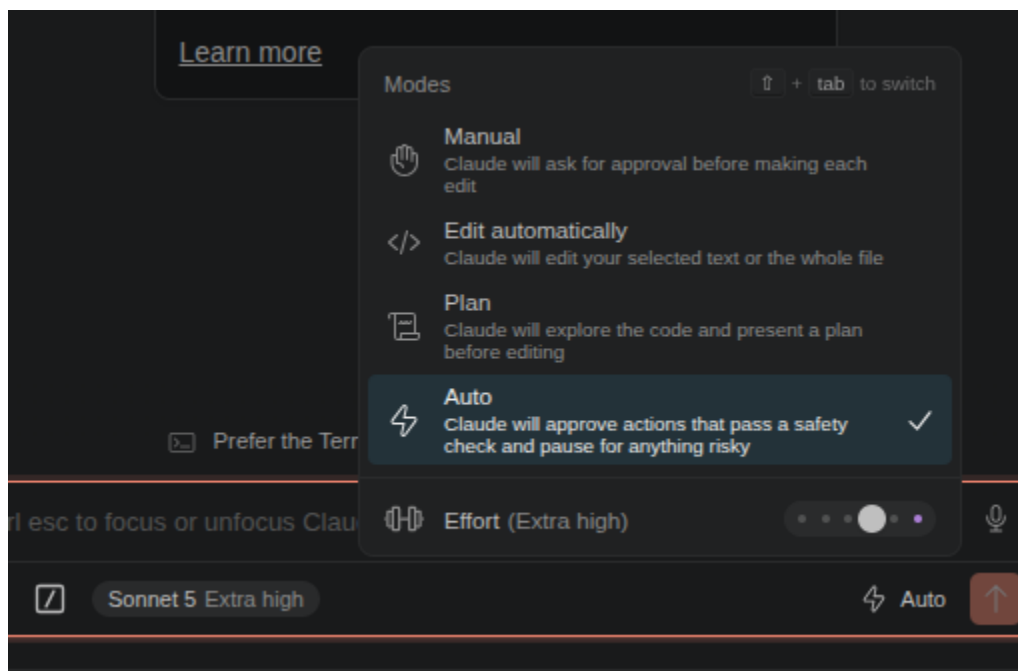


FIGURE 15 – La liste « Modes » : Manual, Edit automatically, Plan, Auto, et le curseur « Effort ».

8.2 Le mode Plan : réfléchir avant d'agir

En **mode Plan**, Claude Code **explore le projet et rédige un plan d'action, mais ne modifie rien**. C'est le mode à utiliser :

- **au début d'un travail** : à partir d'un document de conception, il produit la **liste des tâches à réaliser** (la « TODO ») ;

- **avant chaque ajout ou modification** d'un module : on relit ce que Claude compte faire, on corrige, puis on valide.

Pas à pas :

1. **Passez en mode Plan.** Cliquez sur le nom du mode en cours (en bas à droite de la zone de saisie), puis sur « **Plan** » dans la liste. Autres façons : taper `/plan` au début de votre demande, ou appuyer sur Maj + Tab jusqu'à ce que « **Plan** » s'affiche. Vérifiez que le nom du mode indiqué est bien « **Plan** ».
2. **Écrivez votre demande** dans la zone de saisie, par exemple : « Lis le document docs/guides/03_conception_MVP.md et propose un plan de réalisation, module par module. » Envoyez avec **Entrée**.
3. **Attendez.** Claude lit les fichiers et réfléchit. À ce stade, **aucun fichier n'est créé ni modifié**.
4. **Lisez le plan.** Il s'ouvre comme un **document Markdown** dans VS Code : un titre, des parties, une liste d'étapes. Lisez-le en entier, comme vous relirez un document de conception.
5. **Faites corriger si besoin.** Vous pouvez ajouter des commentaires dans le plan, à côté des passages à corriger, comme dans une relecture. Sinon, écrivez vos remarques dans la zone de saisie : Claude révisé le plan.
6. **Approuvez quand le plan vous convient.** Trois réponses sont proposées :
 - **Oui, en mode Auto** : Claude passe à la réalisation avec un minimum d'interruptions ;
 - **Oui, en approuvant chaque modification à la main** (recommandé au début) : Claude demande votre accord avant chaque changement ;
 - **Non, continuer à planifier** : on reste en mode Plan et on précise ce qu'il faut changer.
7. **Gardez le plan.** Une fois validé, il sert de **liste des tâches (TODO)** du projet : copiez-le dans un fichier du dossier du projet (guide 2, section 11.1).

Règle pratique. Un travail qui touche plusieurs fichiers commence **toujours** par un plan. C'est moins cher (pas de fausses pistes) et plus sûr (vous validez avant que le code change).

9. Bonnes pratiques et précautions

9.1 Bien demander

- **Soyez précis.** Au lieu de « corrige le bug », écrivez : « corrige le bug où l'écran reste blanc après une erreur de mot de passe ».
- **Procédez par étapes.** Découpez un gros travail : une base de données, puis les routes, puis l'écran.
- **Faites explorer d'abord.** « Analyse la structure de la base de données » avant « ajoute un tableau de bord ».
- **Parlez comme à un collègue** : décrivez l'objectif, pas seulement la tâche.

9.2 Précautions

- **Ne collez jamais de secrets** (mots de passe, clés d'API, contenu d'un fichier `.env`) dans une conversation.
- **Relisez les modifications** avant de les accepter (VS Code montre les différences ligne par ligne).
- **Travaillez sous Git** : chaque étape validée est enregistrée, on peut revenir en arrière.
- **Vérifiez ce qui compte** : chiffres, références, résultats de tests. Claude peut se tromper, y compris avec assurance.

- **Données réelles de clients** : ne les confiez pas à un outil externe sans autorisation. CREDIX a été conçu sur un jeu de données public pour cette raison.

10. Glossaire et sources officielles

10.1 Glossaire

Terme	Explication simple
Anthropic	L'entreprise qui développe Claude
Abonnement	Formule payante (Pro, Max, Team, Enterprise) qui donne accès à plus d'usage
Session (d'usage)	Période de 5 heures au bout de laquelle la limite de session est remise à zéro
Session (de travail)	Moment de travail suivi sur un objectif, qui se conclut par un document de reprise
Contexte	Ce que Claude « a en mémoire » dans une conversation
Projet	Espace de travail regroupant conversations, instructions et documents
Base de connaissances	Les documents chargés dans un projet
RAG	Technique où Claude cherche les passages utiles dans une grande masse de documents, au lieu de tout lire
Claude Code	L'assistant de programmation qui travaille dans vos dossiers
Mode Plan	Mode où Claude propose un plan sans rien modifier
CLAUDE.md	Fichier de consignes permanentes d'un projet, lu à chaque conversation
Markdown (.md)	Format de texte simple avec titres et listes ; c'est le format « naturel » de Claude

10.2 Sources officielles (consultées le 19 septembre 2026)

- Abonnement Pro : <https://support.claude.com/en/articles/8325606-what-is-the-pro-plan>
- Abonnement Max : <https://support.claude.com/en/articles/11049741-what-is-the-max-plan>
- Abonnement Team : <https://support.claude.com/en/articles/9266767-what-is-the-team-plan>
- Abonnement Enterprise : <https://support.claude.com/en/articles/9797531-what-is-the-enterprise-plan>
- Limites d'usage et de longueur : <https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work>
- Projets : <https://support.claude.com/en/articles/9517075-what-are-projects>
- Recherche web, réflexion étendue et recherche approfondie : <https://support.claude.com/en/articles/11095361-when-should-i-use-web-search-extended-thinking-and-research>
- Claude Code dans VS Code : <https://code.claude.com/docs/en/vs-code>
- Claude Code, démarrage rapide : <https://code.claude.com/docs/en/quickstart>
- Modes de permission (dont le mode Plan) : <https://code.claude.com/docs/en/permission-modes>
- Mémoire du projet (CLAUDE.md) : <https://code.claude.com/docs/en/memory>

PROJET DE FIN D'ÉTUDES

École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique

Année académique 2025–2026

Guide 2 : la méthode de conception

Concevoir une application avec Claude, de A à Z

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Table des matières

Guide 2 : la méthode de travail avec Claude, de A à Z	1
1. Le principe en une page	1
2. Vue d'ensemble : trois niveaux et neuf étapes	2
3. Le cycle de session : ouvrir, travailler, fermer	3
4. Les documents qui font tenir le projet	5
5. Étape 0 : préparer l'espace de travail	8
6. Étape 1 : du besoin au cahier des charges	9
7. Étape 2 : la recherche dans la littérature	10
8. Étape 3 : les choix méthodologiques, exemple du WoE	11
9. Étape 4 : choisir les langages et les technologies	13
10. Étape 5 : le document de conception	14
11. Étapes 6 et 7 : Claude Code, du plan au prototype testé	14
12. Étape 8 : livrer, lancer, tester à la main	16
13. Étape 9 : modifier, documenter, transmettre	16
14. Les erreurs et les pièges réels	17
15. Bibliothèque de prompts	17
16. Modèles de documents	18
17. Rejouer la méthode : le mini-exercice	19

Guide 2 : la méthode de travail avec Claude, de A à Z

Comment conduire un projet d'application avec Claude, du besoin à l'application lancée. Illustrée par la conception réelle de CREDIX.

Comment lire ce guide. Les blocs intitulés « **Prompt** » sont à copier dans Claude, après avoir remplacé ce qui est entre crochets [...]. Les encadrés « **Dans CREDIX** » montrent ce qui a été réellement fait, d'après les documents du projet (instructions du projet, journaux de sessions et de rédaction, rapports, documents de passation). Les prompts marqués « **extrait réel** » sont tirés de ces fichiers ; les autres sont des prompts types, construits sur le même modèle.

Ce guide suppose que vous avez lu le **guide 1** (abonnement, mode Projet, Claude Code).

1. Le principe en une page

Concevoir une application avec Claude ne consiste pas à lui dire « fais-moi une application de crédit ». Cela donne un résultat fragile, sans traçabilité. La méthode qui a servi pour CREDIX repose sur **six principes**.

Principe	Ce que cela veut dire concrètement
1. On conçoit avant de coder	Un cahier des charges, puis une conception écrite, avant la première ligne de code. Règle du projet professionnel : « <i>ne jamais coder avant d'avoir clarifié la conception</i> »
2. Claude propose, l'humain décide	Claude explique ce qu'il va produire avant de le produire. Aucun livrable final sans accord explicite
3. Une seule tâche à la fois	Le travail est découpé ; chaque tâche suit les mêmes étapes

Principe	Ce que cela veut dire concrètement
4. Chaque décision est écrite	Un registre consigne le choix, les alternatives écartées, la justification et le niveau de preuve
5. Tout passe par des fichiers Markdown	Le travail d'une session devient un document, qui alimente la session suivante
6. On vérifie par l'exécution	Chaque module est testé ; les commandes de lancement sont rejouées sur une machine neuve

Pourquoi le Markdown (.md) ? Claude écrit naturellement en Markdown : du texte simple, avec titres, listes et tableaux. Il est lisible par une personne **et** par un assistant comme Claude Code, comparable ligne à ligne dans Git, et convertible ensuite en PDF, en Word ou en présentation. **On garde toujours le .md comme source**, même si le livrable final est un PDF (les documents de ce dossier sont produits ainsi, avec LaTeX : voir le guide 4).

Le fil rouge de CREDIX, écrit dans le mémoire : « *chaque bloc du pipeline sert un objectif précis, et on le prouve.* » La même exigence vaut pour la méthode : *chaque étape produit un document vérifiable.*

2. Vue d'ensemble : trois niveaux et neuf étapes

2.1 Les trois niveaux de la méthode

Niveau	Ce que c'est	Où en parle-t-on
A. Les outils	Un Projet claude.ai (instructions et base de connaissances), Claude Code dans VS Code, des fichiers Markdown	Guide 1
B. Le cycle de session	Une façon d'ouvrir, de conduire et de fermer chaque séance de travail, pour ne jamais perdre le fil	Section 3
C. Les documents du projet	Instructions, journal, registre de décisions, pièges, TODO, documents de passation : la mémoire du projet	Section 4

2.2 Les neuf étapes d'un projet

Étape	Ce qu'on fait	Où	Livrable
0	Préparer l'espace : Projet Claude, instructions, journal	claude.ai	Projet configuré, journal
1	Passer du besoin au cahier des charges	claude.ai (Projet)	cahier_des_charges.md
2	Chercher dans la littérature	claude.ai (recherche approfondie) + articles PDF	Fiches de lecture, références

Étape	Ce qu'on fait	Où	Livrable
3	Faire les choix méthodologiques et les consigner	claude.ai (Projet)	Registre des décisions
4	Choisir les langages et technologies	claude.ai (Projet)	Section « choix techniques »
5	Rédiger le document de conception (UML, flux, données)	claude.ai (Projet)	conception.md
6	Passer à Claude Code : mode Plan , TODO	VS Code	Plan validé
7	Prototyper module par module , avec tests	VS Code	Modules testés
8	Livrer : README de lancement, tests à la main	VS Code + terminal	Projet lançable
9	Modifier , documenter, transmettre	VS Code (mode Plan)	Documents de reprise

Chaque étape est conduite à l'intérieur d'une **session**, selon le cycle de la section suivante.

3. Le cycle de session : ouvrir, travailler, fermer

Un projet long se déroule sur **de nombreuses sessions**. Claude ne se souvient pas d'une session à l'autre : c'est **vous** qui organisez la continuité, avec des documents. Toutes les sessions de CREDIX suivent le même cycle.

3.1 Ouvrir la session : le prompt de démarrage

Principe. On ne pose pas de question tout de suite. Le premier message impose **ce que Claude doit lire, dans quel ordre**, et lui interdit de commencer avant votre accord.

Prompt de démarrage (extrait réel, phase de rédaction du chapitre 2 de CREDIX)

Voici la suite du travail sur le mémoire CREDIX, chapitre 2. Avant de répondre quoi que ce soit, lis dans cet ordre, entièrement :

1. HANDOVER_SESSION_SUIVANTE.md : à jour, fin de la session précédente.
2. JOURNAL_REDACTION_CREDIX_CH2_A_JOUR.md : lis en entier la section « Pièges méthodologiques déjà identifiés » avant toute chose : ce ne sont pas des recommandations générales, ce sont des erreurs déjà commises dans ce projet.
3. chap02_en_cours.tex : le chapitre en cours.
4. TODO_CHAPITRE2_CREDIX_MAJ.md : le plan détaillé à jour. Le journal fait foi en cas de désaccord entre les deux fichiers.
5. Dans le project knowledge : [les rapports à lire en entier avant d'écrire].
6. Le notebook joint : nécessaire pour vérifier tout chiffre cité.

Avant d'ouvrir la section suivante, j'ai des remarques groupées sur ma relecture : je te les donne d'abord, on les traite, puis on passe à la suite. Ne commence pas la rédaction avant que je te dise que c'est bon.

Ce que ce prompt impose, et pourquoi :

Élément	Effet
Ordre de lecture explicite	Claude part du même état que vous, sans oublier les documents essentiels
Le journal « fait foi » en cas de désaccord	Il y a une source de vérité
Lire d'abord les pièges	On évite de refaire des erreurs déjà commises
« Ne commence pas avant que je te dise que c'est bon »	Vous gardez la main : Claude propose, vous décidez

Variante pour un projet de programmation (extrait réel, projet professionnel)

Tu es mon pair-programmer principal. Règles absolues : tu travailles une seule tâche à la fois ; tu respectes l'architecture existante et les conventions ; tu ne codes jamais avant d'avoir clarifié la conception. Méthode obligatoire en 5 étapes : 1) analyse de l'existant, 2) conception fonctionnelle, 3) conception technique, 4) mise en place, 5) tests. Je vais te fournir le contexte (fichiers, TODO, éventuellement un rapport de passation). Attends mes instructions.

Et le message de démarrage de ce même kit : « *Ta première mission AVANT toute action : 1. lire CONTEXT.md pour comprendre la stack et les fichiers clés ; 2. lire TODO.md et identifier les tâches non cochées ; 3. me dire quelle est la prochaine tâche logique ; 4. attendre ma validation avant de commencer. **Ne propose aucun code pour l'instant.*** »

3.2 Conduire la session : une tâche à la fois, en cinq étapes

Pendant la session, on respecte les règles inscrites dans les instructions du projet (voir section 4.1) :

- **Claude explique chaque initiative avant de produire quoi que ce soit.**
- **On travaille sur une seule tâche**, définie au début de la session, et on ne touche pas au reste. Exemple réel : « *Cette session a un seul objectif : exécuter le bloc A de la TODO. Ne pas toucher au backend. Ne pas travailler sur le mémoire. Uniquement le notebook.* »
- Pour une tâche technique, on suit les **cinq étapes** : **analyse de l'existant** → **conception fonctionnelle** → **conception technique** → **mise en place** → **tests**.
- Pour une tâche de rédaction, on procède **sous-section par sous-section** : décider ce qui entre, ce qui sort, ce qui part dans un autre chapitre ; proposer un sous-plan ; discuter le contenu ; rédiger ; faire une vérification technique courte ; réserver l'**audit critique complet** aux sous-sections à risque, identifiées ensemble.
- **Claude signale quand le contexte devient chargé** et propose de changer de session. Si vous acceptez, il prépare la documentation de clôture.

3.3 Fermer la session : le rapport de passation

Prompt de fin de session (extrait réel, projet professionnel)

Nous allons bientôt changer de session. Génère-moi un « Handoff Report » (résumé de passation) très concis. Il doit contenir :

1. Ce qui a été complété avec succès aujourd'hui.
2. Ce qui est actuellement en cours (avec le fichier exact et la ligne si possible).
3. La prochaine étape immédiate selon notre méthode en 5 étapes.
4. Les éventuels points d'attention ou bugs laissés en suspens.

Ne génère pas de code, juste ce rapport en markdown.

La règle de clôture des instructions du projet de CREDIX : *les livrables de clôture ne sont produits qu'après l'accord explicite de l'auteur, et Claude explique ce qu'il va produire avant de le produire.* Ce sont :

1. les **instructions du projet** mises à jour, si elles ont changé ;
2. une **nouvelle entrée du journal de bord**, datée, en **ajout seulement** (on ne réécrit jamais le passé) ;
3. le **plan** et la **TODO** mis à jour ;
4. le **document de passation**, qui devient l'entrée de la session suivante.

Ces documents sont ensuite **déposés dans le projet Claude** (ou dans le dossier du projet de code) : c'est ce qui rend la session suivante possible.

Dans CREDIX. Le résultat est visible dans le dépôt : des documents de passation (HANDOVER_...), un journal de bord, des rapports de session par sujet (par exemple le rapport de la séance sur le flux de contrôle des profils atypiques), une « copie miroir » des notes que Claude conserve d'une session à l'autre, et des prompts de démarrage propres à chaque type de séance (notebook, rédaction du mémoire, codage).

4. Les documents qui font tenir le projet

4.1 Les instructions du projet (six sections)

C'est le document le plus important : il est chargé **une fois** dans le Projet Claude et s'applique à **toutes** les conversations. Voici la structure réelle de celui de CREDIX.

Section	Contenu	Pourquoi
1. Contexte	Qui, quoi, pour qui, dates clés, encadrants, sujet, état d'avancement	Claude sait dans quel cadre il travaille
2. Ressources permanentes	La liste classée des documents du projet, avec pour chacun « consulter pour... » : modèles de référence, documents techniques, normes de rédaction, norme bibliographique, modèle LaTeX, version courante du mémoire	Claude sait où chercher selon la tâche
3. Décisions techniques figées	Treize décisions prises, à ne remettre en question que sur contradiction logique majeure (modèle final, ordre des étapes du pipeline, traitement des manquants, découpage des données, métriques principales, pile technique...)	Évite de rediscuter sans cesse les mêmes choix
4. Consignes permanentes	Langue, niveau attendu, style, règles anti-plagiat, outil pour les diagrammes, cohérence code/rapport ; « à faire systématiquement » et « ce que Claude ne doit pas faire »	Uniformise toutes les réponses

Section	Contenu	Pourquoi
5. Workflow de collaboration	Ce qui se passe au début, pendant et à la fin de chaque session (voir section 3)	Le cycle de session est écrit noir sur blanc
6. État courant	Où en est le projet, mis à jour à chaque session	Claude connaît la situation

Extrait réel de la section 4 (consignes permanentes) :

À faire systématiquement :

- Lire le journal de bord en début de chaque session
- Consulter [le document de méthodologie] avant tout chapitre technique
- Consulter [le rapport de pipeline] pour tous les résultats numériques
- Signaler immédiatement tout doublon détecté dans la base de connaissances
- Demander l'OK explicite de l'auteur avant de produire les livrables finaux

Ce que Claude ne doit pas faire :

- Rédiger sans avoir consulté les ressources pertinentes
- Produire des livrables sans OK explicite de l'auteur

Extrait réel de la section 5 (workflow) :

En début de session : Claude lit le journal ; propose un objectif de session ; l'auteur confirme ou ajuste.

Pendant la session : on itère ; Claude explique chaque initiative avant de produire.

En fin de session : Claude produit les livrables de clôture UNIQUEMENT après OK explicite : instructions à jour, journal (nouvelle entrée datée, ajout seulement), plans à jour.

Règle fondamentale : aucun livrable final n'est produit sans OK explicite.

4.2 Les autres documents, en un tableau

Document	Rôle	Règle d'or	Exemple dans CREDIX
Journal de bord	Une entrée par session : <i>réalisé, décisions prises, en attente</i>	Ajout seulement , jamais de réécriture du passé	journal_sessions_memoire_hendrix.txt
Journal de rédaction	État consolidé d'un chapitre : plan, règles de rédaction actées, décisions verrouillées, pièges , points ouverts	Fait foi en cas de désaccord avec la TODO	JOURNAL_REDACTION_CREDIX_CH2_A_JOUR.md
Registre des décisions	Chaque choix : options, justification, niveau de preuve	Toute décision importante y figure	RAPPORT_DECISIONS_METHODOLOGIQUES_CREDIX.md
Rapport de résultats	Résultats validés, consolidés	Aucun chiffre inventé : il vient d'un calcul	RAPPORT_RESULTATS_CONCLUSIONS_CREDIX.md

Document	Rôle	Règle d'or	Exemple dans CREDIX
Pièges méthodologiques	La liste des erreurs déjà commises	À relire avant de reprendre le travail	Section 4bis du journal de rédaction
TODO	Le plan détaillé, tâche par tâche	Une tâche = une case à cocher	TODO_...md
Document de passation	Ce qui est fait, en cours, à faire, règles absolues	Écrit à la fin de chaque session ou phase	HANDOVER_...md
État miroir	Copie des notes que Claude garde d'une session à l'autre	Doit rester identique à la mémoire interne	ETAT_MEMOIRE_CLAUDE_CREDIX.md
Instantané de contexte	Un fichier qui résume tout le code d'un projet, fabriqué par un script	À régénérer avant d'en parler à Claude	scanner.sh produit contexte_fastapi.txt

4.3 Deux exemples réels qui montrent la valeur de ces documents

Des règles de rédaction « actées » (extraits du journal de rédaction, sur trente-huit règles) :

- « *Un paragraphe = une fonction argumentative.* »
- « *Aucun composant sans fonction énonçable en une phrase.* »
- « *Séparation stricte conception / résultats : jamais "meilleure" avant les résultats.* »
- « **Grille des quatre questions pour chaque bloc technique** : ① le problème ② la solution ③ pourquoi celle-là ④ comment elle est validée. »
- « *Terminologie : un seul terme, "dispositif de décision". Jamais "dispositif de contrôle".* »

Des pièges consignés, avec leur histoire (extraits de la section « Pièges méthodologiques ») :

- **Vérifier tout chiffre reçu d'une source externe, même quand le raisonnement est juste.**
Deux documents externes utilisaient une variable comme exemple, avec un indicateur d'une valeur de 0,13. Vérifié contre l'exécution réelle : la valeur réelle était de **0,0199**, dans une autre catégorie. Le raisonnement général restait correct et a été conservé ; **seul le chiffre était faux.**
- **Ne jamais présenter le système autrement qu'il n'est**, « même sur un détail », pour paraître plus conforme à la littérature.
- **Ne jamais dire « on a gardé le moins bon »** : une décision se justifie par des critères, pas par un aveu.

4.4 Comment les créer avec Claude

Prompt : rédiger le journal ou le document de passation

Fais le document de reprise de cette session en Markdown : 1) l'objectif de la session, 2) ce qui a été fait, 3) les décisions prises et leur justification, 4) ce qui a été écarté et pourquoi, 5) les fichiers créés ou modifiés,

6) ce qui reste à faire dans l'ordre, 7) les pièges à éviter. Sois factuel : aucun chiffre que nous n'avons pas obtenu.

Les modèles prêts à remplir sont en section 16.

5. Étape 0 : préparer l'espace de travail

5.1 Ce qu'il faut mettre en place

1. **Un Projet Claude** (guide 1, section 6), avec ses **instructions en six sections** (section 4.1 de ce guide).
2. **Un journal de bord**, en ajout seulement.
3. **Un dossier de projet** sur l'ordinateur, sous **Git**, dès le début.

5.2 La toute première session

Dans CREDIX (27 mai 2026). La première session a servi à **initialiser le projet Claude** : inventaire des ressources de la base de connaissances (cahier des charges v2.0, conception v2.1, méthodologie complète v2, rapport de pipeline, modèles de mémoire de référence), **diagnostic** de l'existant (pipeline documenté à environ 75 %, backend à 0 %, frontend à environ 15 %), rédaction des instructions du projet, d'un plan du mémoire, d'un **plan de travail daté** et du **journal de bord**.

Prompt 0 : faire l'inventaire du projet

Fais l'inventaire de tous les documents de la base de connaissances de ce projet. Pour chacun : son titre, son rôle, sa date, et s'il est à jour ou périmé. Puis fais un diagnostic honnête : ce qui est prêt, ce qui manque, ce qui se contredit. Termine par les trois prochaines actions que tu recommandes.

Pas à pas :

1. Ouvrez votre Projet (colonne de gauche de claude.ai, « **Projets** », puis le nom du projet).
2. Dans la zone « **Contexte** », cliquez sur « + » et ajoutez vos documents (guide 1, section 6.4).
3. Ouvrez une **nouvelle conversation dans le projet** et collez le prompt 0 ci-dessus.
4. **Lisez l'inventaire** : vérifiez que chaque document est bien décrit, et corrigez ce qui est faux.
5. Demandez ensuite la rédaction des **instructions du projet** (section 4.1) et du **journal de bord** (section 16.2).

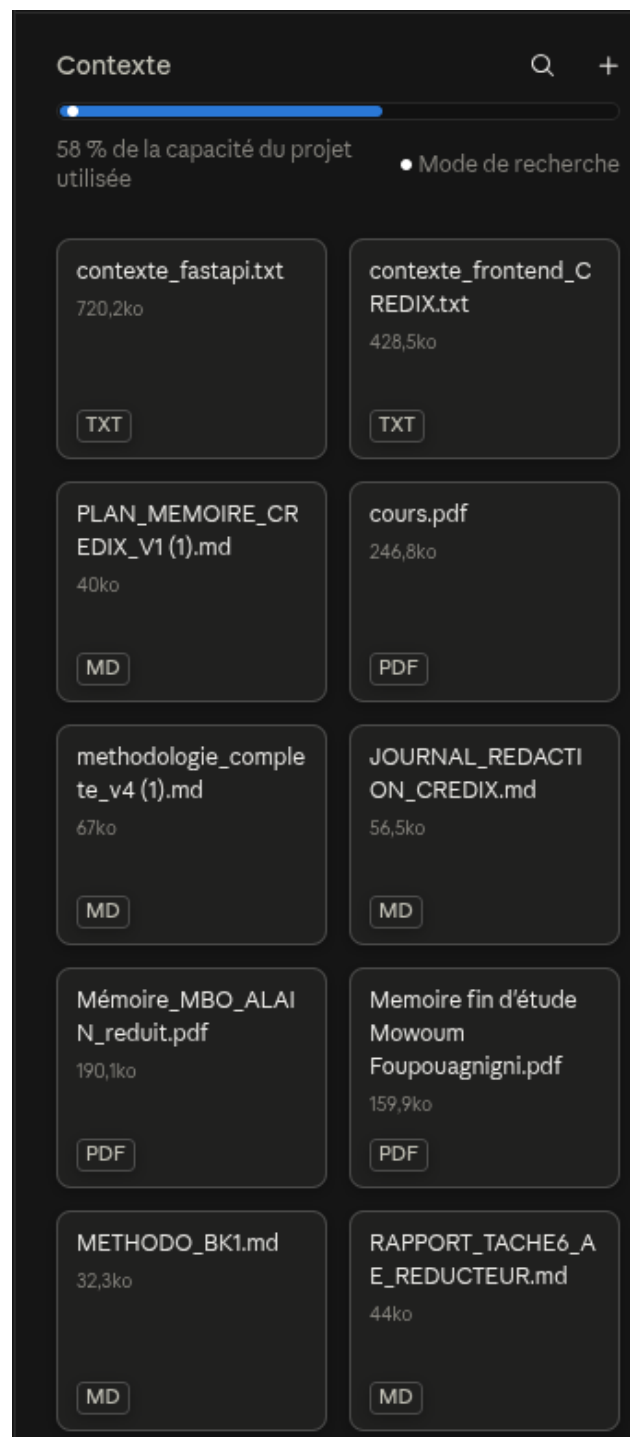


FIGURE 1 – Le « Contexte » du projet : les documents chargés (cahier des charges, méthodologie, journaux), la barre de capacité et le mode de recherche.

6. Étape 1 : du besoin au cahier des charges

6.1 Laisser Claude vous interroger

Le piège du débutant est de rédiger seul un long descriptif. Il vaut mieux **demandeur à Claude de vous poser des questions**, une à la fois.

Prompt 1 : cadrer le besoin par des questions

Je veux concevoir [décrire l'application en une phrase], pour [qui].
 Avant de rédiger quoi que ce soit, pose-moi une question à la fois pour clarifier : le problème à résoudre, les utilisateurs, ce que l'application doit faire, ce qu'elle ne doit pas faire, les contraintes (temps, budget, données, règles légales). Attends ma réponse avant la question suivante.
 Quand tu estimes avoir assez d'éléments, dis-le-moi.

6.2 Rédiger et faire critiquer le cahier des charges

Prompt 2 : rédiger le cahier des charges

À partir de notre échange, rédige le cahier des charges en Markdown avec :

1. Contexte et objectifs
2. Acteurs et leurs rôles
3. Périmètre (inclus) et hors périmètre (exclu)
4. Exigences fonctionnelles, numérotées EF-1, EF-2...
5. Exigences non fonctionnelles (performance, sécurité, traçabilité...), ENF-1...
6. Contraintes et hypothèses
7. Critères d'acceptation : pour chaque exigence, comment vérifier qu'elle est ↪ atteinte

Exigences courtes, vérifiables, sans ambiguïté.

Prompt 3 : faire critiquer le cahier des charges

Relis ce cahier des charges comme un examinateur sévère. Liste : les ambiguïtés, les contradictions, les exigences impossibles à vérifier, ce qui manque.
 Classe tes remarques par gravité. Ne corrige rien : propose seulement.

Dans CREDIX. Les exigences du mémoire sont numérotées (EM-1 à EM-9). Un cahier des charges séparé a été produit pour le frontend (cdc_credix.md : produit, trois rôles, lexique métier, plan de navigation de 36 routes) avant de dessiner les écrans.

7. Étape 2 : la recherche dans la littérature

Un choix technique important s'appuie sur la **littérature**. Claude peut faire une grande partie de ce travail, à condition de **ne rien croire sans vérifier**. On combine trois outils :

1. la **recherche approfondie** de claude.ai (guide 1, section 5.2), qui produit un rapport **avec citations** ;
2. les **articles PDF** chargés dans le Projet : Claude répond **à partir d'eux** ;
3. **votre vérification** : ouvrir les références clés.

Prompt 4 : balayer la littérature

Fais une recherche approfondie sur : [question précise, par exemple « méthodes de sélection de variables pour un scoring de crédit explicable »].
 Je veux : les approches principales, leurs avantages et limites, les références les plus citées (auteur, année, titre). Cite tes sources. Indique ce qui est un consensus et ce qui est discuté. Termine par les références dont tu n'es pas sûr, marquées [à vérifier].

Prompt 5 : interroger un article chargé dans le Projet

Dans l'article [titre ou auteur] chargé dans ce projet :

1. Quelle est la méthode proposée, en termes simples ?
2. Sur quelles données est-elle évaluée, et avec quels résultats ?
3. Quelles sont ses limites, d'après les auteurs et d'après toi ?
4. Que peut-on en retenir pour mon choix de [sujet] ?

Cite les passages sur lesquels tu t'appuies.

Les garde-fous : vérifier chaque référence importante ; faire marquer les références incertaines par [à vérifier] ; **distinguer** ce que dit la littérature, ce qu'ont montré vos essais et ce qui est un choix assumé (échelle de preuve, section 8).

Dans CREDIX. Une session de recherche approfondie a été conservée sous forme de PDF. Pour le seuil de 50 % de valeurs manquantes, la recherche a établi qu'il **n'existe pas de règle universelle** : le package de référence `scorecard` (langage R) fixe 95 % par défaut, et la littérature utilise des seuils très variables. Conclusion consignée : *ne pas attribuer ce seuil à un auteur* ; le présenter comme un **choix assumé**. Des références comme Chow (1970) ou El-Yaniv et Wiener (2010) sont restées marquées « à vérifier » tant qu'elles n'étaient pas contrôlées.

Pas à pas : lancer une recherche approfondie et l'exploiter.

1. Ouvrez une **nouvelle conversation** dans votre Projet (guide 1, section 6).
2. Cliquez sur le bouton « + » à gauche de la zone de saisie, puis sur « **Recherche** » (guide 1, section 5.2).
3. Collez le **prompt 4** ci-dessus, remplacez les crochets par votre question précise, puis envoyez avec Entrée.
4. **Attendez** la fin du travail : Claude enchaîne plusieurs recherches, en général pendant une à trois minutes.
5. **Lisez le rapport** : il est organisé en parties et contient des **citations**, c'est-à-dire des renvois vers les sources.
6. **Vérifiez** : ouvrez les sources des affirmations importantes et contrôlez qu'elles disent bien ce que le rapport prétend. Marquez [à vérifier] tout ce que vous n'avez pas pu contrôler.
7. **Conservez le rapport** : enregistrez-le dans un fichier (dans CREDIX, sous forme de PDF) et ajoutez-le au « Contexte » du projet, pour que les sessions suivantes s'appuient dessus.

8. Étape 3 : les choix méthodologiques, exemple du WoE

C'est le cœur de la méthode. Chaque choix suit la même séquence : **question, options, critères, décision, preuve, consignation**.

8.1 L'échelle de preuve à trois niveaux

Niveau	Signification	Exemple
I : démontré	Établi par une expérience sur nos données (mesure, ablation, partition)	« L'AUC augmente quand la couverture ρ_C augmente »
II : ancré dans la littérature	Fait connu, cité, non refait	« Le WoE est standard en scoring de crédit (Siddiqi) »
III : hypothèse assumée	Choix de conception justifié comme tel	« Seuil de refus à 30 % de probabilité de défaut »

Et la **grille des quatre questions** pour chaque bloc technique : ① **le problème**, ② **la solution**, ③ **pourquoi celle-là**, ④ **comment elle est validée**.

8.2 L'exemple complet : pourquoi le WoE ?

La séquence ci-dessous est **reconstituée à partir des décisions réellement consignées** dans les documents du projet.

A. Poser la question, avec les contraintes.

Mon pipeline de scoring doit rester explicable variable par variable (contrainte réglementaire) et gérer des valeurs manquantes nombreuses. Comment encoder mes variables numériques et catégorielles avant le modèle ? Compare au moins : standardisation + encodage one-hot, encodage par la cible, et le WoE. Critères : explicabilité, traitement des manquants, stabilité, risque de fuite de données. Termine par une recommandation et ses risques.

B. Claude compare. Le WoE (*Weight of Evidence*) découpe chaque variable en tranches et remplace chaque valeur par le logarithme du rapport entre bons et mauvais payeurs de sa tranche.

Critère	WoE	One-hot + standardisation	Encodage par la cible
Explicabilité	Très bonne : chaque tranche a un poids lisible	Moyenne	Faible
Valeurs manquantes	Tranche « Manquant » dédiée	Imputation arbitraire	À gérer à part
Stabilité (tranches monotones)	Contrainte possible	Non	Non
Risque de fuite	Réel : à apprendre sur le train seul	Faible	Élevé

C. L'avocat du diable.

Donne-moi les cinq meilleures objections à ce choix. Pour chacune, dis si elle est décisive ou si on peut la traiter.

D. La décision, écrite dans le registre (faits tirés du récapitulatif de méthodologie du projet) :

Champ	Contenu
Décision	Encodage WoE réalisé par la bibliothèque <code>optbinning</code> (classe <code>OptimalBinning</code>)
Pourquoi	Explicabilité par variable exigée par la réglementation ; gestion native des manquants ; tranches monotones
Mécanisme	Pré-découpage fin par arbre de décision, puis un solveur de programmation par contraintes fusionne les tranches voisines pour maximiser l'IV, sous contraintes de taille minimale, de nombre maximal de tranches et de monotonie du WoE
Valeurs manquantes	Jamais imputées : regroupées dans une tranche « Manquant » dont le WoE est calculé sur la vraie proportion de bons et de mauvais payeurs
Garde-fou anti-fuite	Découpage appris sur le jeu d'entraînement uniquement , puis appliqué à la validation et au test
Filtre associé	Variables retenues si $IV \geq 0,02$ (seuil classique)
Choix assumé	Une variable avec plus de 50 % de manquants est exclue, sauf si son IV atteint 0,10 : plus conservateur que le défaut de certains outils (95 %)
Niveau de preuve	II pour le WoE et le seuil d'IV ; III pour le seuil de 50 % (à présenter comme tel)

Champ	Contenu
Alternatives écartées	Standardisation + one-hot ; encodage par la cible
Risques restants	Perte d'information par découpage en tranches

E. Vérifier par une expérience quand c'est possible. Un contrôle cellule par cellule du notebook a révélé un **défaut réel** : pour recalculer l'IV des colonnes exclues, le code associait la valeur d'une colonne (ordre brut du fichier) à la cible du découpage temporel (ordre trié). Les deux ordres différaient : l'IV de ces colonnes ne mesurait rien de fiable (valeurs quasi nulles pour une quarantaine de colonnes). La correction (fusion sur l'identifiant client) a fait passer 20 colonnes dans la catégorie « couverture faible », **prévue mais jamais déclenchée**. Sans cette vérification, un résultat faux serait entré dans le mémoire.

8.3 Les mêmes réflexes pour tout autre choix

La même séquence a servi pour : LightGBM contre XGBoost, l'autoencodeur pour le contrôle des profils atypiques, la calibration isotonique, les seuils de décision, l'indice de couverture ρ_c .

Prompt 6 : produire une fiche de décision

Nous venons de discuter du choix de [sujet]. Rédige la fiche de décision en Markdown : contexte, options comparées, critères, décision retenue, alternatives écartées avec leur raison, niveau de preuve (I, II ou III), références (marque [à vérifier] si nécessaire), risques restants. N'invente aucune référence ni aucun chiffre.

Pas à pas : tenir le registre des décisions.

1. Après chaque choix important, envoyez le **prompt 6** à Claude, dans la conversation où le choix a été discuté.
2. **Relisez** la fiche obtenue : les options comparées, la décision, le niveau de preuve, les références.
3. **Faites corriger** avec Claude ce qui est faux ou incertain, et marquer [à vérifier] les références non contrôlées.
4. **Copiez** la fiche à la suite des précédentes dans un fichier du dossier du projet (par exemple `registre_des_decisions.md`). On **ajoute**, on n'efface jamais : une décision abandonnée reste écrite, avec la raison de l'abandon.
5. **Ajoutez** ce fichier au « Contexte » du Projet Claude, pour qu'il serve de référence aux sessions suivantes.

Un exemple complet de fiche remplie se trouve en section 8.2 ; le modèle vierge est en section 16.1.

9. Étape 4 : choisir les langages et les technologies

À faire avant tout prototype. Un mauvais choix technique se paie à chaque étape suivante.

Prompt 7 : proposer une pile technologique justifiée

À partir des exigences du cahier des charges (EF et ENF), propose une pile technologique : langage(s), cadre de développement, base de données, outils de test, moyen de lancement. Pour chaque choix : l'exigence qu'il sert, deux alternatives écartées et pourquoi, les risques. Contraintes : je débute sur [technologie], je dois pouvoir tout lancer en local en quelques commandes.

Règle utilisée pour CREDIX : *chaque technologie répond à une exigence formulée plus haut.*
Exemples du mémoire : **Python** pour l'apprentissage **et** l'exploitation, afin d'éviter tout écart silencieux entre le modèle entraîné et le modèle exécuté ; **FastAPI** pour la vérification déclarative des requêtes et le contrôle centralisé des droits ; **Next.js** avec **TypeScript**, dont le typage fait apparaître les écarts de format avant l'exécution ; **MongoDB** pour conserver des demandes de structures variables sans migration ; un **service PDF séparé** pour les comptes rendus.

Dans CREDIX (séance du 30 mai). Les décisions d'architecture ont été prises point par point : quatre conteneurs Docker (API, base de données, suivi des modèles, service PDF), huit collections de données, des seuils de décision configurables depuis l'interface plutôt qu'écrits en dur, une promotion de modèle soumise à validation humaine.

10. Étape 5 : le document de conception

10.1 Ce que contient le document

1. **Contexte, objectifs, périmètre** ;
2. **Acteurs et cas d'utilisation** (diagramme et descriptions) ;
3. **Modèle du domaine** : classes, associations, multiplicités ;
4. **Scénarios principaux** : diagrammes de séquence ;
5. **Architecture** : couches, composants, déploiement ;
6. **Données** : tables ou collections ;
7. **Interfaces** : routes de l'API, avec exemples de requêtes et de réponses ;
8. **Critères d'acceptation et plan de tests** ;
9. **Glossaire**.

Pour que Claude Code puisse **lire** les diagrammes, on les écrit en **texte** avec **Mermaid**, qui s'affiche automatiquement sur GitHub. Un exemple complet est fourni dans le **guide 3** (le MVP).

Prompt 8 : rédiger la conception

À partir du cahier des charges et des décisions du projet, rédige le document de conception en Markdown avec les neuf sections suivantes : [copier la liste du paragraphe 10.1]. Diagrammes en Mermaid. Règles UML : pas d'attribut dont le type est une autre classe du diagramme (utiliser une association, avec multiplicités aux deux extrémités) ; les types énumérés portent le stéréotype <<enumeration>>. Chaque diagramme est précédé d'une phrase d'introduction et suivi d'une légende. Ne rien inventer qui ne découle pas des documents du projet.

Prompt 9 : vérifier la cohérence entre documents

Compare le cahier des charges et le document de conception. Liste : les exigences non couvertes par la conception, les éléments de conception sans exigence, les noms qui diffèrent d'un document à l'autre, les contradictions.

Dans CREDIX. La conception suit les règles du cours de génie logiciel (UML : classes, séquences, cas d'utilisation, composants, déploiement), puis a été dessinée en schémas (docs/diagrammes/).

11. Étapes 6 et 7 : Claude Code, du plan au prototype testé

11.1 Étape 6 : le mode Plan et la TODO

On ouvre le projet dans VS Code, on **dépose le document de conception** dans le dossier, et on passe Claude Code en **mode Plan** (guide 1, section 8).

Prompt 10 : produire le plan de réalisation

```
/plan Lis docs/conception.md et docs/cahier_des_charges.md.
Propose un plan de réalisation en modules ordonnés (du plus indépendant au plus
dépendant). Pour chaque module : son objectif, les fichiers à créer, les tests
à écrire, le critère qui dit qu'il est terminé, et les dépendances. Ne crée
aucun fichier pour l'instant. Signale les points du document de conception
qui te paraissent ambigus.
```

Vous **relisez le plan**, l'annotez dans VS Code, le faites corriger, puis vous **l'approuvez**. Ce plan devient la **TODO** du projet.

Pas à pas dans VS Code (le détail de chaque bouton est dans le guide 1, sections 7 et 8) :

1. **Ouvrez le dossier du projet** (menu **Fichier**, puis **Ouvrir le dossier...**). Il doit contenir le document de conception, par exemple docs/conception.md.
2. **Ouvrez le panneau Claude Code** (icône d'étincelle) et **passez en mode Plan** : cliquez sur le nom du mode en bas à droite de la zone de saisie, puis sur « **Plan** ».
3. **Collez le prompt 10** dans la zone de saisie et envoyez avec Entrée.
4. **Attendez le plan** : il s'ouvre comme un document Markdown. Aucun fichier du projet n'a été créé ni modifié à ce stade.
5. **Relisez-le avec une grille simple** : chaque module a-t-il un objectif, des fichiers, des tests et un critère de fin ? Les modules sont-ils dans le bon ordre, du plus indépendant au plus dépendant ? Les ambiguïtés du document de conception sont-elles signalées ?
6. **Faites corriger** : ajoutez vos commentaires dans le plan, ou écrivez-les dans la zone de saisie.
7. **Approuvez le plan**. Au début, choisissez la réponse « Oui, en approuvant chaque modification à la main ».
8. **Enregistrez le plan** dans un fichier du dossier du projet (par exemple TODO.md) : c'est la liste des tâches que vous cocherez module après module.

11.2 Étape 7 : un module à la fois, avec des tests

Prompt 11 : réaliser un module

```
Réalise le module [M2] du plan. Écris d'abord les tests, puis le code.
Lance les tests et montre-moi le résultat. Ne touche à aucun autre module.
Si tu dois modifier un fichier existant, dis-le avant.
```

Prompt 12 : quand un test échoue

```
Le test [nom] échoue avec ce message : [coller le message]. Avant de modifier
quoi que ce soit, explique la cause probable et propose la correction la plus
petite possible.
```

Chaque module est **validé avant de passer au suivant**.

11.3 Confier une mission à Claude Code : un exemple réel

Le prompt fixe **la mission, les règles et les livrables**. Voici sa structure, dans le prompt réellement utilisé pour la phase 2 de CREDIX.

Prompt (extrait réel)

```
Tu travailles sur mon mémoire CREDIX (scoring de crédit). Le notebook est dans
le workspace. Lis d'abord EN ENTIER le fichier HANDOVER_CLAUDE_CODE_PHASE2.md
(contexte, règles absolues, variables disponibles) avant de coder.
```

```
Ta mission : écrire les cellules de la Section 12.3+ qui démontrent [...].
Ce n'est pas un concours d'outils.
```

RÈGLES

- On étend, on ne supprime pas. Nouvelles cellules en Section 12.3+ UNIQUEMENT.
- Tu n'exécutes pas le notebook. Tu testes ta logique en local sur données factices, puis tu livres le code prêt à coller.
- Anti-fuite : tout ce qui s'apprend s'apprend sur le TRAIN.

Les **règles absolues** du document de passation associé : « *On étend, on ne supprime pas* » ; « *Aucun chiffre inventé : le code produit les chiffres ; on ne les anticipe jamais dans les commentaires ou les rapports* » ; et la **division du travail** : **Claude Code écrit et teste sur des données factices ; l'auteur exécute sur les vraies données et renvoie les sorties.**

12. Étape 8 : livrer, lancer, tester à la main

Prompt 13 : préparer la livraison et le README

Prépare la livraison du projet. 1) Vérifie que tous les tests passent. 2) Rédige le README de lancement pour un débutant : prérequis, commandes exactes dans l'ordre, ce qu'on doit voir après chaque commande, et que faire en cas d'erreur. 3) Génère une archive ZIP du projet, sans les fichiers secrets ni les dossiers volumineux.

Puis, **vous** : dézippez dans un **dossier vide**, suivez le README à la lettre, notez **tout ce qui ne marche pas**, et renvoyez ces constats à Claude.

Dans CREDIX (séance du 30 mai). Le backend a été livré sous forme d'une archive de 51 fichiers, avec l'**ordre de lancement exact** : copier le fichier de configuration, copier les fichiers du modèle, lancer les conteneurs, créer l'administrateur, générer les phrases d'explication, enregistrer le modèle, charger les clients de démonstration, vérifier le point de santé.

Pour ce dossier de remise, le test du README sur une copie neuve a révélé que la **compilation de production du frontend n'avait jamais abouti** : quatre erreurs de types que le mode développement ne signalait pas. **Ce qui n'a jamais été testé de bout en bout n'est pas testé.**

13. Étape 9 : modifier, documenter, transmettre

Prompt 14 : ajouter une fonction (toujours en mode Plan)

/plan Je veux ajouter [fonction]. Analyse l'impact : quels fichiers sont concernés, quels tests existants risquent de casser, quelles nouvelles données sont nécessaires. Propose les étapes et les tests. Ne modifie rien avant mon accord.

Puis on boucle sur le cycle de session (section 3) : le travail est consigné, la passation écrite.

Dans CREDIX. Le frontend d'origine avait été généré à partir de maquettes. Il a été reconstruit (credix-v2) avec Claude Code : cahier des charges de l'interface, système de design, puis réalisation **par phases** (fondations, espace Agent, espace Superviseur, espace Administrateur), **chaque phase vérifiée avant la suivante.**

14. Les erreurs et les pièges réels

Claude se trompe, comme un humain. La méthode ne l'empêche pas : elle **permet de le voir**. Voici des erreurs réelles de CREDIX, et la règle qui en est sortie.

Ce qui s'est passé	Comment c'est apparu	Règle qui en découle
Un seuil de détection d'anomalie allait être calculé sur « les bons payeurs », c'est-à-dire avec la variable cible, dans un mécanisme non supervisé (fuite d'information)	L'auteur a relevé l'erreur ; les seuils ont été chargés depuis les fichiers du modèle , non recalculés	Relire chaque proposition avec ses contraintes de rigueur en tête
Un argument disait que LightGBM « gagnait nettement sur le rappel »	Une comparaison mesurée a montré des écarts non significatifs entre LightGBM et XGBoost ; l'argument a été rejeté , LightGBM gardé sur d'autres critères	Une décision « définitive » peut être révisée si un chiffre la contredit : le registre garde la trace
Des documents externes donnaient un indicateur de 0,13 pour une variable	Vérifié contre l'exécution réelle : 0,0199	Vérifier tout chiffre reçu d'une source externe , même si le raisonnement est juste
Un défaut d'alignement dans le calcul de l'IV (section 8)	Contrôle cellule par cellule	Vérifier les résultats étonnants , y compris les résultats « trop faibles »
Un test comparait « défaut des accordés avec anomalie » et « accordés sains » : hors sujet, car une anomalie n'est pas un défaut	Recentrage acté avec l'auteur	Demander : « <i>ce test répond-il vraiment à mon objectif ?</i> »
Un environnement installait une version trop ancienne d'une bibliothèque	Message d'erreur explicite	Figurer les versions des bibliothèques
La compilation de production du frontend n'avait jamais été lancée	Test sur copie neuve (ce dossier)	Tester comme le fera l'utilisateur final

Les réflexes qui en découlent : relire ; mesurer ; garder la trace ; refuser les chiffres non sourcés ; ne jamais présenter le système autrement qu'il n'est ; tester sur une machine neuve.

15. Bibliothèque de prompts

N°	Moment	Objet
S	Début de session	Prompt de démarrage : ordre de lecture, attendre le feu vert (section 3.1)
F	Fin de session	Rapport de passation (section 3.3)

N°	Moment	Objet
0	Étape 0	Faire l'inventaire du projet
1	Étape 1	Cadrer le besoin en se faisant interroger
2	Étape 1	Rédiger le cahier des charges
3	Étape 1	Faire critiquer le cahier des charges
4	Étape 2	Balayer la littérature (recherche approfondie)
5	Étape 2	Interroger un article chargé
6	Étape 3	Produire une fiche de décision
7	Étape 4	Proposer une pile technologique
8	Étape 5	Rédiger le document de conception
9	Étape 5	Vérifier la cohérence entre documents
10	Étape 6	Produire le plan de réalisation (mode Plan)
11	Étape 7	Réaliser un module avec tests
12	Étape 7	Traiter un test qui échoue
13	Étape 8	Préparer la livraison et le README
14	Étape 9	Ajouter ou modifier une fonction (mode Plan)

Deux prompts utiles à toutes les étapes :

Prompt 15 : l'avocat du diable

Donne-moi les cinq meilleures objections à [ce choix, ce plan, ce texte].
Pour chacune : est-elle décisive ? comment la traiter ?

Prompt 16 : la chasse aux chiffres non sourcés

Relis ce document. Liste chaque chiffre, chaque affirmation factuelle et chaque référence. Pour chacun : d'où vient-il (document du projet, calcul, littérature, ou nulle part) ? Signale tout ce qui n'a pas de source.

16. Modèles de documents

16.1 Fiche de décision

```
## Décision D-[n] : [titre]
- **Date :** [jj/mm/aaaa]
- **Contexte :** [le problème, les contraintes]
- **Options comparées :** [A], [B], [C]
- **Critères :** [explicabilité, coût, stabilité, ...]
- **Décision :** [option retenue]
- **Pourquoi :** [justification en quelques lignes]
- **Alternatives écartées :** [option : raison]
- **Niveau de preuve :** [I démontré / II littérature / III hypothèse assumée]
- **Références :** [auteur, année] [à vérifier si besoin]
- **Risques restants :** [...]
- **Validée par :** [nom] le [date]
```

16.2 Entrée du journal de bord (ajout seulement)

```
## Session [n] : [date]
**Objectif :** [...]
```

```

**Réalisé :** [liste]
**Décisions prises :** [liste, avec renvoi vers le registre]
**En attente / à faire à la prochaine session :** [liste ordonnée]

```

16.3 Instructions du projet (six sections)

```

# Instructions du projet Claude : [titre du projet]
## Section 1 : Contexte
## Section 2 : Ressources permanentes (liste classée, avec « consulter pour... »)
## Section 3 : Décisions techniques figées
## Section 4 : Consignes permanentes (langue, niveau, style, à faire, à ne pas faire)
## Section 5 : Workflow de collaboration (début / pendant / fin de session)
## Section 6 : État courant (mis à jour à chaque session)

```

16.4 Document de passation

```

# Passation : [projet], [phase]
## 1. Contexte minimal
## 2. Règles absolues (non négociables)
## 3. Où en est le projet (ce qui est fait : ne pas refaire)
## 4. Ce qui reste (le travail demandé)
## 5. Fichiers, variables, commandes disponibles

```

16.5 Liste de pièges

```

## Pièges déjà identifiés : à ne jamais reproduire
- **[Sujet]** [Ce qui s'est passé] → [ce qu'il faut faire à la place].

```

17. Rejouer la méthode : le mini-exercice

Pour vérifier que la méthode fonctionne **sans connaître CREDIX**, le **guide 3** fournit un document de conception réduit (MVP) d'un mini-système de scoring de crédit. Le protocole de test devant l'encadrant est dans la **fiche de démonstration**. En résumé :

1. Ouvrir un dossier vide dans VS Code et y déposer 03_conception_MVP.md.
2. Ouvrir Claude Code, passer en **mode Plan**, et donner le **prompt 10**.
3. **Relire le plan** : est-il fidèle au document ? Les modules sont-ils dans le bon ordre ?
4. Approuver, réaliser **un module** avec le **prompt 11**, et vérifier que ses tests passent.

Si le plan est cohérent et que le premier module passe ses tests, **la méthode est validée**.

PROJET DE FIN D'ÉTUDES

École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique

Année académique 2025–2026

Guide 3 : conception du MVP

Un scoring de crédit réduit, à construire avec Claude Code

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Table des matières

CREDIX-MVP : document de conception	1
1. Introduction	1
2. Exigences	2
3. Acteurs et cas d'utilisation	3
4. Modèle du domaine	5
5. Scénario principal : demander un scoring	7
6. Architecture et technologies	7
7. Modèle de données	9
8. Le modèle de scoring	10
9. Interface de programmation (API)	11
10. Interface utilisateur	13
11. Plan de réalisation par modules	13
12. Plan de tests et recette	14
13. Limites et évolution vers CREDIX	15
14. Glossaire	15
15. Annexes	16

CREDIX-MVP : document de conception

Un mini-système de scoring de crédit, conçu pour être construit avec Claude Code en suivant la méthode du guide 2.

1. Introduction

1.1 Contexte

CREDIX est un système complet de scoring de crédit (voir le README du dépôt). Il compte plusieurs dizaines de routes, deux modèles complémentaires, une base documentaire, un service de rapports PDF et trois espaces utilisateurs. Il est **trop grand** pour servir d'exercice à une démonstration.

CREDIX-MVP en est une version **réduite** : le plus petit système qui garde l'idée essentielle. *Un agent saisit une demande de crédit ; le système calcule une probabilité de défaut, la convertit en score, propose une décision et explique les trois facteurs qui pèsent le plus.*

Ce document est écrit pour être donné à **Claude Code** en mode Plan (guide 1, section 8) : il contient tout ce qu'il faut pour produire un plan de réalisation, puis un système qui tourne.

1.2 Objectifs

N°	Objectif
O1	Disposer d'un système de scoring de bout en bout (données, modèle, API, interface, tests) réalisable en quelques modules
O2	Valider la méthode du guide 2 : à partir de ce seul document, Claude Code doit produire un plan cohérent, puis un premier module qui passe ses tests
O3	Garder les principes de CREDIX : score de 300 à 850, décision à trois issues, explication des facteurs, traçabilité, seuils réglables

1.3 Périmètre

Inclus dans le MVP	Exclu (voir la section 13)
Deux rôles : Agent et Administrateur	Rôle Superviseur et file de revue manuelle
Un modèle statistique simple (régression logistique)	Deux flux, autoencodeur, calibration isotonique, WoE
Six variables explicatives	Vingt-sept variables et indice de couverture ρ_C
Base SQLite , aucun Docker	MongoDB, conteneurs, service PDF
Une page web unique	Trois espaces, thème sombre, graphiques
Données synthétiques générées par un script	Jeu de données réel

1.4 Comment utiliser ce document avec Claude Code

1. Créez un dossier vide et ouvrez-le dans **VS Code**.
2. Copiez-y ce fichier sous docs/conception_mvp.md.
3. Ouvrez Claude Code et passez en **mode Plan**.
4. Donnez le prompt de l'**annexe B**.

Le déroulé complet de la démonstration est dans la fiche de démonstration.

2. Exigences

2.1 Exigences fonctionnelles

N°	Exigence	Critère d'acceptation
EF-1	Un utilisateur se connecte avec un identifiant et un mot de passe et reçoit un jeton valable 8 heures	Bons identifiants : jeton reçu ; mauvais identifiants : refus (401)
EF-2	L'accès aux routes dépend du rôle (Agent, Administrateur)	Un Agent qui appelle une route d'administration reçoit 403
EF-3	Un agent crée un client (nom, prénom, âge, revenu mensuel, ancienneté d'emploi, nombre d'incidents de paiement)	Le client créé est retrouvable ; une valeur invalide (âge négatif) est refusée (422)
EF-4	Un agent recherche un client par nom ou prénom et consulte sa fiche	La recherche « ndi » retrouve « Ndiaye »
EF-5	Un agent demande un scoring pour un client, avec un montant et une durée de crédit	Le résultat contient une probabilité de défaut, un score entre 300 et 850 et une décision
EF-6	Le résultat contient trois facteurs qui expliquent le score (libellé, contribution, sens)	Trois facteurs, triés par importance décroissante
EF-7	Un agent consulte l' historique des scorings d'un client	Les scorings sont listés du plus récent au plus ancien

N°	Exigence	Critère d'acceptation
EF-8	Un administrateur crée un compte (agent ou administrateur)	Le nouveau compte peut se connecter
EF-9	Un administrateur règle les deux seuils de décision	Un seuil incohérent (accord supérieur au refus) est refusé (422)
EF-10	Un script génère les données et entraîne le modèle , puis affiche ses performances	Le script écrit le modèle et affiche une AUC d'au moins 0,70

2.2 Exigences non fonctionnelles

N°	Exigence	Critère d'acceptation
ENF-1	Traçabilité : chaque scoring garde la version du modèle, les seuils appliqués, l'utilisateur et la date	Ces quatre informations figurent dans l'historique
ENF-2	Sécurité : mots de passe hachés, jamais en clair ; secret du jeton dans une variable d'environnement	Aucun mot de passe lisible dans la base ni dans le code
ENF-3	Reproductibilité : deux exécutions du script avec la même graine donnent le même modèle	Mêmes coefficients à la troisième décimale
ENF-4	Lancement simple : installation et démarrage en six commandes, sans Docker	Le README est suivi à la lettre sur une machine neuve
ENF-5	Réactivité : le modèle est chargé une seule fois au démarrage	Un scoring répond en moins de 500 ms en local
ENF-6	Testabilité : une commande unique lance tous les tests	pytest passe, avec au moins un test par exigence fonctionnelle

3. Acteurs et cas d'utilisation

Le système a **deux acteurs**. L'**Administrateur** peut faire tout ce que fait l'Agent, et en plus gérer les comptes et les seuils : il **hérite** du rôle Agent.

La figure 1 présente les cas d'utilisation de chaque acteur.

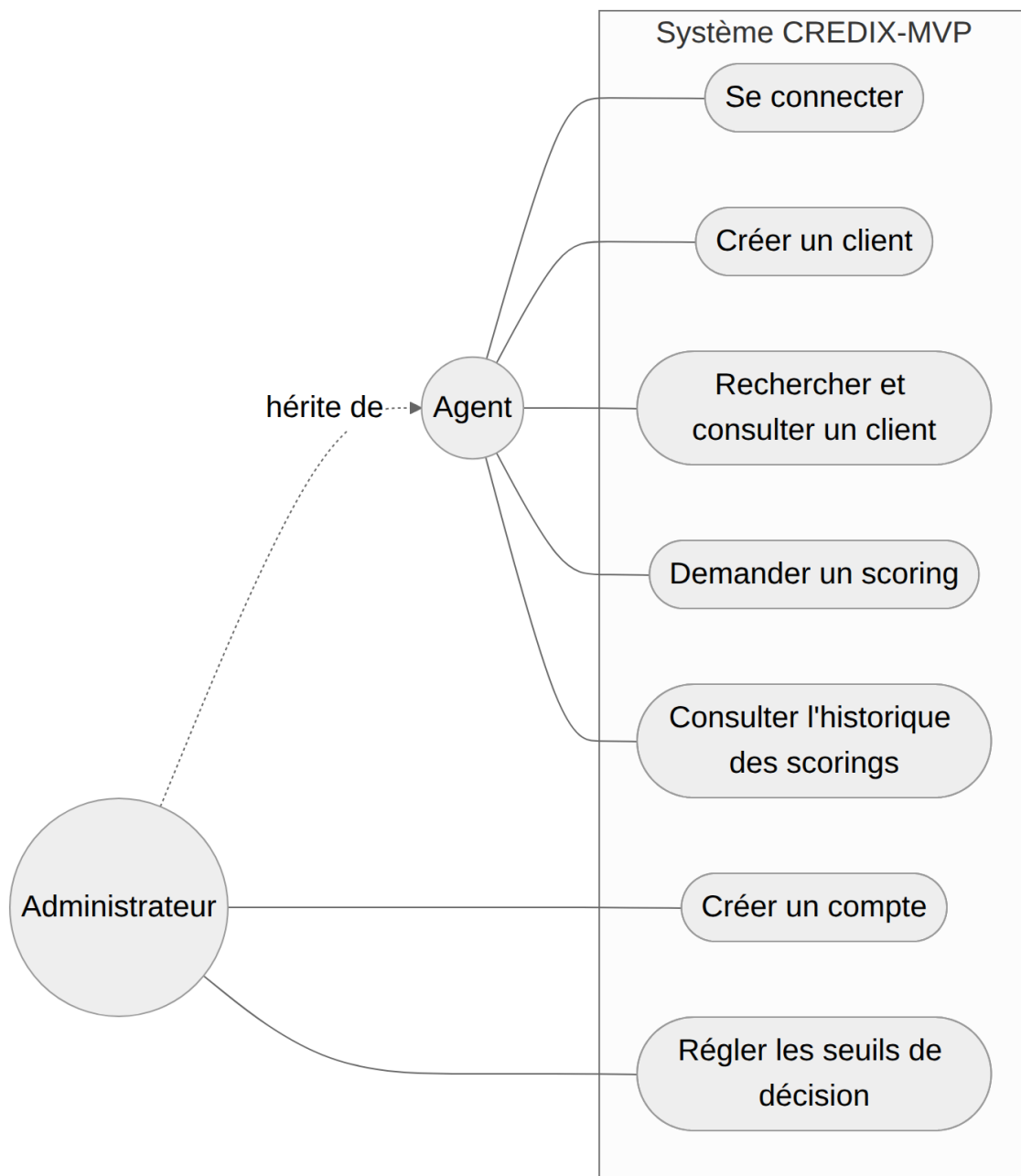


FIGURE 1 – Cas d'utilisation de CREDIX-MVP. Les ovales sont les cas d'utilisation, les cercles les acteurs. La flèche en pointillés indique que l'Administrateur hérite de tout ce que fait l'Agent.

3.1 Spécification du cas « Demander un scoring »

Élément	Contenu
Acteur	Agent (ou Administrateur)
Objectif	Obtenir une décision expliquée pour une demande de crédit
Préconditions	L'agent est connecté ; le client existe

Élément	Contenu
Scénario nominal	1. L'agent ouvre la fiche du client. 2. Il saisit le montant et la durée du crédit. 3. Il valide. 4. Le système calcule la probabilité de défaut, le score et la décision, et sélectionne les trois facteurs principaux. 5. Le système enregistre le scoring. 6. Il affiche le résultat.
Alternatives	3a. Montant ou durée invalides : le système refuse (422) et l'indique. 4a. Le modèle n'est pas disponible : erreur 503 avec un message clair.
Postcondition	Un scoring est enregistré avec sa version de modèle, ses seuils, son auteur et sa date

3.2 Spécification du cas « Créer un compte »

Élément	Contenu
Acteur	Administrateur
Préconditions	L'administrateur est connecté
Scénario nominal	1. L'administrateur saisit un identifiant, un mot de passe (8 caractères minimum) et un rôle. 2. Le système vérifie que l'identifiant est libre. 3. Il enregistre le compte, mot de passe haché .
Alternatives	2a. Identifiant déjà pris : refus (409). 1a. Mot de passe trop court : refus (422).

3.3 Spécification du cas « Régler les seuils de décision »

Élément	Contenu
Acteur	Administrateur
Scénario nominal	1. L'administrateur saisit un seuil d'accord et un seuil de refus (probabilités de défaut). 2. Le système vérifie que $0 < \text{accord} < \text{refus} < 1$. 3. Il enregistre une nouvelle version des seuils (l'historique est conservé).
Règle métier	Probabilité de défaut inférieure au seuil d'accord : décision ACCORDÉ. Supérieure au seuil de refus : REFUSÉ. Entre les deux : REVUE (un humain doit trancher). Valeurs initiales : 0,10 et 0,30 (les mêmes que CREDIX).

4. Modèle du domaine

Le domaine compte cinq classes principales, trois types énumérés, et les associations qui les relient. La figure 2 présente le diagramme de classes.

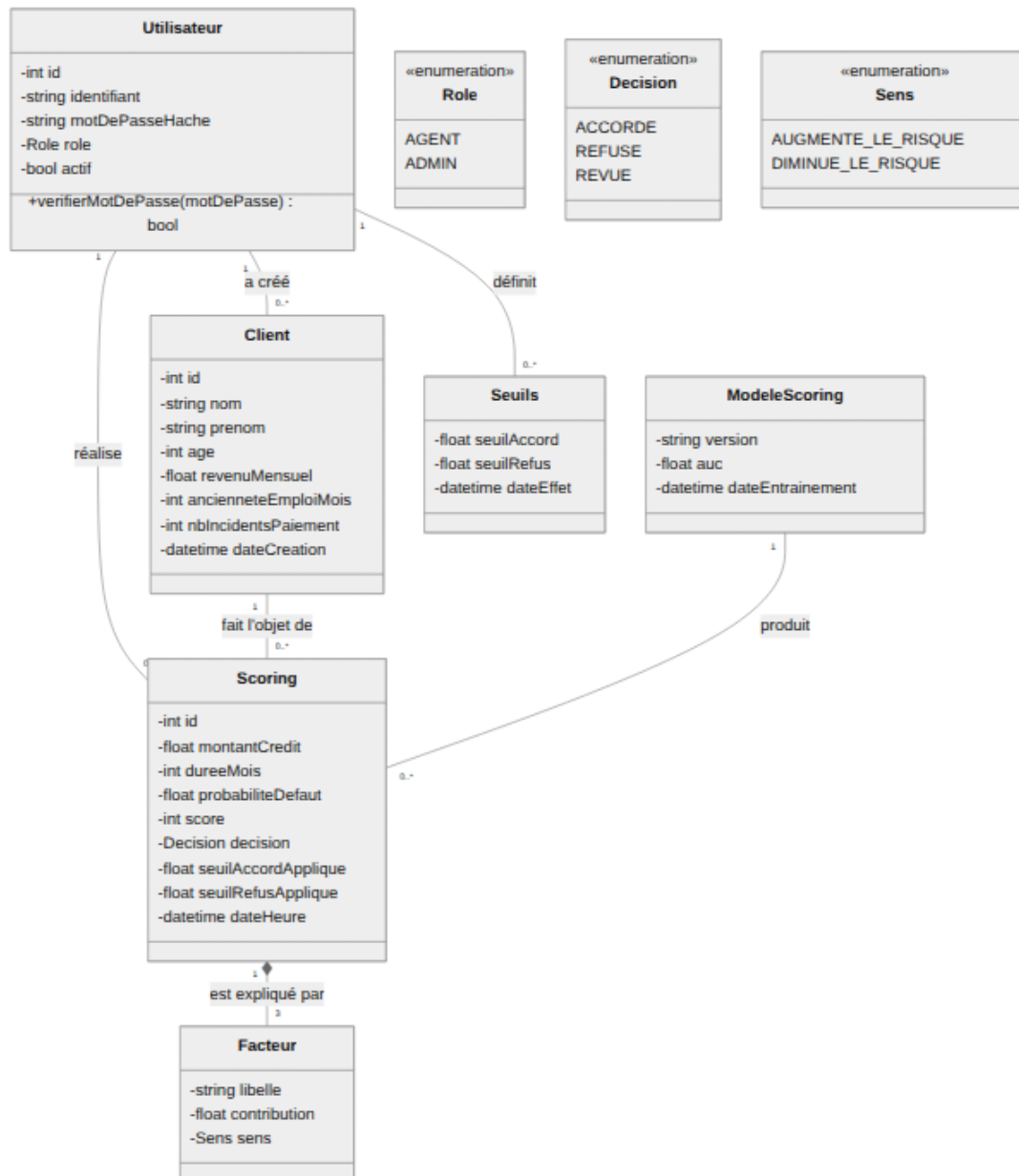


FIGURE 2 – Diagramme de classes du domaine. Les losanges pleins indiquent une composition : un Facteur n'existe que dans le Scoring qu'il explique. Les types énumérés portent le stéréotype « enumeration ». Les attributs sont privés (signe moins), les opérations publiques (signe plus).

Règles de lecture et de conception :

- Aucun attribut n'a pour type une autre classe du diagramme : les liens entre classes sont des **associations**, avec leurs multiplicités aux deux extrémités.
- Un **Scoring** garde les seuils **appliqués** (seuilAccordApplique, seuilRefusApplique) : ils ne changent pas si les seuils sont modifiés plus tard (traçabilité, ENF-1).
- Un **Scoring** est toujours produit par **un** modèle, **un** utilisateur, pour **un** client.

5. Scénario principal : demander un scoring

La figure 3 montre les échanges entre l'agent et les composants du système lors d'un scoring.

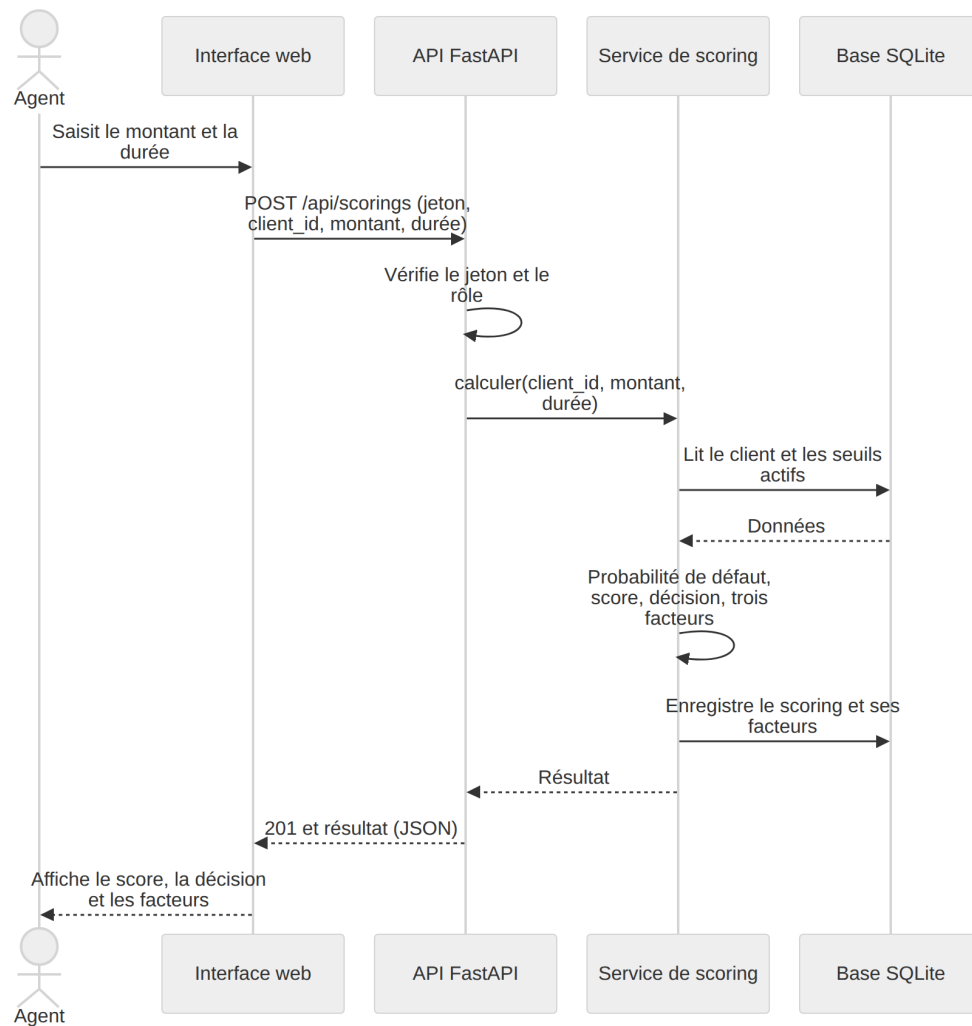


FIGURE 3 – Diagramme de séquence du cas « Demander un scoring ». Les flèches pleines sont des appels, les flèches pointillées des retours.

6. Architecture et technologies

6.1 Vue d'ensemble

La figure 4 présente les composants et leurs liens.

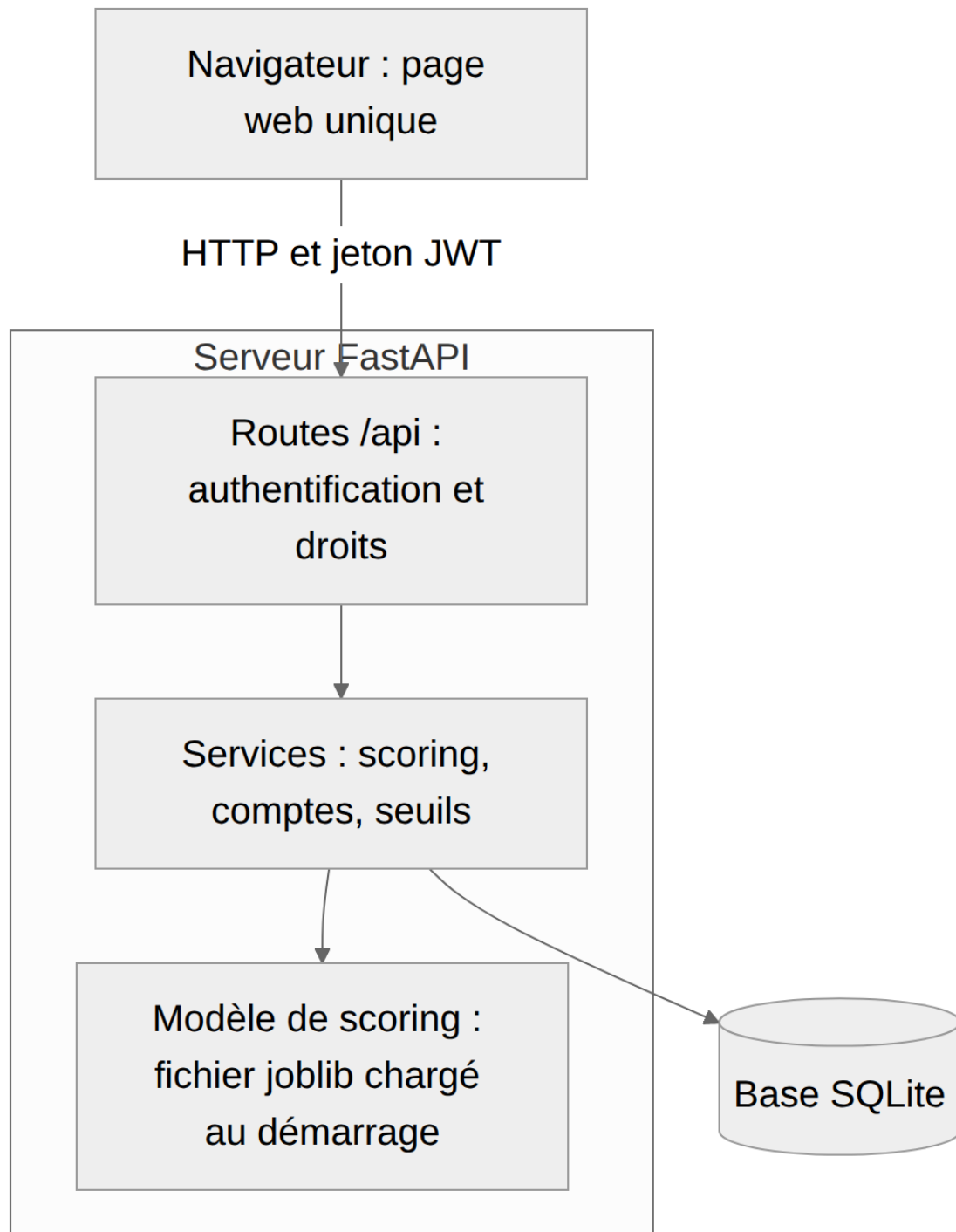


FIGURE 4 – Architecture de CREDIX-MVP. Le navigateur ne parle qu'aux routes ; les services portent la logique ; le modèle est chargé une seule fois en mémoire.

6.2 Choix technologiques

Chaque choix répond à une exigence.

Choix	Sert l'exigence	Alternative écartée
Python 3.12	EF-10, ENF-3 : un seul langage pour l'entraînement et l'exploitation, pas d'écart entre les deux	Java : deux langages à faire dialoguer
FastAPI	EF-2, EF-3, ENF-5 : validation automatique des données, droits centralisés, rapidité	Flask : validation à écrire à la main
SQLite (via SQLAlchemy)	ENF-4 : aucune installation, un fichier	PostgreSQL, MongoDB : trop lourds pour ce MVP
scikit-learn (régression logistique)	EF-6, ENF-3 : modèle explicable (contribution = coefficient × valeur standardisée), reproductible	LightGBM : explications plus complexes (voir CREDIX)
PyJWT et hashlib (PBKDF2)	EF-1, ENF-2 : jeton signé, mots de passe hachés, sans dépendance lourde	bcrypt : dépendance native inutile ici
Page HTML et JavaScript servie par FastAPI	ENF-4 : aucun outil de compilation	React, Next.js : chaîne de compilation en plus
pytest et httpx	ENF-6	—

6.3 Organisation des dossiers

```

credix-mvp/
|-- README.md
|-- requirements.txt
|-- .env.example          (SECRET_JWT)
|-- app/
|   |-- main.py           création de l'application, chargement du modèle
|   |-- config.py
|   |-- database.py       connexion SQLite, création des tables
|   |-- models.py         tables (Utilisateur, Client, Scoring, ...)
|   |-- schemas.py        formats des requêtes et des réponses
|   |-- security.py       hachage, jeton, contrôle des rôles
|   |-- routers/          auth.py, clients.py, scorings.py, admin.py
|   |-- services/         scoring.py, comptes.py, seuils.py
|-- scripts/
|   |-- generer_donnees.py données synthétiques (annexe A)
|   |-- entrainer_modele.py entraînement et sauvegarde du modèle
|   |-- creer_admin.py    premier compte administrateur
|-- modele/               modele.joblib et meta.json (générés)
|-- frontend/             index.html, app.js, style.css
|-- tests/                un fichier de tests par module

```

7. Modèle de données

Base SQLite, une table par classe (les facteurs sont stockés avec leur scoring).

Table	Colonnes	Contraintes
utilisateur	id (clé), identifiant, mot_de_passe_hache, role, actif	identifiant unique ; role parmi AGENT, ADMIN
client	id (clé), nom, prenom, age, revenu_mensuel, anciennete_emploi_mois, nb_incidents_paiement, date_creation, cree_par (vers utilisateur)	age entre 18 et 80 ; revenu_mensuel > 0 ; ancienneté et incidents ≥ 0
modele_scoring	id (clé), version, auc, date_entrainement	version unique
seuils	id (clé), seuil_accord, seuil_refus, date_effet, defini_par (vers utilisateur)	$0 < \text{accord} < \text{refus} < 1$
scoring	id (clé), client_id, utilisateur_id, modele_id, montant_credit, duree_mois, probabilite_default, score, decision, seuil_accord_applique, seuil_refus_applique, date_heure	montant_credit > 0 ; duree_mois entre 6 et 60
facteur	id (clé), scoring_id, libelle, contribution, sens	trois lignes par scoring

8. Le modèle de scoring

8.1 Variables du modèle

Six variables explicatives, dont deux **calculées** à partir des données saisies :

Variable	Origine	Libellé affiché
age	Fiche du client	« Âge »
log_revenu	Logarithme népérien du revenu mensuel	« Revenu mensuel »
anciennete_emploi_mois	Fiche du client	« Ancienneté dans l'emploi »
taux_effort	(montant du crédit ÷ durée) ÷ revenu mensuel : part du revenu mensuel consacrée au remboursement	« Taux d'effort »
duree_mois	Saisie de la demande	« Durée du crédit »
nb_incidents_paiement	Fiche du client	« Incidents de paiement passés »

8.2 Entraînement

1. Le script `generer_donnees.py` produit 3 000 clients synthétiques (annexe A), avec une variable cible default (0 ou 1).

2. `entraîner_modelle.py` sépare les données (80 % entraînement, 20 % test, **graine 42**, découpage stratifié).
3. Les six variables sont **standardisées** (moyenne 0, écart-type 1), en apprenant les moyennes et écarts-types **sur l'entraînement seulement**.
4. Une **régression logistique** est entraînée.
5. Le script affiche l'**AUC sur le jeu de test** et sauvegarde le modèle, les moyennes, les écarts-types, les coefficients et l'AUC dans `modele/`.

8.3 Calcul d'un scoring

Étape	Formule
Probabilité de défaut	Sortie de la régression logistique sur les variables standardisées
Score	$\text{score} = 515,06 - 28,85 \times \ln(PD \div (1 - PD))$, borné entre 300 et 850, arrondi à l'entier (les constantes de CREDIX)
Décision	PD < seuil d'accord : ACCORDÉ ; PD > seuil de refus : REFUSÉ ; sinon REVUE
Contribution d'une variable	$\text{coefficient} \times \text{valeur standardisée}$
Trois facteurs	Les trois variables dont la contribution est la plus grande en valeur absolue ; le sens est AUGMENTE_LE_RISQUE si la contribution est positive, DIMINUE_LE_RISQUE sinon

8.4 Valeurs de référence

Avec le jeu de données de l'annexe A (graine 42), un premier essai a donné une **AUC de test d'environ 0,75**, un taux de défaut d'environ 8 % et, sur les 3 000 clients, environ 78 % d'accords, 17 % de revues et 5 % de refus. Les valeurs exactes dépendent de l'implémentation ; **les critères de recette de la section 12 sont donc écrits avec des marges**.

Trois profils d'exemple, avec le résultat **indicatif** obtenu :

Caractéristique	A solide	B intermédiaire	C fragile
Âge	42	33	24
Revenu	450 000	220 000	120 000
Ancienneté	120 mois	36 mois	6 mois
Montant	1 500 000	2 600 000	1 800 000
Durée	24 mois	24 mois	12 mois
Incidents	0	2	3
PD	0,5 %	24,5 %	95 %
Score	667	548	429
Décision	ACCORDÉ	REVUE	REFUSÉ

9. Interface de programmation (API)

Format des échanges : **JSON**, avec des noms en `snake_case`. Toutes les routes, sauf `/health` et `/api/auth/login`, exigent l'en-tête `Authorization: Bearer <jeton>`.

Méthode et route	Rôle requis	Description	Codes
GET /health	aucun	État du système et du modèle	200
POST /api/auth/login	aucun	Connexion	200, 401
POST /api/clients	Agent, Admin	Créer un client	201, 422
GET /api/clients?recherche=	Agent, Admin	Rechercher (nom ou prénom)	200
GET /api/clients/{id}	Agent, Admin	Fiche d'un client	200, 404
POST /api/scorings	Agent, Admin	Demander un scoring	201, 404, 422, 503
GET /api/scorings?client_id=	Agent, Admin	Historique d'un client	200
POST /api/admin/utilisateurs	Admin	Créer un compte	201, 403, 409, 422
GET /api/admin/seuils	Admin	Seuils actifs	200, 403
PUT /api/admin/seuils	Admin	Nouvelle version des seuils	200, 403, 422

Sans jeton ou avec un jeton invalide : **401**. Avec un rôle insuffisant : **403**.

Exemple : connexion

```
POST /api/auth/login
{ "identifiant": "agent1", "mot_de_passe": "MotDePasse2026" }

200 OK
{ "jeton": "eyJhbGciOi...", "role": "AGENT", "expire_a": "2026-09-20T09:00:00Z" }
```

Exemple : demande de scoring

```
POST /api/scorings
{ "client_id": 12, "montant_credit": 2600000, "duree_mois": 24 }

201 Created
{
  "id": 57,
  "client_id": 12,
  "probabilite_defaut": 0.245,
  "score": 548,
  "decision": "REVUE",
  "facteurs": [
    { "libelle": "Incidents de paiement passés", "contribution": 0.931, "sens":
      ↳ "AUGMENTE_LE_RISQUE" },
    { "libelle": "Taux d'effort", "contribution": 0.408, "sens": "AUGMENTE_LE_RISQUE"
      ↳ },
    { "libelle": "Âge", "contribution": 0.256, "sens": "AUGMENTE_LE_RISQUE" }
  ],
  "modele_version": "1.0",
  "seuil_accord": 0.10,
  "seuil_refus": 0.30,
  "date_heure": "2026-09-19T14:03:11Z"
}
```

10. Interface utilisateur

Une **page web unique**, sans compilation, servie à la racine /. Trois zones, affichées selon le rôle.

Zone	Contenu
Connexion	Identifiant, mot de passe, bouton « Se connecter » ; message clair en cas d'échec
Clients et scoring (Agent et Admin)	Barre de recherche ; formulaire « Nouveau client » ; fiche d'un client avec le formulaire de scoring (montant, durée) ; résultat : score, décision (colorée : vert, orange, rouge), probabilité de défaut, trois facteurs avec leur sens ; historique du client
Administration (Admin seulement)	Création d'un compte ; réglage des deux seuils, avec le rappel de la règle « accord < refus »

Un bouton « Se déconnecter » efface le jeton.

11. Plan de réalisation par modules

L'ordre va du plus indépendant au plus dépendant. Chaque module se termine quand ses tests passent.

Module	Objectif	Fichiers principaux	Terminé quand
M1 Socle	Structure du projet, configuration, base et tables	app/config.py, database.py, models.py, requirements.txt	Les tables sont créées ; test de connexion à la base
M2 Données et modèle	Générer les données, entraîner et sauvegarder le modèle (EF-10, ENF-3)	scripts/generer_donnees.py, scripts/entraîner_modele.py	AUC de test au moins égale à 0,70 ; deux exécutions donnent les mêmes coefficients
M3 Service de scoring	PD, score, décision, trois facteurs (EF-5, EF-6)	app/services/scoring.py	Les profils A et C donnent ACCORDÉ et REFUSÉ ; plus d'incidents implique une PD plus élevée
M4 Sécurité et comptes	Hachage, jeton, rôles (EF-1, EF-2, EF-8)	app/security.py, routers/auth.py, routers/admin.py, scripts/creer_admin.py	Connexion réussie ou refusée ; 403 pour un Agent sur une route d'administration
M5 API métier	Clients, scorings, historique, seuils (EF-3, EF-4, EF-7, EF-9, ENF-1)	routers/clients.py, routers/scorings.py, services/seuils.py	Tous les tests d'API passent

Module	Objectif	Fichiers principaux	Terminé quand
M6 Interface et livraison	Page web, README de lancement (ENF-4)	frontend/*, README.md	Le README est suivi à la lettre sur une machine neuve

Dépendances : M1 avant tout ; M2 avant M3 ; M4 avant M5 ; M3 et M5 avant M6.

Règles de conduite pour Claude Code : un seul module à la fois ; les tests d'abord ; ne jamais modifier un module déjà validé sans le dire ; ne jamais écrire de mot de passe ni de secret dans le code.

12. Plan de tests et recette

12.1 Tests automatiques

Une commande, `pytest`, lance tous les tests. Il y a **au moins un test par exigence fonctionnelle**, plus :

- **monotonie** : à profil identique, la PD **augmente** avec le nombre d'incidents (0, 1, 2, 3) ;
- **bornes** : le score est toujours entre 300 et 850 ;
- **traçabilité** : un scoring enregistré contient la version du modèle et les seuils appliqués ;
- **sécurité** : le hachage du mot de passe est différent du mot de passe, et la vérification réussit.

12.2 Scénario de recette (à la main)

N°	Action	Résultat attendu
1	Lancer les commandes du README	L'application répond sur <code>http://localhost:8000 ; /health</code> indique « ok »
2	Se connecter avec le compte administrateur	Accès à l'écran d'administration
3	Créer un compte « agent1 »	Le compte apparaît ; il peut se connecter
4	Se connecter en agent1	Pas d'accès à l'administration
5	Créer le client du profil A	Le client est retrouvable par recherche
6	Demander un scoring (1 500 000 sur 24 mois)	Décision ACCORDÉ , score élevé, trois facteurs
7	Créer le client du profil C et demander un scoring (1 800 000 sur 12 mois)	Décision REFUSÉ
8	Consulter l'historique du client A	Le scoring figure, avec version du modèle et seuils
9	En administrateur, passer le seuil de refus de 0,30 à 0,20, puis rejouer le scoring du profil B (probabilité de défaut d'environ 24,5 %)	La décision passe de REVUE à REFUSÉ ; l'ancien scoring garde ses seuils d'origine (0,30)
10	Essayer un seuil d'accord de 0,40 avec un seuil de refus de 0,20	Le système refuse (422) et explique que l'accord doit être inférieur au refus

12.3 Critères de succès de la démonstration

Résultat	Signification
Le plan produit par Claude Code couvre tous les modules M1 à M6, dans un ordre cohérent	Le document de conception est exploitable
Le module M2 livre une AUC d'au moins 0,70	Le premier module tient ses promesses
Les tests du module M3 passent	La logique métier est correcte

13. Limites et évolution vers CREDIX

Ce que fait CREDIX	Le MVP	Comment y arriver
Deux flux : score et détection des profils atypiques (autoencodeur)	Un seul flux	Ajouter un second modèle non supervisé qui peut durcir une décision
WoE, sélection de variables, LightGBM	Régression logistique sur six variables	Remplacer le modèle derrière la même interface de scoring
Calibration isotonique des probabilités	Aucune	Ajouter une étape entre le modèle et le score
Indice de couverture ρ_c et clients à dossier incomplet	Aucun	Calculer la part d'information disponible par client
Explications par SHAP, phrases en langage naturel	Contributions du modèle linéaire	Remplacer par SHAP et un gabarit de phrases
Rôle Superviseur et file de revue	La décision REVUE n'a pas de suite	Ajouter le rôle, la file, la décision justifiée
MongoDB, Docker, service PDF, journal d'audit	SQLite, aucun conteneur	Voir l'architecture de CREDIX

Le MVP n'a **aucune valeur prédictive réelle** : ses données sont synthétiques. Il sert à **valider la méthode**, pas à évaluer un risque.

14. Glossaire

Terme	Définition
Probabilité de défaut (PD)	Chance estimée qu'un client ne rembourse pas son crédit
Score	La PD convertie en un nombre de 300 à 850 : plus il est haut, plus le risque est faible
Décision	ACCORDÉ, REFUSÉ ou REVUE (à trancher par un humain)
Seuils	Les deux probabilités qui séparent les trois décisions

Terme	Définition
Taux d'effort	Part du revenu mensuel consacrée au remboursement du crédit
AUC	Mesure de la capacité d'un modèle à séparer bons et mauvais payeurs (0,5 : aucun pouvoir ; 1 : parfait)
Régression logistique	Modèle statistique simple qui calcule une probabilité à partir d'une somme pondérée de variables
Variable standardisée	Variable ramenée à une moyenne de 0 et un écart-type de 1
JWT (jeton)	Justificatif temporaire délivré après connexion
PBKDF2	Méthode de hachage de mot de passe
Recette	Vérification finale, à la main, que le système répond au besoin

15. Annexes

Annexe A : générateur de données synthétiques (référence)

Ce script définit le « monde » du MVP. Le donner tel quel à Claude Code garantit des données identiques d'une exécution à l'autre (graine 42).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 3000
age = np.clip(rng.normal(38, 10, n), 21, 65).round().astype(int)
revenu = np.clip(np.exp(rng.normal(np.log(250000), 0.5, n)), 80000, 2000000).round(-3)
anciennete = np.clip(rng.exponential(60, n), 0, 360).round().astype(int)
duree = rng.choice([6, 12, 18, 24, 36, 48], n, p=[.10, .25, .20, .25, .15, .05])
montant = (revenu * rng.uniform(1, 10, n)).round(-4)
incidents = np.clip(rng.poisson(0.6, n), 0, 8)
taux_effort = (montant / duree) / revenu

logit = (-4.4 - 0.035 * (age - 38) - 1.1 * (np.log(revenu) - np.log(250000))
        - 0.004 * (anciennete - 60) + 3.0 * taux_effort
        + 0.02 * (duree - 18) + 0.55 * incidents)
probabilite = 1 / (1 + np.exp(-logit))
default = rng.binomial(1, probabilite)

donnees = pd.DataFrame({
    "age": age, "revenu_mensuel": revenu, "anciennete_emploi_mois": anciennete,
    "montant_credit": montant, "duree_mois": duree,
    "nb_incidents_paiement": incidents, "default": default,
})
donnees.to_csv("clients_synthetiques.csv", index=False)
```

Annexe B : prompt de démonstration

À donner à Claude Code, en **mode Plan**, dans un dossier contenant ce document sous docs/conception_mvp.md.

```
/plan Lis docs/conception_mvp.md en entier. Propose un plan de réalisation par
modules (M1 à M6 ou mieux, si tu vois une meilleure découpe). Pour chaque
module : l'objectif, les fichiers à créer, les tests à écrire, le critère qui
```

dit qu'il est terminé, les dépendances. Signale les points du document qui te paraissent ambigus ou contradictoires. Ne crée aucun fichier pour l'instant.

Puis, après relecture et approbation du plan :

Réalise le module M2 du plan. Écris d'abord les tests, puis le code. Lance les tests et montre-moi le résultat, dont l'AUC de test. Ne touche à aucun autre module.

Annexe C : commandes de lancement attendues à la fin

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python scripts/generer_donnees.py && python scripts/entraîner_modelle.py
python scripts/creer_admin.py
uvicorn app.main:app --port 8000
```

Annexe D : ce que ce document ne dit pas volontairement

Le **code** des modules M3 à M6, la structure exacte des tests et l'aspect visuel de la page. C'est le travail de Claude Code, et c'est ce que la démonstration cherche à observer.

École Nationale Supérieure Polytechnique de Yaoundé
Université de Yaoundé I

Diplôme d'Ingénieur de Conception en Génie Informatique
Année académique 2025–2026

Fiche de démonstration

Déroulé de la séance avec l'encadrant

Présenté par

SINGHE PENKA Hendrix Donavan

Version du 19/09/2026

Table des matières

Fiche de démonstration	1
1. Objectifs et déroulé de la séance	1
2. Avant la séance : les préparatifs indispensables	1
3. Partie 1 : vérifier que le projet fonctionne depuis Git	3
4. Partie 2 : comprendre comment le projet a été conçu	4
5. Partie 3 : tester la méthode sur le MVP	4
6. Questions probables et éléments de réponse	6
7. Liste de contrôle de la séance	6

Fiche de démonstration

Déroulé de la séance avec l'encadrant : vérifier le projet, comprendre la méthode, tester la méthode sur un exemple.

1. Objectifs et déroulé de la séance

L'encadrant veut vérifier **trois choses**, dans cet ordre :

Partie	Question de l'encadrant	Ce qu'on lui montre	Document
1	« <i>Ce qui a été fait est-il bien sur Git, fonctionne-t-il et est-il utilisable ?</i> »	Le projet récupéré depuis GitHub, lancé et testé	README
2	« <i>Comment avez-vous procédé pour y arriver ?</i> »	La méthode de travail avec Claude, avec les vrais documents du projet	Guide 2 (et guide 1 pour les outils)
3	« <i>Cette méthode marche-t-elle vraiment ? Montrez-le-moi.</i> »	Une conception réduite (MVP) confiée à Claude Code, en direct	Guide 3 (MVP)

Répartition indicative du temps (à ajuster) : partie 1, environ 15 minutes ; partie 2, environ 25 minutes ; partie 3, environ 40 minutes.

Ordre conseillé. L'installation de la partie 1 est **longue** (jusqu'à 50 minutes la première fois). On ne l'attend donc pas sans rien faire :

- **Si l'encadrant a déjà installé le projet** : parties 1, 2, puis 3.
- **Sinon** : dès les premières minutes, il lance la commande d'installation du README (section 5.2). Pendant qu'elle travaille, vous faites la **partie 2**, puis la **partie 3**. Quand l'installation est terminée, vous faites la **partie 1** (les tests).

2. Avant la séance : les préparatifs indispensables

Le point critique : la première installation est longue. La construction des conteneurs Docker télécharge environ 700 Mo et a duré **environ 50 minutes** sur une connexion lente lors de nos essais. Elle ne doit **jamais** être lancée en direct pendant la séance. **Elle doit être faite avant.**

2.1 Accès au dépôt (à faire la veille)

1. **Inviter l'encadrant** sur GitHub : ouvrir le dépôt, puis *Settings*, *Collaborators*, *Add people*, et saisir son nom d'utilisateur GitHub. Le dépôt est **privé** : sans invitation acceptée, il ne peut pas le cloner.
2. Lui demander de **créer un jeton d'accès personnel** (README, section 4.1) : sans lui, Git refuse le mot de passe du compte.
3. Se rappeler qu'un collaborateur d'un dépôt personnel peut **lire et modifier** le dépôt.

2.2 Ordinateur de l'encadrant (idéalement la veille)

À vérifier	Minimum
Système	Linux 64 bits
Espace disque libre	10 Go (la construction a besoin d'environ 4 Go de marge temporaire à la fin)
Outils	Git, Docker et Docker Compose, Node.js 18.18 ou plus, curl, OpenSSL (README, section 3)
Ports libres	3001, 8080, 8001, 5000, 27017

Lancer, la veille, la première installation en suivant le README (sections 4 à 6). Le README indique, à chaque étape, ce qu'il faut voir. Le lancement suivant ne prend qu'une minute environ.

2.3 Pour la partie 3 (Claude Code)

- **VS Code 1.94 ou plus** avec l'extension **Claude Code** installée (guide 1, section 7) ;
- un **abonnement Claude payant** (Pro au minimum) ou un compte Console : sans lui, Claude Code ne démarre pas ;
- **être connecté** dans l'extension avant la séance ;
- avoir **copié** docs/guides/03_conception_MVP.md dans un dossier de travail vide, sous le nom docs/conception_mvp.md.

2.4 Côté auteur

- Garder **votre propre pile en marche** (Docker et frontend) : elle sert de **plan B** si l'installation de l'encadrant n'est pas terminée.
- Avoir sous la main : le lien du dépôt, le README en PDF, les guides en PDF, le journal et les documents de passation du projet, l'abonnement Claude.
- Une connexion Internet stable.

2.5 Plan B

Problème	Solution
L'installation de l'encadrant n'est pas terminée	Faire la partie 1 sur votre machine (partage d'écran), et laisser l'encadrant terminer son installation pendant les parties 2 et 3
Pas d'abonnement Claude disponible pour l'encadrant	Faire la partie 3 sur votre compte (partage d'écran), l'encadrant relit et dirige

Problème	Solution
Claude Code est lent ou saturé (limite d'usage atteinte)	Vérifier votre consommation (guide 1, section 4.3) ; se limiter au plan et au module M2

3. Partie 1 : vérifier que le projet fonctionne depuis Git

Objectif : l'encadrant constate que ce qui est sur GitHub se lance et se teste.

3.1 Déroulé

Étape	Action	Référence
1	Cloner le projet	README, section 4.2
2	Vérifier la copie : 16 fichiers de modèle	README, section 4.3
3	Vérifier l'état du backend : <code>docker compose ps</code> , les logs, <code>/health</code>	README, section 5.3
4	Premier lancement (administrateur et données de démonstration)	README, section 5.4
5	Vérifier le frontend	README, section 6.4
6	Parcours de test : connexion, création d'un agent, scoring de HC-100001, dossier en revue avec HC-100241, journal d'audit	README, section 7

Comptes de test. `admin`, `agent.test` et `superviseur.test`, tous les trois avec le mot de passe d'exemple `Demo12345` (README, sections 5.4 et 7). Ce mot de passe est sans risque : il ne sert que sur la machine de test.

3.2 Ce que l'encadrant doit constater

Contrôle	Réussi si...
Récupération Backend	Le <code>git clone</code> réussit ; <code>ls scoring-backend/artefacts</code>
Frontend	4 conteneurs Up, l'API <code>healthy</code> ; <code>/health</code> : <code>"statut": "ok"</code> , 27 variables, 27 phrases, <code>"mock_mode": false</code>
Scoring	La page de connexion s'affiche ; les trois espaces (Agent, Superviseur, Administrateur) sont accessibles
Traçabilité	HC-100001 : score 603, ACCORDÉ ; HC-100241 : score 571, EN REVUE ; HC-101327 : score 536, REFUSÉ
	Le journal d'audit montre connexions, création de compte, scorings et décision du superviseur

3.3 Si quelque chose ne marche pas

Ne pas improviser : ouvrir la **section 10 du README** (dépannage), qui liste les symptômes et les solutions vérifiées. Les cas les plus probables sont un port déjà utilisé, un manque d'espace disque, et l'oubli de redémarrer l'API après le chargement des données.

4. Partie 2 : comprendre comment le projet a été conçu

Objectif : l'encadrant comprend la **méthode**, pas seulement le résultat. On s'appuie sur le **guide 2**.

4.1 Les cinq messages à faire passer

N°	Message	Preuve à montrer
1	On conçoit avant de coder , et on écrit tout en Markdown	Le cahier des charges et la conception ; le README lui-même
2	Claude propose, l'humain décide : aucun livrable sans accord explicite	Les instructions du projet (section 5, workflow de collaboration)
3	Le projet tient sur des sessions , grâce à des documents : instructions, journal, passations, registre des décisions	Le prompt de démarrage de session, le rapport de passation, le journal
4	Chaque choix est justifié et classé par niveau de preuve	L'exemple du WoE (guide 2, section 8)
5	Claude se trompe ; la méthode permet de le voir	Le tableau des erreurs réelles (guide 2, section 14)

4.2 Les fichiers à ouvrir pendant la présentation

- les **instructions du projet** Claude (six sections) ;
- un **prompt de démarrage de session** et un **rapport de passation** ;
- le **journal de bord** et le **journal de rédaction** (règles actées, pièges) ;
- le **registre des décisions méthodologiques** ;
- le fichier `scanner.sh` et l'instantané de contexte qu'il produit.

4.3 La démonstration des outils

Montrer, dans l'ordre : le **Projet** claude.ai (instructions, base de connaissances) ; la page **Utilisation** (limites de session) ; **Claude Code** dans VS Code et son **mode Plan** (guide 1).

5. Partie 3 : tester la méthode sur le MVP

Objectif : montrer que la méthode **fonctionne sur un cas qu'on ne connaît pas**. L'encadrant regarde, ou fait lui-même, ce qui suit. On s'appuie sur le **guide 3**.

5.1 Protocole pas à pas

Étape	Action	Durée indicative
1	Créer un dossier vide <code>demo_mvp</code> , l'ouvrir dans VS Code	1 min
2	Y copier <code>03_conception_MVP.md</code> sous <code>docs/conception_mvp.md</code>	1 min
3	Ouvrir Claude Code, passer en mode Plan (indicateur de mode)	1 min
4	Donner le prompt de l'annexe B du guide 3	1 min
5	Lire le plan proposé et le juger avec la grille ci-dessous	5 à 10 min
6	Corriger si besoin (commentaires dans le plan), puis approuver	5 min
7	Demander la réalisation du module M2 (données et modèle)	5 à 10 min
8	Vérifier le résultat : l'AUC de test et la reproductibilité	3 min

5.2 Grille d'évaluation du plan (étape 5)

Question	Oui / Non
Le plan couvre-t-il les six modules (socle, données et modèle, service de scoring, sécurité, API, interface) ?	
Les modules sont-ils ordonnés selon leurs dépendances (M1, puis M2, puis M3...)?	
Chaque module a-t-il ses fichiers , ses tests et son critère de fin ?	
Le plan reprend-il les exigences (EF et ENF) du document, sans en inventer ?	
Signale-t-il des ambiguïtés du document ? (un bon signe)	
Ne crée-t-il aucun fichier à ce stade (mode Plan) ?	

5.3 Critères de succès du module M2

Vérification	Attendu
Le script de génération produit 3 000 clients	Fichier <code>clients_synthetiques.csv</code> de 3 000 lignes
L'AUC de test affichée	Au moins 0,70 (environ 0,75 avec les données de l'annexe A)
Deux exécutions successives	Mêmes coefficients à la troisième décimale
Les tests du module	Ils passent

5.4 Si le plan ou le module n'est pas satisfaisant

C'est **normal et instructif** : c'est ce que la méthode est censée révéler.

1. **Identifier ce qui manque** dans le plan (un module oublié, un ordre incohérent).
2. **Le dire à Claude** en mode Plan (« il manque ... », « M3 dépend de M2 »), ou par commentaire dans le plan.

3. Si le document lui-même est ambigu, **le corriger** : la conception est la source de vérité.
4. **Recommencer le plan**, puis approuver.

5.5 Ce que la séance ne cherche pas à prouver

Que Claude écrit un système parfait en une passe. Elle cherche à prouver que **la méthode (conception écrite, mode Plan, un module à la fois, tests)** donne un travail **vérifiable et repreneable**.

6. Questions probables et éléments de réponse

Question de l'encadrant	Éléments de réponse
<i>Pourquoi concevoir avant de coder ?</i>	Une conception écrite est relisible et corrigeable à coût presque nul ; du code écrit sans conception cache ses erreurs. La conception est aussi la source de vérité que Claude Code lit.
<i>Comment savez-vous que Claude ne s'est pas trompé ?</i>	On ne le croit pas sur parole : chiffres vérifiés contre l'exécution réelle, tests, relecture, registre des décisions. Des erreurs réelles ont été trouvées ainsi (guide 2, section 14).
<i>Que vaut la littérature apportée par Claude ?</i>	Elle est un point de départ . Chaque référence importante est vérifiée, et les incertaines restent marquées « à vérifier ».
<i>Peut-on refaire la démarche sur un autre sujet ?</i>	Oui : le guide 2 donne les prompts, les modèles de documents et le cycle de session. Le MVP en est un exemple complet.
<i>Combien cela coûte-t-il ?</i>	Un abonnement Pro à 20 \$ par mois suffit pour commencer ; Max (100 ou 200 \$) si l'usage de Claude Code est intensif (guide 1, section 3). Prix vérifiés le 19 septembre 2026.
<i>Et la confidentialité des données ?</i>	CREDIX a été conçu sur un jeu de données public (Home Credit), sans données de clients réels. Ne jamais confier de données réelles ni de secrets à un outil externe sans autorisation.
<i>Que fait CREDIX que le MVP ne fait pas ?</i>	Deux flux (score et détection des profils atypiques), WoE, calibration, indice de couverture ρ_c , trois rôles, base documentaire, PDF (guide 3, section 13).
<i>Peut-on modifier le projet ?</i>	Oui : README, section 9 (Git), et méthode du guide 2, section 13 (toujours en mode Plan).

7. Liste de contrôle de la séance

La veille

- ☐ L'encadrant est **invité** sur le dépôt et a accepté l'invitation
- ☐ Il a créé son **jeton d'accès**

- ☐ Sa **première installation** est lancée, ou terminée (`docker compose ps` : 4 conteneurs, API healthy)
- ☐ VS Code, l'extension Claude Code et l'abonnement sont prêts
- ☐ Votre propre pile fonctionne (plan B)
- ☐ Espace disque suffisant (au moins 10 Go libres) des deux côtés

Pendant la séance

- ☐ Partie 1 : `git clone`, `/health`, connexion, un scoring
- ☐ Partie 2 : les cinq messages, les vrais documents du projet
- ☐ Partie 3 : plan en mode Plan, grille d'évaluation, module M2

Après la séance

- ☐ Noter les remarques et corrections de l'encadrant
- ☐ Mettre à jour les documents concernés (le `.md` est la source ; le `.tex` et le PDF se régénèrent avec `docs/build/build_all.sh`)
- ☐ Enregistrer et envoyer les modifications avec Git (README, section 9)